

Contents

全书常用符号说明	9
1 世界模型基本概念与历史脉络	10
1.1 问题 1: 什么是世界模型? 它和普通预测模型有什么区别?	10
1.2 问题 2: 世界模型为什么被认为是通向 Physical AI 和 AGI 的关键?	11
1.3 问题 3: Ha 和 Schmidhuber 的 World Models 提出了什么核心框架?	11
1.4 问题 4: V、M、C 架构分别表示什么?	12
1.5 问题 5: 世界模型和环境模拟器有什么区别?	13
1.6 问题 6: 世界模型和视频生成模型有什么区别?	14
1.7 问题 7: 世界模型和大语言模型有什么区别?	14
1.8 问题 8: 世界模型和多模态大模型有什么关系?	15
1.9 问题 9: 为什么世界模型必须建模动作, 而不仅仅是建模观察?	16
1.10 问题 10: 为什么“预测未来”不等于“理解世界”?	17
2 状态、动作、观测与环境动态	18
2.1 问题 11: MDP 和 POMDP 在世界模型中分别对应什么?	18
2.2 问题 12: 为什么真实世界更接近 POMDP?	18
2.3 问题 13: state、observation、action、reward 在世界模型中分别是什么?	19
2.4 问题 14: 为什么世界模型通常需要 latent state?	20
2.5 问题 15: transition model 为什么是世界模型的核心?	20
2.6 问题 16: observation model 和 decoder 有什么关系?	21
2.7 问题 17: reward model 为什么对 agent 世界模型很重要?	22
2.8 问题 18: termination model 在强化学习 world model 中起什么作用?	23
2.9 问题 19: deterministic dynamics 和 stochastic dynamics 有什么区别?	23
2.10 问题 20: uncertainty modeling 为什么是世界模型的关键能力?	24
3 模型式强化学习与 Dreamer 系列	25
3.1 问题 21: 什么是 Model-Based Reinforcement Learning?	25
3.2 问题 22: model-based RL 和 model-free RL 有什么区别?	26
3.3 问题 23: 世界模型如何提升 sample efficiency?	27
3.4 问题 24: 什么是 imagination rollout?	27
3.5 问题 25: Dreamer 为什么要在 latent space 里 rollout?	28
3.6 问题 26: RSSM 为什么适合部分可观测环境?	28
3.7 问题 27: Dreamer 的 reconstruction loss、reward loss、KL loss 分别起什么作用?	29
3.8 问题 28: actor、critic 和 world model 如何联合训练?	30
3.9 问题 29: imagination horizon 太长为什么会带来 model bias?	30
3.10 问题 30: model bias 和 compounding error 有什么区别?	31

4	JEPA 基础、I-JEPA、V-JEPA 与 V-JEPA 2	32
4.1	问题 31: 什么是 JEPA? 为什么它不是普通 autoencoder?	32
4.2	问题 32: I-JEPA 为什么预测 embedding 而不是重建 pixel?	32
4.3	问题 33: JEPA 为什么被 LeCun 认为是世界模型路线?	33
4.4	问题 34: V-JEPA 如何把 JEPA 从图像扩展到视频?	34
4.5	问题 35: V-JEPA 2 为什么被称为 self-supervised foundation world model?	35
4.6	问题 36: V-JEPA 2 如何结合互联网视频和少量 robot trajectories?	35
4.7	问题 37: V-JEPA 2 如何实现 zero-shot robot control?	36
4.8	问题 38: JEPA 路线和生成式 world model 路线有什么根本区别?	36
4.9	问题 39: JEPA 的优势是抽象预测, 代价是什么?	37
4.10	问题 40: JEPA 能否单独构成完整 world model?	38
5	V-JEPA 2.1、LeWorldModel 与 Causal-JEPA	39
5.1	问题 41: V-JEPA 2.1 相比 V-JEPA 2 的关键升级是什么?	39
5.2	问题 42: Dense Predictive Loss 为什么能增强 spatial-temporal grounding?	39
5.3	问题 43: Deep Self-Supervision 为什么有助于提升中间层表示?	40
5.4	问题 44: Multi-Modal Tokenizers 为什么对 image-video unified training 很重要?	40
5.5	问题 45: LeWorldModel 的核心思想是什么?	41
5.6	问题 46: LeWorldModel 为什么要解决 JEPA 的端到端训练稳定性?	42
5.7	问题 47: Gaussian latent regularizer 在 LeWorldModel 中起什么作用?	42
5.8	问题 48: Causal-JEPA 为什么强调 object-level masking?	43
5.9	问题 49: Causal-JEPA 如何区分 correlation 和 causation?	44
5.10	问题 50: 为什么 object-level intervention 对可控世界模型重要?	44
6	生成式世界模型、Genie 与交互式环境	45
6.1	问题 51: 视频生成模型一定是世界模型吗?	45
6.2	问题 52: Sora、Veo、Genie 和普通 text-to-video 模型有什么区别?	46
6.3	问题 53: 什么是 interactive world generation?	46
6.4	问题 54: Genie 2 到 Genie 3 的核心能力变化是什么?	47
6.5	问题 55: Genie 3 为什么被称为 general-purpose world model?	47
6.6	问题 56: Project Genie 为什么说明 world model 正在走向用户可交互产品?	48
6.7	问题 57: real-time exploration 为什么比 long video generation 更接近 world model?	49
6.8	问题 58: persistent state 为什么是 interactive world 的核心?	49
6.9	问题 59: object permanence 为什么是世界模型能力的重要指标?	50
6.10	问题 60: 如何判断一个 video generator 是否真正具备 world model 能力?	50
7	World Foundation Model、Cosmos 与 Physical AI	51
7.1	问题 61: 什么是 World Foundation Model?	51

7.2	问题 62: World Foundation Model 和传统 task-specific world model 有什么区别?	52
7.3	问题 63: 为什么 Physical AI 需要 world foundation model?	52
7.4	问题 64: NVIDIA Cosmos 为什么是 world foundation model platform?	53
7.5	问题 65: Cosmos Predict 如何统一 Text2World、Image2World、Video2World?	54
7.6	问题 66: Cosmos-Predict2.5 为什么结合 Cosmos-Reason1?	54
7.7	问题 67: Physical AI Data Factory 如何生成 long-tail synthetic data?	55
7.8	问题 68: 为什么 world model 需要 guardrails 和 evaluator?	55
7.9	问题 69: closed-loop policy training 为什么需要 world simulation?	56
7.10	问题 70: World Foundation Model 的核心评估指标是什么?	57
8	自动驾驶世界模型	57
8.1	问题 71: 为什么自动驾驶需要世界模型?	57
8.2	问题 72: 自动驾驶世界模型和普通感知模型有什么区别?	58
8.3	问题 73: future physical world generation 在自动驾驶中解决什么问题?	59
8.4	问题 74: image-based、BEV-based、occupancy-based、point-cloud-based world model 各有什么优缺点?	60
8.5	问题 75: 4D occupancy forecasting 为什么重要?	60
8.6	问题 76: GAIA-2 为什么被称为 controllable multi-view generative world model?	61
8.7	问题 77: Waymo World Model 如何用于 rare edge-case simulation?	62
8.8	问题 78: driving action control、scene layout control、language control 分别是什么?	62
8.9	问题 79: 为什么自动驾驶 world model 必须支持 counterfactual simulation?	63
8.10	问题 80: 如何评估自动驾驶 world model 的安全性和可控性?	64
9	VLA Driving World Model	64
9.1	问题 81: VLA 为什么需要 world model?	64
9.2	问题 82: DriveWorld-VLA 为什么要统一 world modeling 和 planning?	65
9.3	问题 83: latent-space imagination 为什么比 pixel-space rollout 更适合驾驶规划?	66
9.4	问题 84: DriveWorld-VLA 如何让 planner 使用 world model latent state?	66
9.5	问题 85: DriveVLA-W0 如何用 future image prediction 缓解 supervision deficit?	67
9.6	问题 86: VLA-World 为什么结合 predictive imagination 和 reflective reasoning?	67
9.7	问题 87: 为什么“生成未来帧后再反思轨迹”有助于自动驾驶解释性?	68
9.8	问题 88: Vega 这类 Vision-Language-World-Action 模型为什么结合 AR 和 diffusion 范式?	69
9.9	问题 89: VLA driving world model 如何处理语言指令、世界状态和轨迹生成?	69
9.10	问题 90: 自动驾驶 VLA world model 的最大风险是什么?	70
10	机器人世界模型与 VLA-JEPA	70
10.1	问题 91: 机器人世界模型和自动驾驶世界模型有什么不同?	71
10.2	问题 92: 为什么机器人任务更依赖 contact dynamics?	71

10.3 问题 93: manipulation task 为什么比 navigation task 更难建模?	72
10.4 问题 94: VLA-JEPA 为什么要学习 action-relevant state transition?	73
10.5 问题 95: VLA-JEPA 如何避免 pixel variation 和 nuisance motion?	73
10.6 问题 96: learned simulator 如何用于 robot policy post-training?	74
10.7 问题 97: 什么是 control-consistent future?	75
10.8 问题 98: 为什么 plausible future 不等于 useful future?	75
10.9 问题 99: world model 和 robot policy 应该分开训练还是联合训练?	76
10.10 问题 100: 世界模型如何支持 sim-to-real transfer?	76
11 空间智能、3D 世界与可交互环境	77
11.1 问题 101: 什么是 Spatial Intelligence? 它和 world model 有什么关系?	77
11.2 问题 102: 为什么 2D image understanding 不足以支持世界模型?	78
11.3 问题 103: 3D geometry 在 world model 中起什么作用?	78
11.4 问题 104: NeRF、3D Gaussian Splatting、occupancy、mesh 在世界模型中各有 什么优缺点?	79
11.5 问题 105: 可交互 3D 环境和普通 3D asset generation 有什么区别?	80
11.6 问题 106: object permanence 如何在 3D world model 中体现?	80
11.7 问题 107: affordance understanding 为什么是空间智能的核心?	81
11.8 问题 108: 生成式 3D 环境如何用于 embodied agent 训练?	81
11.9 问题 109: spatial memory 为什么比视觉短期一致性更难?	82
11.10 问题 110: spatial intelligence 和 language intelligence 的根本区别是什么?	82
12 Agentic World Modeling、评估与未来趋势	83
12.1 问题 111: 什么是 Agentic World Modeling?	83
12.2 问题 112: L1 Predictor、L2 Simulator、L3 Evolver 分别代表什么能力层级? . . .	84
12.3 问题 113: world model 中的 law regime 是什么?	84
12.4 问题 114: 什么是 action-conditioned imagination?	85
12.5 问题 115: 什么是 counterfactual simulation?	85
12.6 问题 116: world model 如何支持 long-horizon planning?	86
12.7 问题 117: 如何评估 physical consistency?	87
12.8 问题 118: 如何评估 causal consistency?	87
12.9 问题 119: 如何评估 action controllability?	88
12.10 问题 120: 世界模型会不会成为下一代 Agent 的核心基础设施?	89

写作说明

本题集聚焦世界模型方向，从早期模型式强化学习中的 latent dynamics，到 JEPA 路线、交互式世界生成、World Foundation Model、自动驾驶 VLA world model、机器人 learned simulator、空间智能和 Agentic World Modeling。全书不是把世界模型当成视频生成的附录，而是把它定义为连接感知、预测、行动、规划和反事实模拟的核心智能基础设施。

本书每道题统一采用三段式写法：**通俗解释**用于面试表达，**数学推导**用于建立公式化理解，**总结**用于形成可复述的回答。本次逐题精修版不再使用章节级通用模板，而是对每一题分别明确：它问的到底是哪一个概念边界、应该从哪一个公式切入、面试中最容易答浅或答偏的位置在哪里。贯穿全书的 30 个新概念包括：World Foundation Model、Physical AI、V-JEPA 2.1、Dense Predictive Loss、Deep Self-Supervision、Multi-Modal Tokenizers、LeWorldModel、Gaussian Latent Regularizer、Causal-JEPA、Object-Level Latent Intervention、VLA-JEPA、Action-Relevant State Transition、DriveWorld-VLA、Latent-Space Imagination、VLA-World、Predictive Imagination、Reflective Reasoning、DriveVLA-W0、Supervision Deficit、Genie 3、Interactive World Generation、Waymo World Model、Cosmos-Predict2.5、Cosmos-Reason1、Physical AI Data Factory、Control-Consistent Future、Learned Simulator、Agentic World Modeling、Counterfactual Simulation、L1 Predictor / L2 Simulator / L3 Evolver。

美国的牛蒡博士

全书常用符号说明

下面的符号表用于降低数学部分的阅读门槛。表中解释全书反复出现的公共符号；每道题的数学推导后面还会补充该题中特有符号的含义。

符号	含义
t	时间步，表示序列或交互过程中的第 t 个时刻。
H	规划或 rollout 的时间长度，也就是从当前时刻向未来看多少步。
T	一段轨迹、视频或 episode 的总长度。
o_t	第 t 时刻的观测，例如图像、视频帧、传感器读数或多模态输入。
$o_{\leq t}$	从开始到第 t 时刻为止的历史观测。
s_t	环境真实状态，通常包含位置、速度、物体关系、任务阶段等完整信息；真实系统中往往不可完全观测。
z_t	模型学习到的 latent state，用来近似智能体决策所需的内部状态或 belief state。
b_t	belief state，即在部分可观测场景中，根据历史观测和动作推断出的状态分布。
a_t	第 t 时刻智能体执行的动作，例如转向、刹车、机械臂控制量或工具调用。
$a_{t:t+H}$	从当前到未来 H 步的动作序列。
r_t	第 t 时刻的 reward 或任务反馈，用于衡量状态和动作是否有利于目标完成。
d_t	episode 是否终止的指示变量，常用于表示碰撞、失败、成功完成或任务结束。
τ	trajectory，表示一整段状态、动作、观测和奖励组成的交互轨迹。
π	策略函数，根据当前状态或历史信息输出动作分布或具体动作。
$p_{\theta}(\cdot)$	由参数 θ 表示的概率模型，例如 transition model、observation model 或生成模型。
$f_{\phi}, g_{\theta}, F_{\theta}$	神经网络函数。下标通常表示该函数的可学习参数。
$\mathbb{E}[\cdot]$	期望，表示对未来不确定性、策略采样或模型分布求平均。
γ	折扣因子，用于降低远期 reward 在当前决策中的权重。
$J(\pi)$	策略 π 的优化目标，通常是期望累计回报。
$\mathcal{L}$	损失函数，训练模型时需要最小化的目标。
λ, β, μ	权重系数，用于平衡不同损失项、风险项或正则项的重要性。
$\text{sg}(\cdot)$	stop-gradient 操作，前向计算保留数值，反向传播时阻断梯度。
$\hat{z}, \hat{o}, \hat{r}$	模型预测出来的 latent、观测或 reward；帽子符号表示预测值而非真实值。
$C(\cdot)$	代价、约束或风险函数，例如碰撞代价、路线偏离代价或控制不平滑代价。
$\arg \max, \arg \min$	选择使目标最大或最小的变量，常用于动作搜索和轨迹优化。
$\mathcal{D}$	数据集，通常由真实轨迹、观测、动作和反馈组成。
$\mathcal{W}_{\theta}$	World Foundation Model 或世界模型算子，表示可生成、预测或评估世界状态的统一模型。

1 世界模型基本概念与历史脉络

本章主线 本章答题主线：先把世界模型定义成“动作条件下的内部环境”，再区分它和预测模型、视频模型、语言模型、多模态模型的边界。

1.1 问题 1：什么是世界模型？它和普通预测模型有什么区别？

1.1.1 通俗解释

世界模型不是简单的“下一帧预测器”，而是一个可以被智能体反复查询的内部环境模型。普通预测模型只关心“未来观测大概长什么样”，世界模型还要回答三件事：如果我采取某个动作，(1) 隐藏状态会怎样变化，(2) 奖励或风险会怎样变化，(3) 未来是否仍然满足物理和任务约束。换句话说，普通预测模型是被动观察者，世界模型是可被动作驱动的模拟器。

它和几个相邻概念的边界值得讲清楚。和**视频生成模型**相比，世界模型必须支持动作条件，无论刹车还是加速生成的未来要不同；和**感知模型**相比，它必须建模时间动态，而不只是当前帧理解；和**规则式模拟器**相比，它从数据中学习而非被人工编写；和 **LLM** 相比，它建模连续时空状态而非离散 token。这些边界共同决定了一个模型是否真的是 world model，而不只是看名字像。

1.1.2 数学推导

普通时序预测模型只对历史观测建模：

$$p_{\theta}(o_{t+1} | o_{\leq t}).$$

世界模型必须显式引入动作和隐状态，并同时预测奖励：

$$p_{\theta}(z_{t+1}, o_{t+1}, r_t | z_t, a_t).$$

这样做的目的是让模型可以服务规划——通过在模型上搜索最优动作序列：

$$a_{t:t+H}^* = \arg \max_{a_{t:t+H}} \mathbb{E}_{p_{\theta}} \left[\sum_{k=0}^H \gamma^k r_{t+k} \right].$$

要 quantitatively 验证一个模型是否“真正”是世界模型，可以检查动作敏感性：

$$D_{\text{act}}(a, a') = d(p_{\theta}(z_{t+1} | z_t, \text{do}(a)), p_{\theta}(z_{t+1} | z_t, \text{do}(a'))),$$

当 $D_{\text{act}} \rightarrow 0$ 时，模型对动作不敏感，本质上仍是普通预测模型；只有 D_{act} 与真实环境一致时，它才能驱动决策。

本题符号说明 其中， $o_{\leq t}$ 表示历史观测； o_{t+1} 表示下一帧观测； z_t, z_{t+1} 表示 latent state； $a_t, a_{t:t+H}$ 表示动作或动作序列； r_t 表示奖励； p_{θ} 表示学习到的概率模型； $\text{do}(\cdot)$ 表示因果干预； D_{act} 表示动作敏感度； $d(\cdot, \cdot)$ 表示分布距离； γ 是折扣因子； H 表示规划 horizon。

1.1.3 总结

世界模型的本质不是“把未来画出来”，而是学习动作如何改变内部状态并支持反事实查询。普通预测模型回答“接下来像什么”，世界模型回答“如果这样做世界会怎样变”——这一从观察到干预的提升，正是它能服务规划、反事实推理和决策的根本原因，也是 Physical AI 系统离不开它的根本理由。

1.2 问题 2：世界模型为什么被认为是通向 Physical AI 和 AGI 的关键？

1.2.1 通俗解释

Physical AI 面对的不是静态数据集，而是会被动作改变的真实环境。模型如果只会识别图像或生成文本，仍然不知道自己动手以后会发生什么。世界模型把感知、动作、未来和反馈连起来，使智能体可以先在内部模拟，再选择更安全、更有效的动作。Physical AI 的关键不是把现实描述成文本，而是在行动前评估不同动作的后果，世界模型正好提供这种内部试错空间。

1.2.2 数学推导

给定策略 $\pi(a_t | z_t)$ ，世界模型诱导闭环轨迹分布：

$$p_\theta(\tau) = p(z_0) \prod_{t=0}^T \pi(a_t | z_t) p_\theta(z_{t+1}, r_t | z_t, a_t).$$

智能体优化的是内部世界中的期望回报：

$$J(\pi) = \mathbb{E}_{\tau \sim p_\theta} \left[\sum_{t=0}^T \gamma^t r_t \right].$$

本题符号说明 其中， z_t 表示模型内部 latent state； a_t 表示当前动作或控制输入； r_t 表示 reward 或任务反馈； p_θ 表示参数为 θ 的学习式概率模型； π 表示策略函数； J 表示优化目标； $\mathbb{E}$ 表示期望。

1.2.3 总结

Physical AI 需要在真实环境中行动，AGI 需要在行动前预估后果。世界模型提供的正是这种内部试错能力：先模拟不同选择的未来，再把高风险、低收益或不符合物理约束的动作排除掉。

1.3 问题 3：Ha 和 Schmidhuber 的 World Models 提出了什么核心框架？

1.3.1 通俗解释

Ha 和 Schmidhuber 的 World Models 重要性不在于模型规模，而在于把感知、动态和控制拆成清晰的三部分：先把画面压缩成 latent，再用循环模型预测 latent 动态，最后用 controller 在内部状态上行动。这奠定了后续 latent world model 的基本范式。V-M-C 框架的价值在于把“看见、记住、控制”三件事分开，后续方法无论模型多大，仍然绕不开这三个模块。

1.3.2 数学推导

视觉压缩为：

$$z_t = E_\phi(o_t).$$

动态模型为：

$$h_t = f_\theta(h_{t-1}, z_t, a_{t-1}), \quad p_\theta(z_{t+1} | h_t, a_t).$$

控制器输出动作：

$$a_t = C_\psi(z_t, h_t).$$

本题符号说明 其中, o_t 表示当前观测; z_t 表示模型内部 latent state; a_t 表示当前动作或控制输入; p_θ 表示参数为 θ 的学习式概率模型; h_t 表示隐藏状态、历史记忆或接触状态。

1.3.3 总结

Ha 和 Schmidhuber 的贡献是把世界模型拆成视觉压缩、动态记忆和控制器三个可训练模块。这个框架证明了智能体不一定要直接在像素空间行动, 而可以在压缩后的 latent world 中学习控制。

1.4 问题 4: V、M、C 架构分别表示什么?

1.4.1 通俗解释

V (Vision)、M (Memory/Model)、C (Controller) 是 Ha & Schmidhuber 在 2018 年的 World Models 提出的三模块拆分: V 把高维观测压缩成 latent, M 根据历史和动作预测未来, C 根据内部状态输出动作。这个结构把”智能体在环境中行动”还原成三个能力: 看见 $\rightarrow$ 想象 $\rightarrow$ 行动, 而且每个模块训练目标清晰可分。

它的工程价值远超架构提议本身——后续几乎所有 latent world model (Dreamer 系列、PlaNet、TD-MPC、Genie) 都能映射到这个三段式: encoder = V, dynamics + RNN/transformer = M, policy/MPC = C。三者还可以分阶段训练: V 用自监督表示学习预训, M 在 V 提供的 latent 上学动态, C 在 M 提供的 imagination 上学策略。这种分阶段策略避免了端到端训练的不稳定 (尤其是像素重建梯度淹没策略梯度的问题), 是该框架至今仍被广泛复用的关键。

1.4.2 数学推导

V 模块完成观测压缩:

$$z_t = V_\phi(o_t).$$

M 模块包括确定性记忆更新和动作条件状态预测两部分:

$$m_t = M_\theta(m_{t-1}, z_{t-1}, a_{t-1}), \quad \hat{z}_{t+1} \sim p_\theta(z_{t+1} \mid m_t, a_t).$$

这里 m_t 可以看作 belief state 的神经近似, 把整段历史压缩成可递推的固定维度向量。C 模块根据 latent 与记忆输出动作:

$$a_t = C_\psi(z_t, m_t).$$

在原始论文中, V 用 VAE 训练 (最大化 $\log p_\psi(o_t \mid z_t) - \beta D_{\text{KL}}$), M 用 mixture density network 预测下一步 latent, C 用进化策略 (CMA-ES) 在 imagined rollout 上优化回报:

$$\max_{\psi} J(C_\psi) = \mathbb{E}_{m, z \sim M \circ V} \left[\sum_t \gamma^t r_t \right].$$

整个系统的优雅之处在于 C 维度极小 (几千参数), 可以在内部 imagined trajectory 上反复评估, 而不需要昂贵的真实环境交互。

本题符号说明 其中, o_t 表示观测; z_t 表示 latent; m_t 表示历史记忆/belief; a_t 表示动作; V_ϕ 、 M_θ 、 C_ψ 分别为三个模块; p_θ 表示动态模型; $\hat{z}_{t+1}$ 表示预测 latent; D_{KL} 表示 KL 散度; β 表示 KL 权重; $J(C_\psi)$ 表示 controller 的优化目标; γ 是折扣因子。

1.4.3 总结

V、M、C 分别对应表示、动态、控制三类能力：V 把观察变成状态，M 预测状态如何随时间和动作变化，C 根据内部状态输出动作。这个三段式不仅是历史架构，更是后续 latent world model 的统一抽象——理解 V/M/C 就理解了”看见—想象—行动”如何在 latent 空间形成闭环。

1.5 问题 5：世界模型和环境模拟器有什么区别？

1.5.1 通俗解释

环境模拟器（CARLA、Isaac Sim、MuJoCo 等）通常由人工规则、物理引擎和资产库构建，优势是可解释、可复现、状态可观测、约束可显式编程，但开发成本高、覆盖范围有限，长尾场景（异常天气、罕见事故、复杂接触）很难枚举完全。世界模型从大规模真实数据中学习状态转移，优势是泛化能力强、可微（可以反传梯度到策略）、能覆盖训练分布中的长尾，但代价是缺乏物理保证、可能产生 hallucination、对分布外输入不可靠。

工程实践中，二者通常不是二选一，而是分工协作：物理模拟器提供刚体、碰撞、关节等”硬”约束作为骨架，世界模型在它之上补足真实世界中难以建模的细节（接触摩擦、行人行为、天气光照分布）。这也是 sim-to-real 和 world model + simulator 残差架构的常见思路。

1.5.2 数学推导

工程模拟器是确定性、可解析的状态转移函数：

$$s_{t+1} = F_{\text{sim}}(s_t, a_t; \xi),$$

其中 ξ 是物理参数（质量、摩擦系数等），可显式调节。学习式世界模型则是概率分布：

$$p_{\theta}(s_{t+1} | s_t, a_t),$$

从数据中估计动态。二者常以残差形式结合，让模拟器负责粗粒度物理，让神经网络负责剩余误差：

$$s_{t+1} = F_{\text{sim}}(s_t, a_t; \xi) + \Delta_{\theta}(s_t, a_t, o_t),$$

或者反过来用学习式模型做先验、用模拟器做事后约束投影：

$$s_{t+1}^{\text{final}} = \Pi_{C_{\text{phys}}}(p_{\theta}(s_{t+1} | s_t, a_t)),$$

其中 $\Pi_{C_{\text{phys}}}$ 表示把生成结果投影到物理约束集合 C_{phys} 上。

本题符号说明 其中， s_t 、 s_{t+1} 表示真实环境状态； a_t 表示动作； o_t 表示观测； F_{sim} 表示工程模拟器； ξ 表示物理参数； p_{θ} 表示学习式世界模型； Δ_{θ} 表示学习残差； C_{phys} 表示物理约束集合； Π 表示投影算子。

1.5.3 总结

模拟器通常是”人写出来的世界”，强调可解释和可复现；世界模型通常是”从数据学出来的世界”，强调灵活性和覆盖度。两者的根本权衡是 可控性与真实分布覆盖：前者可控但难穷举，后者覆盖但难保证。真实系统往往让模拟器提供物理边界、世界模型补足真实世界细节，二者残差组合或约束投影形成混合模拟器。

1.6 问题 6：世界模型和视频生成模型有什么区别？

1.6.1 通俗解释

两者目标不同：视频生成模型追求视觉连续和语义合理，世界模型追求动作后果可预测。视频生成模型可以根据文本生成一段画面看起来合理的视频，但它通常没有动作输入接口——给一段“汽车开过”的 prompt，模型生成什么完全由数据先验决定，无法在中途指令“刹车”让车停下。这种模型本质是条件视频采样器。

世界模型必须让未来随动作变化。如果一个模型无论 agent 选择刹车还是加速都生成几乎相同的未来，那它对决策没有信息量。差异可以从四个层面区分：(1) **接口**——是否接受动作 a 作为输入；(2) **敏感度**——不同 a 是否产生明显不同的 $p(x' | a)$ ；(3) **一致性**——同样动作在相似状态下应产生稳定结果；(4) **反事实**——同一初始状态下能否比较不同动作的未来。少了任何一条，模型至多是漂亮的视频续写器。

1.6.2 数学推导

普通视频生成的条件分布只依赖文本/图像条件 c ：

$$p_{\theta}(x_{1:T} | c),$$

整段视频一次性采样，动作 a 不出现。动作条件世界模型必须把动作显式作为转移条件：

$$p_{\theta}(x_{t+1:t+H} | x_{\leq t}, a_{t:t+H-1}, c).$$

量化”是否真是世界模型”的指标是动作敏感度——给定相同历史，不同动作应产生不同分布：

$$D_{\text{act}}(a, a') = d(p_{\theta}(x_{t+1:t+H} | x_{\leq t}, a), p_{\theta}(x_{t+1:t+H} | x_{\leq t}, a')).$$

当 $D_{\text{act}} \rightarrow 0$ ，模型实际上忽略了动作，退化为：

$$p_{\theta}(x_{t+1:t+H} | x_{\leq t}, a) \approx p_{\theta}(x_{t+1:t+H} | x_{\leq t}, a') \approx p_{\theta}(x_{t+1:t+H} | x_{\leq t}),$$

即与无动作的视频生成等价。这正是为什么很多”号称 world model”的视频模型在严格 benchmark 上不达标——动作敏感度太弱。

本题符号说明 其中， $x_{1:T}$ 表示视频序列； $x_{\leq t}$ 表示历史帧； $x_{t+1:t+H}$ 表示未来帧； $a_{t:t+H-1}$ 表示动作序列； a, a' 表示不同动作； c 表示文本/图像条件； p_{θ} 表示生成分布； D_{act} 表示动作敏感度； $d(\cdot, \cdot)$ 表示分布距离； T, H 表示视频长度与生成 horizon。

1.6.3 总结

视频生成模型追求视觉连续和语义合理，世界模型追求动作后果可预测。只会生成漂亮视频的模型不一定能用于决策；只有当未来会随动作、状态和约束发生一致变化、并能支撑反事实查询时，它才从”视频合成”进入”世界建模”的范畴。判别指标是动作敏感度——动作不能改变未来的模型，无论画质多高，都不是 world model。

1.7 问题 7：世界模型和大语言模型有什么区别？

1.7.1 通俗解释

LLM 建模 token 序列，擅长语言推理和工具调用；世界模型建模状态转移，擅长预测动作后果。LLM 可以告诉 agent 应该考虑什么，世界模型告诉 agent 这样做会发生什么，两者结合才更接近可行动智能。在这种组合中，LLM 更像高层 planner，world model 更像可查询的 dynamics engine。

1.7.2 数学推导

LLM 目标为：

$$p_{\theta}(w_t \mid w_{<t}).$$

世界模型目标为：

$$p_{\theta}(z_{t+1} \mid z_t, a_t).$$

层级组合可写成：

$$g_t = \text{LLM}(l, m), \quad a_t = \pi(g_t, z_t), \quad z_{t+1} \sim p_{\theta}(\cdot \mid z_t, a_t).$$

本题符号说明 其中， z_t 表示模型内部 latent state； a_t 表示当前动作或控制输入； p_{θ} 表示参数为 θ 的学习式概率模型； π 表示策略函数； l 表示语言指令或任务文本； m 表示 mask、模态条件或交互控制信号。

1.7.3 总结

LLM 建模语言 token 的条件分布，世界模型建模环境状态的条件转移。LLM 更适合做任务理解、计划分解和工具调度，世界模型更适合做物理后果预测；二者结合后，一个负责想任务，一个负责验后果。

1.8 问题 8：世界模型和多模态大模型有什么关系？

1.8.1 通俗解释

MLLM 让模型把图像、视频、语言对齐到统一语义空间，回答“这是什么、在做什么、和什么相关”——但它通常仍然偏向理解任务。世界模型在此基础上继续建模动作条件下的时间动态：动作如何改变物体、场景如何随时间演化、不同选择会导致什么不同后果。

VLA（Vision-Language-Action）正是这两条线的交叉点：它既要利用 MLLM 提供的语义 grounding（这是杯子、那是机械臂、任务是把杯子放进盒子），又要利用世界模型提供的动作后果预测（这样夹会滑、那样推会撞），最后把两者接到动作输出。三者的分工可以理解为：MLLM 做“现在”的语义理解，world model 做“未来”的动态预测，VLA 把两端接到具体可执行的动作上。

实际系统中三者经常深度融合：用 MLLM backbone 编码视觉和语言，用 latent dynamics 头预测未来 latent，再用 action head 输出控制信号——这正是 V-JEPA 2、VLA-JEPA、DriveVLA-W0 等系统的共同范式。

1.8.2 数学推导

MLLM 把多模态输入对齐到语义表示：

$$z_t = f_{\theta}(o_t, l),$$

其中 o_t 是图像/视频， l 是语言指令。世界模型在此之上引入动作条件下的状态转移：

$$p_{\phi}(z_{t+1} \mid z_t, a_t, l), \quad p_{\rho}(r_t \mid z_t, a_t, l).$$

VLA 把策略也建立在同一 latent 上：

$$a_t \sim \pi_{\psi}(z_t, l),$$

并通过 latent rollout 选择最优动作：

$$a_{t:t+H}^* = \arg \max_{a_{t:t+H}} \mathbb{E}_{p_\phi, \pi_\psi} \left[\sum_{k=0}^H \gamma^k r_{t+k} \mid l \right].$$

当语言条件 l 同时进入 encoder、dynamics 和 policy 时，整个系统就形成了 *language-grounded world model + policy*，可以用语言指定不同任务并复用同一套 dynamics。

本题符号说明 其中， o_t 表示视觉观测； l 表示语言指令； z_t, z_{t+1} 表示融合 latent； a_t 表示动作； r_t 表示奖励； f_θ 表示多模态 encoder； p_ϕ 表示世界动态； p_ρ 表示 reward model； π_ψ 表示策略； γ 是折扣因子； H 表示规划 horizon。

1.8.3 总结

多模态大模型解决的是跨模态语义对齐，世界模型进一步解决时间动态和动作后果。MLLM 让系统知道”这是什么”，world model 让系统知道”我做了这个以后会怎样”，VLA 则把两者接到具体的动作输出，形成”理解 → 想象 → 行动”的闭环。三者关系不是替代而是叠加：VLA 系统几乎都同时依赖 MLLM 的语义 grounding 和 world model 的动态预测。

1.9 问题 9：为什么世界模型必须建模动作，而不仅仅是建模观察？

1.9.1 通俗解释

没有动作，模型只能预测世界自然如何演化；有了动作，模型才能预测”我做了某件事以后会怎样”。自动驾驶中的刹车和变道、机器人中的推拉夹取，都要求模型能区分不同动作导致的不同未来。如果模型对所有候选动作都给出几乎相同的未来，那它对决策就没有信息量——它最多是一台时序生成器，而不是 world model。

更进一步，建模动作还引入了一个关键能力：反事实预测。智能体可以问”如果我换一个动作，未来会怎样”，从而在不真实执行高风险动作的前提下，比较不同候选并选出最优。从概率论角度看，没有动作条件就是观察分布 $p(o \mid o_{\leq t})$ ，加入动作后变成干预分布 $p(o \mid o_{\leq t}, \text{do}(a))$ ，两者的差异正是 Pearl 因果阶梯的第二层与第三层。

1.9.2 数学推导

无动作的时序预测只对历史观测建模：

$$p(o_{t+1} \mid o_{\leq t}).$$

这一形式只能描述世界”放任自流”时的演化。一旦引入动作 a_t ，模型变为干预分布：

$$p(o_{t+1} \mid o_{\leq t}, \text{do}(a_t)).$$

两个不同动作 a 与 a' 在同一历史下的反事实差异定义为：

$$\Delta(a, a') = \mathbb{E}[o_{t+1} \mid o_{\leq t}, \text{do}(a)] - \mathbb{E}[o_{t+1} \mid o_{\leq t}, \text{do}(a')].$$

如果模型对所有动作输出几乎相同的预测，即 $\Delta(a, a') \rightarrow 0$ ，则它对决策没有信息量；只有 Δ 与真实环境一致时，世界模型才能驱动 planning：

$$a^* = \arg \min_a C(o_{t+1}^a, a), \quad o_{t+1}^a \sim p_\theta(o_{t+1} \mid o_{\leq t}, \text{do}(a)).$$

本题符号说明 其中, $o_{\leq t}$ 表示历史观测; o_{t+1} 表示下一时刻观测; a_t 、 a 、 a' 表示动作; $\text{do}(\cdot)$ 表示因果干预算子; $\Delta(a, a')$ 表示反事实差异; C 表示决策代价; $\mathbb{E}$ 表示期望。

1.9.3 总结

动作是世界模型从”时序预测”走向”智能决策”的分界线。没有动作条件, 模型只能描述环境自然演化, 对应观察分布; 引入动作后, 模型变为干预分布, 可以比较刹车、加速、转向、抓取等不同动作带来的不同未来, 从而支撑 planning 与反事实推理。这是世界模型与普通视频预测器的本质差异。

1.10 问题 10: 为什么”预测未来”不等于”理解世界”?

1.10.1 通俗解释

很多模型可以靠统计相关性预测出像素级合理的短期画面, 但并不真正掌握物体永久性、接触关系、遮挡、碰撞和因果约束。一个典型的失败例子是: 模型在视频里能让球滚下斜坡看起来很自然, 但当我们从初始状态稍微移动一下斜坡的角度, 它输出的未来仍然是同一段记忆, 而不是按物理规律重新计算的轨迹——这说明它”会画”而不是”会算”。

世界理解的标准比预测要严苛得多: 未来不仅要看起来像, 还要在长 horizon、不同动作干预、不同物体配置、不同物理参数下都保持自治。具体可以从五个维度检验: (1) object permanence——遮挡后物体是否还存在; (2) physical consistency——是否违反守恒、碰撞、不可穿透; (3) action sensitivity——不同动作是否产生不同未来; (4) counterfactual consistency——干预某个变量后未来是否合理变化; (5) compositionality——熟悉物体的新组合是否仍然合理。仅靠像素相似度训练的模型, 往往在前几帧通过, 到第二条之后就会出问题。

1.10.2 数学推导

单纯的像素预测目标只衡量外观相似度:

$$\mathcal{L}_{\text{pixel}} = \mathbb{E} \|\hat{o}_{t+1} - o_{t+1}\|.$$

这个目标即使被完美最小化, 也不保证理解世界。理解世界要求模型同时满足一组结构约束:

$$C_m(z_t, a_t, z_{t+1}) \leq 0, \quad m = 1, \dots, M,$$

其中 C_m 包括碰撞约束、动力学约束、质量/能量守恒、对象身份保持、任务安全等。更强的判据是干预一致性: 对真实环境干预 $\text{do}(a)$ 与模型预测干预的差异要小,

$$\text{Err}_{\text{causal}} = \sum_a d(p_{\theta}(o' \mid \text{do}(a)), p_{\text{env}}(o' \mid \text{do}(a))).$$

当 $\mathcal{L}_{\text{pixel}} \rightarrow 0$ 但 $\text{Err}_{\text{causal}}$ 仍然较大时, 说明模型只学到了相关性而非世界结构。

本题符号说明 其中, o_{t+1} 、 $\hat{o}_{t+1}$ 表示真实和预测的下一帧; z_t 、 z_{t+1} 表示相邻 latent state; a_t 表示动作; C_m 表示第 m 个结构约束; M 表示约束总数; $\text{do}(a)$ 表示因果干预; $d(\cdot, \cdot)$ 表示分布距离; p_{env} 表示真实环境分布。

1.10.3 总结

”预测未来”只要求外观或统计趋势合理, ”理解世界”还要求模型抓住可干预、可迁移、可组合的因果结构。一个模型可能把下一帧画得很逼真, 却仍然不知道哪个物体可被推动、哪个动作会导致碰撞、改变摩擦系数会带来什么后果。世界模型的评估不能只盯像素相似度, 而要看物理一致性、动作可控性、反事实一致性这些更难造假的指标。

2 状态、动作、观测与环境动态

本章主线 本章答题主线：所有复杂世界模型都绕不开状态空间建模。先讲 MDP/POMDP，再讲 latent state、transition、observation、reward、termination 和 uncertainty。

2.1 问题 11：MDP 和 POMDP 在世界模型中分别对应什么？

2.1.1 通俗解释

MDP 和 POMDP 把世界模型放进状态空间框架中理解：智能体看到的是观测，真正需要推断的是隐藏状态和动作条件动态。MDP 假设状态完全可观测，是理想化抽象；POMDP 承认观测不完整，更接近真实机器人和自动驾驶。世界模型通常学习的是 belief state，而不是直接可见的真实 state。

2.1.2 数学推导

基本结构为：

$$z_t = f_\phi(o_{\leq t}, a_{< t}), \quad p_\theta(z_{t+1} | z_t, a_t).$$

本题符号说明 其中， $o_{\leq t}$ 表示历史观测； z_t 表示模型内部 latent state； a_t 表示当前动作或控制输入； p_θ 表示参数为 θ 的学习式概率模型。

2.1.3 总结

MDP 假设状态完全可观测，POMDP 承认智能体只能看到世界的一部分。世界模型中的 latent state 或 belief state，本质上就是为了解决 POMDP 中“看不全但还要行动”的问题。

2.2 问题 12：为什么真实世界更接近 POMDP？

2.2.1 通俗解释

真实世界几乎从来不可完全观测。这种“看不全”来自多个层面：(1) **物理遮挡**——前车挡住了行人、卡车挡住了路口；(2) **传感器限制**——相机有视野盲区、LiDAR 有距离上限、毫米波雷达分辨率有限；(3) **噪声与延迟**——任何传感器都有读数噪声和处理延迟；(4) **未观测变量**——其他车辆驾驶员的意图、行人下一步走向、物体内部状态、外部扰动（风、湿度）；(5) **部分可见的对象**——夜间逆光、雾天能见度低。

这些因素让 agent 看到的永远只是真实状态的一个投影，因此必须用历史观测序列推断“目前世界处于哪种可能状态”——这个分布就是 belief state。世界模型的 latent state z_t 实质上是 belief 的神经近似：它压缩历史信息，让 agent 即使看不全也能行动。这也是为什么 RSSM、Dreamer 等现代 world model 都把 history-aware encoder 作为标准组件，而不是单帧 encoder。

2.2.2 数学推导

POMDP 由 $(S, \mathcal{A}, \mathcal{O}, T, Z, R, \gamma)$ 定义，其中观测 o_t 由观测函数 $Z(o | s)$ 给出，对真实状态 s 仅是部分映射。Bayesian belief 更新为：

$$b_{t+1}(s') \propto Z(o_{t+1} | s') \sum_s T(s' | s, a_t) b_t(s).$$

精确求解需对所有 s 积分，在高维连续空间不可行。神经世界模型把 belief 用 latent 隐式近似：

$$z_t = f_\phi(o_{\leq t}, a_{< t}),$$

并用 RNN/transformer 让递推可学：

$$z_t = g_\phi(z_{t-1}, o_t, a_{t-1}).$$

性能差距由 belief approximation error 控制：

$$|J_{\text{POMDP}}(\pi) - J_{\text{model}}(\pi)| \leq \frac{R_{\max}}{1-\gamma} \mathbb{E}[\text{TV}(b_t, p(s | z_t))],$$

其中 TV 是 total variation。当 latent 维度过低或历史信息聚合不足时，这个差距会显著拉大——这是 POMDP 类任务里 world model 比单帧感知模型更难训练好的根本原因。

本题符号说明 其中, $\mathcal{S}, \mathcal{A}, \mathcal{O}$ 表示状态、动作、观测空间; T 表示状态转移; Z 表示观测函数; R 表示奖励; b_t 表示 belief state; $o_{\leq t}$ 表示历史观测; $a_{< t}$ 表示历史动作; z_t 表示 latent; f_ϕ 、 g_ϕ 表示 encoder 与 RNN 形式的递推; TV 表示 total variation; $R_{\max}$ 表示单步最大奖励; γ 是折扣因子。

2.2.3 总结

真实世界更接近 POMDP，因为传感器有遮挡、噪声、延迟、视野限制，agent 还无法观测他人意图与外部扰动。智能体看到的是 observation 而非完整 state，因此世界模型必须维护一个 history-aware 的 latent state 作为 belief 的神经近似——这一点直接决定了为什么单帧感知不够、为什么 RSSM 类 history-aware 架构是 world model 的标配。

2.3 问题 13: state、observation、action、reward 在世界模型中分别是什么？

2.3.1 通俗解释

state 是决策所需的充分信息，observation 是传感器看到的片段，action 是智能体施加的干预变量，reward 是任务反馈。世界模型要从 observation 反推出 state 的 belief，再预测 action 造成的 state transition；这是它与单纯感知模型的根本区别。

2.3.2 数学推导

四个变量关系为：

$$s_{t+1} \sim p(s_{t+1} | s_t, a_t), \quad o_t \sim p(o_t | s_t), \quad r_t \sim p(r_t | s_t, a_t).$$

latent 形式为：

$$z_t \approx p(s_t | o_{\leq t}, a_{< t}).$$

本题符号说明 其中, $o_{\leq t}$ 表示历史观测; o_t 表示当前观测; s_t 表示真实环境状态; z_t 表示模型内部 latent state; a_t 表示当前动作或控制输入; r_t 表示 reward 或任务反馈。

2.3.3 总结

state 是环境真实但通常不可完全观测的内部情况，observation 是传感器看到的证据，action 是智能体施加的干预，reward 是任务反馈。把这四者分清楚，才能把世界模型从视觉预测问题提升为决策建模问题。

2.4 问题 14：为什么世界模型通常需要 latent state?

2.4.1 通俗解释

直接在像素空间做预测既昂贵又脆弱：图像维度成千上万，单帧已经几十 KB，多步 rollout 计算量爆炸；而且像素中绝大部分信息（光照、纹理、背景细节）和决策无关，模型容量很容易被这些 nuisance 噪声占用。世界模型的解法是先把高维观测压缩到 latent state，让动态预测在紧凑、低维、决策相关的隐空间中进行。

更深层的原因是真实环境通常是 POMDP——单帧观测看不全完整状态。latent state 实际上承担了 belief state 的角色：它聚合历史观测和动作，逼近“在不完整观测下，对真实状态的可能分布”。一个好的 latent 应满足三条标准：充分性（包含预测未来所需信息）、紧凑性（去掉与控制无关的细节）、平滑性（相邻状态在 latent 中也相邻，便于 rollout）。

2.4.2 数学推导

latent state 由 encoder 从历史观测和动作推断出来：

$$z_t = f_\phi(o_{\leq t}, a_{< t}).$$

在这个 latent 上定义动态模型：

$$p_\theta(z_{t+1} | z_t, a_t), \quad p_\psi(o_t | z_t), \quad p_\rho(r_t | z_t, a_t).$$

“latent state 是充分统计量”可以形式化为条件独立性条件：

$$p(o_{t+1:T}, r_{t:T} | o_{\leq t}, a_{< t}, a_{t:T}) = p(o_{t+1:T}, r_{t:T} | z_t, a_{t:T}).$$

即给定 z_t 后，未来的观测和奖励对历史不再依赖。多步 rollout 在 latent 中递推，开销只与 $\dim(z)$ 相关：

$$\hat{z}_{t+k+1} \sim p_\theta(\cdot | \hat{z}_{t+k}, a_{t+k}),$$

比直接在像素空间 rollout 通常便宜 10^2 到 10^4 倍。

本题符号说明 其中， $o_{\leq t}$ 表示历史观测； $a_{< t}$ 表示历史动作； z_t 表示 latent state； f_ϕ 表示 encoder； p_θ 、 p_ψ 、 p_ρ 分别为 transition、observation、reward 模型； r_t 表示奖励； $\dim(z)$ 表示 latent 维度。

2.4.3 总结

latent state 的价值是把冗余观测压缩成对预测和控制有用的“充分统计量”。它不保留每个像素，但必须保留物体位置、运动趋势、任务目标和潜在风险这些会影响未来决策的变量。它同时承担三重角色：POMDP 中的 belief state、规划 rollout 的低成本载体、以及多种损失（reconstruction、reward、KL）共同优化的中介表示。

2.5 问题 15：transition model 为什么是世界模型的核心?

2.5.1 通俗解释

transition model 描述世界如何因动作而变化，是世界模型的“发动机”。感知模型告诉你现在有什么，observation model 告诉你 latent 长什么样，reward model 告诉你目标好不好——但这些都建立在同一个前提之上：你能预测下一步状态。没有 transition model，模型无法 rollout，无法做长 horizon 规划，也无法比较不同动作的后果。

transition model 的质量直接决定了世界模型整体的天花板：encoder 再好、decoder 再清晰，只要 transition 错了，rollout 几步就会偏到模型分布外，后续所有 reward、value、planning 都会失真。所以工程上很多 trick（KL 正则、target encoder EMA、stop-gradient、horizon truncation、uncertainty penalty）本质都是为了让 transition 更稳。

2.5.2 数学推导

single-step transition:

$$p_{\theta}(z_{t+1} \mid z_t, a_t).$$

多步 rollout 是单步转移按 Markov 性的连乘：

$$p_{\theta}(z_{t+1:t+H} \mid z_t, a_{t:t+H-1}) = \prod_{k=0}^{H-1} p_{\theta}(z_{t+k+1} \mid z_{t+k}, a_{t+k}).$$

在 actor-critic 中，transition model 与 policy 联合定义了 latent imagination 上的回报：

$$J(\pi) = \mathbb{E}_{p_{\theta}, \pi} \left[\sum_{k=0}^H \gamma^k r_{t+k} \right].$$

如果 transition 单步 KL 误差为 ϵ ，由 simulation lemma 可以得到长 horizon 上的真实环境性能差距上界：

$$|J_{\text{env}}(\pi) - J_{\theta}(\pi)| \leq \frac{2\gamma R_{\max}}{(1-\gamma)^2} \epsilon,$$

即 transition 误差会被 $1/(1-\gamma)^2$ 放大成累积误差。这就是为什么 transition 哪怕一点点不准，长 horizon planning 也会失败。

本题符号说明 其中， z_t 表示 latent state； a_t 表示动作； r_t 表示奖励； p_{θ} 表示 transition model； π 表示策略； $J(\pi)$ 表示策略回报； γ 是折扣因子； H 表示 rollout horizon； $R_{\max}$ 表示单步最大奖励； ϵ 表示 transition 单步 KL 误差。

2.5.3 总结

transition model 是世界模型的“心脏”，因为它直接决定 rollout 的可信度，而 rollout 又是 planning、actor-critic、反事实模拟的共同基础。encoder 决定看到什么，decoder 决定还原什么，reward 决定是否符合任务，但只有 transition 的质量决定模型能否被“用起来”。

2.6 问题 16：observation model 和 decoder 有什么关系？

2.6.1 通俗解释

observation model 把内部状态映射回可观测信号。它类似 decoder，但在世界模型里更强调“状态解释观测”，而不是单纯重建图像。它可以帮助训练 latent state，也可以用于可视化未来。需要注意的是：observation model 或 decoder 负责把 latent 映射回可观测空间，但控制任务未必需要高保真的像素重建——清晰的画面并不等于可靠的动态。

2.6.2 数学推导

观测模型为：

$$p_{\psi}(o_t | z_t).$$

若用于变分训练，目标包含：

$$\mathcal{L}_{\text{rec}} = -\log p_{\psi}(o_t | z_t).$$

但规划通常只需要 z_t ，不一定每步都解码到像素。

本题符号说明 其中， o_t 表示当前观测； z_t 表示模型内部 latent state； $\mathcal{L}$ 表示训练损失。

2.6.3 总结

observation model 或 decoder 负责把内部状态映射回可观测空间。它可以用于重建、可视化和训练约束，但不能把 decoder 的清晰度误认为世界模型的好坏；关键仍然是 latent dynamics 是否正确。

2.7 问题 17: reward model 为什么对 agent 世界模型很重要？

2.7.1 通俗解释

agent 不仅需要知道未来长什么样，还需要知道未来“好不好”。reward model 把 latent state 和动作映射到任务价值，让世界模型不仅能 rollout，还能从多个想象轨迹中选出最优。这是世界模型从“观察者”变成“决策者”的关键一环。

reward model 的实际作用还有几个不那么直观的方面。第一，它充当 latent 的额外监督信号——即使 reward 稀疏，reward 预测也会反向迫使 latent 编码任务相关变量；第二，它和 termination model 配合可以正确处理 episode 中段失败；第三，在 offline 或 sim-to-real 场景中，它经常被分解成 task reward + safety cost + smoothness cost 等多项，分别由不同子模型预测，再加权组合作为 planner 的目标函数。

2.7.2 数学推导

基本形式是状态-动作条件下的奖励分布：

$$p_{\rho}(r_t | z_t, a_t).$$

在长 horizon 规划中，agent 在 imagined rollout 上选择能最大化期望累积回报的动作序列：

$$a_{t:t+H}^* = \arg \max_{a_{t:t+H}} \mathbb{E}_{p_{\theta}, p_{\rho}} \left[\sum_{k=0}^H \gamma^k r_{t+k} \right].$$

在 Dreamer 类方法里，critic 进一步学习一个 value function 来 bootstrap 长程回报：

$$V_{\xi}(z_t) \approx \mathbb{E} \left[\sum_{k=0}^{\infty} \gamma^k r_{t+k} \right],$$

使 actor 不必依赖完整 horizon。在驾驶或机器人场景中，单一 reward 通常不够，工程上常分解为多项加权代价：

$$C(z_t, a_t) = \lambda_1 r_{\text{task}} + \lambda_2 c_{\text{safety}} + \lambda_3 c_{\text{smooth}} + \lambda_4 c_{\text{rule}},$$

每一项各自由 reward model 的子模块预测，再交给 planner 联合优化。

本题符号说明 其中, z_t 表示 latent state; a_t 表示动作; r_t 表示奖励; p_ρ 表示 reward model; V_ξ 表示价值函数; C 表示综合代价; λ_i 表示各项权重; γ 是折扣因子; H 表示规划 horizon。

2.7.3 总结

reward model 把世界预测和任务目标连接起来, 让智能体不只是”想象未来”, 而是”评价未来”。它不仅是 planning 的目标函数, 还充当 latent 的额外监督信号, 并在多目标系统中被分解成 task、safety、smoothness、rule 等多个子模块, 使 world model 能直接服务安全约束下的真实决策。

2.8 问题 18: termination model 在强化学习 world model 中起什么作用?

2.8.1 通俗解释

termination model 预测 episode 是否结束。它在机器人和强化学习中很重要, 因为失败、碰撞、掉落、任务完成都可能终止轨迹。如果忽略 termination, 模型会把已经失败的状态继续 rollout, 从而误导策略。换言之, termination model 防止模型把已经结束的轨迹继续想象下去, 对安全和长期规划都很关键。

2.8.2 数学推导

终止概率为:

$$p_\eta(d_t = 1 \mid z_t, a_t).$$

带终止的回报为:

$$G_t = \sum_{k=0}^H \gamma^k \left(\prod_{j=0}^k (1 - d_{t+j}) \right) r_{t+k}.$$

本题符号说明 其中, z_t 表示 latent state; a_t 表示动作; d_t 表示终止标志 (1 表示 episode 结束); p_η 表示参数为 η 的终止概率模型; r_t 表示奖励; G_t 表示带终止权重的累积回报; γ 是折扣因子; H 表示 rollout 步数。

2.8.3 总结

termination model 预测 episode 何时结束或任务何时失败。它在安全任务中特别重要, 因为碰撞、出界、失稳、掉落等终止事件往往比普通 reward 更稀疏但更关键。

2.9 问题 19: deterministic dynamics 和 stochastic dynamics 有什么区别?

2.9.1 通俗解释

deterministic dynamics 假设同一状态和动作必然产生同一未来; stochastic dynamics 承认环境有不确定性。真实世界中其他车辆意图、物体摩擦、传感器噪声都会导致多种可能未来, 因此很多世界模型需要概率建模。一般来说, deterministic dynamics 适合规则稳定的部分, stochastic dynamics 用来表达多种可能未来以及不可观测因素。

2.9.2 数学推导

确定性模型:

$$z_{t+1} = f_{\theta}(z_t, a_t).$$

随机模型:

$$z_{t+1} \sim p_{\theta}(z_{t+1} | z_t, a_t).$$

可用高斯参数化:

$$p_{\theta} = \mathcal{N}(\mu_{\theta}(z_t, a_t), \Sigma_{\theta}(z_t, a_t)).$$

本题符号说明 其中, z_t 表示 latent state; a_t 表示动作; p_{θ} 表示学习到的概率模型; f_{θ} 表示确定性 transition; $\mathcal{N}$ 表示高斯分布; μ_{θ} 和 Σ_{θ} 表示预测分布的均值和协方差。

2.9.3 总结

deterministic dynamics 假设同一状态和动作只有一个未来, stochastic dynamics 承认未来可能分叉。真实世界中行人、车辆、接触和传感噪声都会带来不确定性, 因此世界模型通常需要预测分布而不是单一路径。

2.10 问题 20: uncertainty modeling 为什么是世界模型的关键能力?

2.10.1 通俗解释

不确定性告诉 agent 哪些未来可靠、哪些未来只是模型在“硬猜”。没有不确定性估计, 世界模型可能在分布外场景里依然非常自信地输出错误未来——这在长尾驾驶 (少见的天气、道路异常)、机器人接触 (首次接触的新物体) 和 offline planning (数据未覆盖的动作) 里尤其危险。

工程上要区分两类不确定性: *aleatoric* 是环境本身的随机性 (行人未来意图、传感器噪声), 无法通过加数据消除, 需要在模型里显式预测分布; *epistemic* 是模型由于训练数据有限而产生的认知不确定性, 可以通过 ensemble、Bayesian 近似或 dropout 估计。前者对应“我知道这件事不确定”, 后者对应“我不知道我不知道”。世界模型在分布外失败大多源于 epistemic uncertainty 没被正确估计。

2.10.2 数学推导

将 transition 写成参数化高斯:

$$p_{\theta}(z_{t+1} | z_t, a_t) = \mathcal{N}(\mu_{\theta}(z_t, a_t), \Sigma_{\theta}(z_t, a_t)),$$

其中 Σ_{θ} 表达 aleatoric 部分。epistemic 部分可用 ensemble 估计: 训练 K 个独立模型, 预测均值和方差为:

$$\mu(z, a) = \frac{1}{K} \sum_{k=1}^K \mu_{\theta_k}(z, a), \quad \sigma_{\text{epi}}^2(z, a) = \frac{1}{K} \sum_{k=1}^K (\mu_{\theta_k} - \mu)^2.$$

规划时把不确定性纳入目标。风险敏感优化常写为:

$$J(a) = \mathbb{E}[R(\tau)] - \lambda \text{Var}[R(\tau)],$$

或用 CVaR 形式衡量分位风险:

$$\text{CVaR}_{\alpha}[R] = \mathbb{E}[R | R \leq \text{VaR}_{\alpha}[R]].$$

在 offline RL 中，常用 uncertainty penalty 抑制 OOD 动作：

$$\tilde{r}(z, a) = \hat{r}(z, a) - \beta \sigma_{\text{epi}}(z, a),$$

让 planner 主动避开模型不确定的区域。

本题符号说明 其中， z_t 表示 latent state； a_t 表示动作； p_θ 表示 transition 模型； μ_θ 、 Σ_θ 分别为预测分布的均值和协方差； θ_k 表示第 k 个 ensemble 成员； σ_{epi} 表示 epistemic 不确定性； $R(\tau)$ 表示轨迹回报； Var 表示方差； CVaR_α 表示分位风险； λ 、 β 是风险厌恶系数。

2.10.3 总结

uncertainty modeling 让“世界模型”知道自己何时不确定”。它的价值有三层：让预测带置信度 (aleatoric vs epistemic 区分)、指导主动探索 (去高 epistemic 的区域采数据)、支撑安全决策 (在 OOD 状态自动保守、触发人类接管或降级策略)。没有不确定性估计，模型很容易把幻觉当成确定事实，尤其在分布外场景。

3 模型式强化学习与 Dreamer 系列

本章主线 本章答题主线：Dreamer 系列的核心是把真实环境交互转化为 latent imagination，并在想象轨迹上训练 actor-critic。

3.1 问题 21：什么是 Model-Based Reinforcement Learning?

3.1.1 通俗解释

Model-Based RL (MBRL) 的核心是先学习一个环境模型，再在模型中做规划、生成想象数据或反事实评估。它的训练循环通常是“采集真实交互 → 更新模型 → 在模型中训练策略 → 用更新后的策略再去采集”。和 model-free RL 比，最大优势是样本效率：一次真实采样不仅训练当前策略，还训练内部模型，再由模型生成大量 imagined rollout 喂给 actor-critic，使真实环境的昂贵交互被复用。

MBRL 适合三类场景：(1) 真实采样昂贵或危险 (机器人、自动驾驶)；(2) 需要 long-horizon planning (机械臂多步操作)；(3) 需要反事实评估 (同一初始状态下比较多个动作)。它的代价主要是 *model bias*——模型预测一旦在某些区域系统性偏差，策略就会优化到这些“模型漏洞”上，而不是真实环境的最优行为。这就是为什么 MBRL 的工程难点不在“学到模型”，而在“控制 rollout 误差不被策略利用”。

3.1.2 数学推导

通用 RL 目标在策略和环境联合分布上最大化期望折扣回报：

$$J(\pi) = \mathbb{E}_{\tau \sim p, \pi} \left[\sum_t \gamma^t r_t \right].$$

MBRL 显式学习环境近似：

$$p_\theta(z_{t+1}, r_t \mid z_t, a_t),$$

并提供两类用法。第一类是 **planning** (如 MPC、CEM)：在每个决策时刻直接搜索最优动作序列：

$$a_{t:t+H}^* = \arg \max_{a_{t:t+H}} \mathbb{E}_{p_\theta} \left[\sum_{k=0}^H \gamma^k r_{t+k} \right].$$

第二类是 **Dyna-style imagination**: 用模型生成 imagined transitions $\hat{\tau}$ 与真实数据 $\mathcal{D}_{\text{real}}$ 一起训练 actor-critic:

$$\nabla J(\pi) \approx \mathbb{E}_{\mathcal{D}_{\text{real}} \cup \hat{\mathcal{D}}} \left[\nabla_{\pi} \sum_t \gamma^t r_t \right].$$

对应的性能差距由 simulation lemma 给出:

$$|J_{\text{env}}(\pi) - J_{\theta}(\pi)| \leq \frac{2\gamma R_{\text{max}}}{(1-\gamma)^2} \mathbb{E}_{(z,a) \sim d^{\pi}} [D_{\text{KL}}(p_{\text{env}} \| p_{\theta})],$$

这个不等式量化了”为什么 model bias 会被长 rollout 放大成大策略差距”。

本题符号说明 其中, z_t 表示 latent state; a_t 表示动作; r_t 表示奖励; p_{θ} 表示学习到的环境模型; p_{env} 表示真实环境分布; π 表示策略; d^{π} 表示策略诱导的状态-动作分布; $\hat{\tau}$ 、 $\hat{\mathcal{D}}$ 表示 imagined 轨迹与数据集; D_{KL} 表示 KL 散度; R_{max} 表示单步最大奖励; γ 是折扣因子; H 表示规划 horizon。

3.1.3 总结

Model-Based RL 不直接学习”状态 $\rightarrow$ 动作”的映射, 而是先建立或使用一个环境模型, 再在模型上做 planning 或 imagined rollout。它把一部分试错搬到内部模型里, 因此更适合采样昂贵、风险较高、需要长程规划或反事实评估的场景; 代价是必须用 KL 正则、ensemble、uncertainty penalty、horizon truncation 等手段控制 model bias 被策略利用。

3.2 问题 22: model-based RL 和 model-free RL 有什么区别?

3.2.1 通俗解释

Model-Based RL 先学习环境动态, 再利用这个模型做规划或生成想象数据; Model-Free RL 则跳过环境建模, 直接从经验中学习策略或价值函数。前者样本效率高、可做 long-horizon planning, 但容易受 model bias 拖累; 后者实现简单、不受模型误差影响, 但通常需要更多真实交互才能收敛。

3.2.2 数学推导

Model-Free 直接在真实环境分布上优化:

$$J_{\text{MF}}(\pi) = \mathbb{E}_{\tau \sim p_{\text{env}}, \pi} \left[\sum_t \gamma^t r_t \right].$$

Model-Based 则先估计:

$$p_{\theta}(z_{t+1}, r_t | z_t, a_t),$$

再在 p_{θ} 中优化策略:

$$J_{\text{MB}}(\pi) = \mathbb{E}_{\tau \sim p_{\theta}, \pi} \left[\sum_t \gamma^t r_t \right].$$

两者优化目标形式相同, 差别在于 rollout 来自真实环境还是学得模型。

本题符号说明 其中, z_t 表示模型内部 latent state; a_t 表示当前动作; r_t 表示 reward; p_{env} 表示真实环境分布; p_{θ} 表示学习到的环境模型; π 表示策略; J 表示优化目标; γ 是折扣因子。

3.2.3 总结

model-free RL 直接学习从状态到动作或价值的映射，model-based RL 额外学习环境动态。前者简单直接但样本需求大，后者样本效率高但受模型误差影响；两者的差异不是有没有策略，而是有没有显式 dynamics model。

3.3 问题 23：世界模型如何提升 sample efficiency?

3.3.1 通俗解释

世界模型提高 sample efficiency 的方式是把一次真实交互变成多次内部想象。真实环境只提供少量轨迹，模型学到动态后可以生成大量候选未来，用于训练 actor、critic 或 MPC。这里的 latent imagination 不是生成视频，而是在隐空间中展开未来，用预测奖励和值函数训练策略。

3.3.2 数学推导

真实数据训练模型：

$$\mathcal{D}_{real} = \{(z_t, a_t, z_{t+1}, r_t)\}.$$

模型产生想象数据：

$$\hat{\mathcal{D}} = \{(\hat{z}_t, \hat{a}_t, \hat{z}_{t+1}, \hat{r}_t)\}.$$

策略在二者上训练：

$$\nabla J \approx \nabla J(\mathcal{D}_{real} \cup \hat{\mathcal{D}}).$$

本题符号说明 其中， z_t 表示模型内部 latent state； a_t 表示当前动作或控制输入； r_t 表示 reward 或任务反馈；带帽子的变量表示模型预测值； ∇ 表示梯度。

3.3.3 总结

世界模型提升 sample efficiency 的原因是它复用了真实交互。一次真实采样不仅更新当前策略，还能训练内部模型，再由内部 rollout 生成更多训练信号，从而减少真实环境中的昂贵试错。

3.4 问题 24：什么是 imagination rollout?

3.4.1 通俗解释

imagination rollout 是在学到的世界模型内部展开未来轨迹。它的目的不是生成好看画面，而是让策略提前“试走”不同动作序列，估计这些动作会把 latent state、reward 和风险带向哪里。对 Dreamer 这类方法来说，rollout 发生在 latent space 中，因此计算更便宜，也更适合训练 actor-critic。

3.4.2 数学推导

从当前 latent state 出发，模型递推未来状态：

$$\hat{z}_{t+k+1} \sim p_{\theta}(\hat{z}_{t+k+1} | \hat{z}_{t+k}, \hat{a}_{t+k}), \quad \hat{a}_{t+k} \sim \pi(\cdot | \hat{z}_{t+k}).$$

想象轨迹上的累计回报为：

$$\hat{G}_t = \sum_{k=0}^H \gamma^k \hat{r}_{t+k}.$$

actor 试图选择能最大化 imagined return 的动作分布，critic 则学习评估这些 imagined states 的长期价值。

本题符号说明 其中， $\hat{z}_{t+k}$ 表示模型想象出的第 $t+k$ 步 latent state； $\hat{a}_{t+k}$ 表示想象轨迹中的动作； p_θ 表示世界模型的 latent transition； π 表示策略； H 表示 rollout horizon； γ 表示折扣因子； $\hat{r}_{t+k}$ 表示模型预测的奖励。

3.4.3 总结

imagination rollout 是世界模型服务决策的核心接口。它让智能体不必每次都在真实环境中试错，而是在内部 latent world 中比较多个未来，再用这些想象轨迹训练策略和价值函数。

3.5 问题 25：Dreamer 为什么要在 latent space 里 rollout?

3.5.1 通俗解释

Dreamer 在 latent space 里 rollout，是为了避开像素空间的高维噪声和昂贵生成开销。像素未来既难以长期保持一致，又容易把模型容量浪费在纹理、光照等与控制无关的细节上。在 latent space 中递推未来则更紧凑、更可微，也更聚焦于任务相关变量，因此更适合驱动 actor-critic 的长期策略学习。

3.5.2 数学推导

Dreamer 中 actor-critic 在 latent imagination 上联合训练。critic 学习 latent state 的长期价值：

$$V_\xi(\hat{z}_t) \approx \mathbb{E} \left[\sum_{k=0}^H \gamma^k \hat{r}_{t+k} \right].$$

actor 优化 latent rollout 上的期望回报：

$$J(\pi_\psi) = \mathbb{E}_{p_\theta, \pi_\psi} \left[\sum_{k=0}^H \gamma^k \hat{r}_{t+k} \right].$$

整个过程不需要解码到像素，因此每步 rollout 的成本远低于直接生成视频。

本题符号说明 其中， $\hat{z}_t$ 表示 latent imagination 中的状态； $\hat{r}_t$ 表示模型预测的奖励； V_ξ 表示价值函数； π_ψ 表示策略； p_θ 表示 latent dynamics； γ 是折扣因子； H 表示 rollout horizon。

3.5.3 总结

Dreamer 把 rollout 放在 latent space，是为了避免像素空间的高维噪声和生成误差。latent rollout 更紧凑、更可微，也更聚焦于任务相关变量，因此更适合长期策略学习。

3.6 问题 26：RSSM 为什么适合部分可观测环境?

3.6.1 通俗解释

RSSM 适合部分可观测环境，是因为它同时维护两类信息：deterministic hidden state 负责记住历史，stochastic latent state 负责表达当前状态的不确定性。很多环境里，单帧观测并不能确

定真实状态，例如遮挡物体、隐藏速度、接触关系或对方意图。RSSM 用历史记忆形成 belief，再用随机变量表示多个可能未来。

3.6.2 数学推导

RSSM 的确定性记忆更新为：

$$h_t = f_\theta(h_{t-1}, z_{t-1}, a_{t-1}).$$

先验状态由历史记忆给出：

$$z_t \sim p_\theta(z_t | h_t).$$

看到当前观测后，后验状态修正为：

$$z_t \sim q_\phi(z_t | h_t, o_t).$$

训练时模型学习让 prior 能接近 posterior，这样在没有未来真实观测时也能稳定想象。

本题符号说明 其中， h_t 表示确定性历史记忆； z_t 表示随机 latent state； a_{t-1} 表示上一时刻动作； o_t 表示当前观测； p_θ 表示先验动态模型； q_ϕ 表示由观测修正后的后验模型。

3.6.3 总结

RSSM 的关键价值是把“记住过去”和“表达不确定性”分开建模。确定性状态保留长期上下文，随机状态表达当前 belief，因此它比单纯 RNN 或单纯 VAE 更适合部分可观测环境中的 world modeling。

3.7 问题 27: Dreamer 的 reconstruction loss、reward loss、KL loss 分别起什么作用？

3.7.1 通俗解释

Dreamer 的 world model 不是只学一个好看的 latent，而是要让 latent 同时满足三件事：能解释观测，能预测任务反馈，能在没有真实未来观测时继续 rollout。reconstruction loss 保证 latent 没有丢掉关键视觉信息；reward loss 保证 latent 和任务目标相关；KL loss 约束 posterior 和 prior，让模型从真实观测切换到 imagined rollout 时不会突然崩掉。

3.7.2 数学推导

典型训练目标可以写成：

$$\mathcal{L} = \mathcal{L}_{rec} + \lambda_r \mathcal{L}_{reward} + \beta D_{KL}(q_\phi(z_t | h_t, o_t) || p_\theta(z_t | h_t)).$$

其中 reconstruction loss 可写成：

$$\mathcal{L}_{rec} = -\log p_\theta(o_t | z_t, h_t),$$

reward loss 可写成：

$$\mathcal{L}_{reward} = -\log p_\theta(r_t | z_t, h_t).$$

本题符号说明 其中， $\mathcal{L}_{rec}$ 表示观测重建损失； $\mathcal{L}_{reward}$ 表示奖励预测损失； D_{KL} 表示 KL divergence； q_ϕ 表示 posterior encoder； p_θ 表示 prior dynamics； h_t 表示历史记忆； z_t 表示 latent state； λ_r 和 β 是损失权重。

3.7.3 总结

reconstruction loss 让 latent 保留可解释观测, reward loss 让 latent 对任务有用, KL loss 让模型能够从真实观测过渡到内部想象。三者合起来, 才让 Dreamer 的 latent world 既可学习、又可 rollout、还能服务控制。

3.8 问题 28: actor、critic 和 world model 如何联合训练?

3.8.1 通俗解释

Dreamer 中 world model 负责想象轨迹, critic 估计 latent state 的长期价值, actor 学习在想象轨迹中选择动作。三者不是简单串联, 而是在同一个 latent dynamics 里形成闭环训练。常见的工程处理包括短视野 rollout、value bootstrap、latent regularization 和 uncertainty penalty, 用来缓解长 horizon 下的误差累积。

3.8.2 数学推导

critic 学习 latent state 的长期价值:

$$V_{\xi}(z_t) \approx \mathbb{E} \left[\sum_k \gamma^k r_{t+k} \right].$$

actor 在 world model 给出的想象轨迹上优化期望回报:

$$\max_{\psi} \mathbb{E}_{p_{\theta}, \pi_{\psi}} \left[\sum_k \gamma^k r_{t+k} \right].$$

world model 自身则用 reconstruction、reward 和 KL 等损失同步训练, 三者构成同一组 latent dynamics 上的闭环。

本题符号说明 其中, z_t 表示 latent state; r_t 表示奖励; V_{ξ} 表示价值函数; π_{ψ} 表示策略; p_{θ} 表示 world model; ξ 、 ψ 、 θ 分别为各自模块的参数; γ 是折扣因子; $\mathbb{E}$ 表示期望。

3.8.3 总结

world model 学习环境, actor 在想象环境中选择动作, critic 评估这些动作的长期价值。三者联合训练的关键是分工明确: 模型负责可信未来, actor 负责行动, critic 负责价值判断。

3.9 问题 29: imagination horizon 太长为什么会带来 model bias?

3.9.1 通俗解释

rollout 越长, 策略能看到越远的后果, 但模型误差也会逐步累积。短 horizon 容易近视, 长 horizon 则可能被错误的想象欺骗——尤其是 offline 训练的 world model, 分布外动作的后果几乎全靠外推, 规划时很容易出现虚假高回报。

3.9.2 数学推导

设单步预测误差为 ϵ ，则 H 步 rollout 的累积误差大致满足：

$$\text{Err}(H) \lesssim \sum_{k=1}^H \epsilon \cdot \prod_{j < k} L_j,$$

其中 L_j 表示第 j 步的 Lipschitz 因子或动态放大系数。若 transition 含有系统偏差 b ，长期预测则会沿错误动态稳步漂移：

$$\hat{z}_{t+H} - z_{t+H} \sim H \cdot b.$$

本题符号说明 其中， H 表示 rollout 步数； ϵ 表示单步预测误差； L_j 表示动态放大系数； b 表示模型系统偏差； $\hat{z}$ 与 z 分别为预测和真实状态。

3.9.3 总结

imagination horizon 越长，越能看到长期回报，但模型误差也会不断累积。过长的 horizon 可能让策略优化到模型漏洞上，而不是优化真实环境中的有效行为；这就是 model bias 在长 rollout 中放大的根源。

3.10 问题 30：model bias 和 compounding error 有什么区别？

3.10.1 通俗解释

model bias 描述模型本身和真实环境的偏差——即使只跑一步，预测也可能不准；compounding error 描述这种偏差在 rollout 中被反复使用后被逐步放大的过程。换句话说，model bias 是”病灶”，compounding error 是”病程”，两者一个来自模型质量，一个来自使用方式。

3.10.2 数学推导

设真实动态为 f^* ，模型为 f_θ ，单步偏差为：

$$b(z, a) = f_\theta(z, a) - f^*(z, a).$$

model bias 就是 $\|b\|$ 本身。多步 rollout 中，误差则会沿轨迹累积：

$$\|\hat{z}_{t+H} - z_{t+H}\| \lesssim \sum_{k=1}^H \|b(z_{t+k-1}, a_{t+k-1})\| \cdot \prod_{j < k} L_j.$$

即使 $\|b\|$ 很小，只要 H 足够大或 $L_j > 1$ ，误差仍可能爆炸。

本题符号说明 其中， f^* 表示真实动态； f_θ 表示学习到的动态； $b(z, a)$ 表示单步偏差； L_j 表示动态放大系数； H 表示 rollout 步数。

3.10.3 总结

model bias 是学到的模型和真实环境之间的系统性偏差，compounding error 则是 rollout 中一步步误差累积后被放大的结果。前者来自模型本身不准，后者来自多步反复使用不准的模型。

4 JEPA 基础、I-JEPA、V-JEPA 与 V-JEPA 2

本章主线 本章答题主线：JEPA 的本质是表示预测路线。它不追求像素级重建，而追求抽象层面的未来可预测性。

4.1 问题 31：什么是 JEPA？为什么它不是普通 autoencoder？

4.1.1 通俗解释

JEPA 是 Joint Embedding Predictive Architecture，它学习的是“在表示空间里预测目标区域”，而不是把输入像素完整重建出来。普通 autoencoder 容易把容量花在颜色、纹理和局部噪声上；JEPA 则通过预测 target embedding，逼模型关注更稳定的语义结构、物体关系和时空上下文。因此，JEPA 更像是在学习世界的抽象结构，而不是学习像素压缩和复原。

4.1.2 数学推导

Autoencoder 的目标是重建输入：

$$\mathcal{L}_{AE} = \|D_{\psi}(E_{\phi}(x)) - x\|_2^2.$$

JEPA 的目标是用 context 表示预测 target 表示：

$$z_c = E_{\phi}(x_{context}), \quad z_t = E_{\xi}(x_{target}),$$
$$\mathcal{L}_{JEPA} = \|P_{\theta}(z_c) - \text{sg}(z_t)\|_2^2.$$

这里 target encoder 通常使用 stop-gradient，避免两端同时坍塌。

本题符号说明 其中， $x_{context}$ 表示可见上下文区域； x_{target} 表示被遮挡或需要预测的目标区域； E_{ϕ} 表示 context encoder； E_{ξ} 表示 target encoder； P_{θ} 表示 predictor； D_{ψ} 表示 decoder； $\text{sg}(\cdot)$ 表示 stop-gradient。

4.1.3 总结

JEPA 和 autoencoder 的根本区别是学习目标不同：autoencoder 追求像素复原，JEPA 追求表示预测。前者更像压缩器，后者更像抽象世界结构的预测器。

4.2 问题 32：I-JEPA 为什么预测 embedding 而不是重建 pixel？

4.2.1 通俗解释

I-JEPA 预测 embedding，是为了避免把学习目标浪费在低层像素细节上。图像里的纹理、颜色和局部噪声并不总是对理解世界有帮助；相反，物体身份、空间关系和上下文语义才更适合作为下游任务的基础。I-JEPA 通过用上下文 block 预测目标 block 的 embedding，让模型学习“这块区域在语义上应该是什么”，而不是“每个像素应该是什么颜色”。

4.2.2 数学推导

设图像被分成 context blocks 和 target blocks。context encoder 得到：

$$z_c = E_{\phi}(x_c).$$

target encoder 给出目标表示:

$$z_t = E_\xi(x_t).$$

predictor 根据 context 和目标位置编码预测 target embedding:

$$\hat{z}_t = P_\theta(z_c, pos_t).$$

训练目标为:

$$\mathcal{L}_{I-JEPA} = \|\hat{z}_t - \text{sg}(z_t)\|_2^2.$$

本题符号说明 其中, x_c 表示 context blocks; x_t 表示 target blocks; pos_t 表示目标区域的位置编码; z_c 和 z_t 分别表示上下文和目标 embedding; $\hat{z}_t$ 表示预测的目标 embedding; $\text{sg}(\cdot)$ 表示 stop-gradient.

4.2.3 总结

I-JEPA 不重建 pixel, 是因为它想让模型学习语义结构, 而不是学习纹理复制。embedding prediction 把监督信号从低层视觉细节提升到高层表示空间, 这是它区别于 masked autoencoder 的关键。

4.3 问题 33: JEPA 为什么被 LeCun 认为是世界模型路线?

4.3.1 通俗解释

LeCun 长期主张: 智能体不该花容量去逐像素重建世界, 也不该完全依赖语言自回归来理解物理现实。他的核心论点是——世界中绝大部分细节(光照、纹理、噪声、不可预测的微小扰动)对决策毫无意义, 只有抽象层面的状态、对象、关系和动态才值得预测。JEPA 正好对应这个思想: 用一个 encoder 把世界压缩成 embedding, 再用一个 predictor 在 embedding 空间预测未被观测的部分; 像素重建被显式排除在训练目标之外。

这条路线和两条对手路线划清了界限: 与生成式 world model (Sora、Cosmos) 相比, JEPA 不输出像素, 因此不会被像素细节牵着走、也不会因为生成器太弱而限制理解能力; 与 LLM 风格的 token autoregression 相比, JEPA 在视觉/物理表示空间预测, 更适合连续时空动态, 避免了把世界硬塞进离散符号空间。V-JEPA 把这种思想扩展到视频, 再到 V-JEPA 2 加上动作条件, 已经在机器人 zero-shot 控制中体现出价值, 正是 LeCun 所说的“abstract predictive world model”的雏形。

4.3.2 数学推导

JEPA 的核心范式可以写成三件事: encoder、predictor、目标只在 embedding 空间计算。设 context 与 target 分别为 x_c 、 x_t , 对应的 embedding 为:

$$z_c = E_\phi(x_c), \quad z_t = E_\xi(x_t).$$

predictor 根据 context 与目标位置/动作预测 target 的 embedding:

$$\hat{z}_t = P_\theta(z_c, c),$$

其中 c 可以是位置编码、动作序列或时间偏移。损失只在 embedding 空间, target 端用 stop-gradient 防止 collapse:

$$\mathcal{L}_{JEPA} = \|\hat{z}_t - \text{sg}(z_t)\|_2^2.$$

扩展到视频和动作条件后：

$$\hat{z}_{t+k} = P_{\theta}(E_{\phi}(o_{\leq t}), a_{t:t+k-1}), \quad \mathcal{L} = \sum_{t,k} \|\hat{z}_{t+k} - \text{sg}(E_{\xi}(o_{t+k}))\|_2^2.$$

整个目标里没有 $\|\hat{o} - o\|$ ，因此模型不会把容量花在像素重建上——这是它与 MAE/diffusion world model 的关键差异。

本题符号说明 其中, x_c, x_t 分别表示 context 与 target 区域; E_{ϕ}, E_{ξ} 分别为 context 与 target encoder; P_{θ} 表示 predictor; z_c, z_t 表示对应 embedding; $\hat{z}_t$ 表示预测 embedding; $o_{\leq t}, o_{t+k}$ 表示历史与未来观测; $a_{t:t+k-1}$ 表示动作序列; c 表示位置编码或条件; $\text{sg}(\cdot)$ 表示 stop-gradient。

4.3.3 总结

LeCun 看重 JEPA，是因为它把世界建模从”还原像素”转向”预测抽象状态”。世界模型真正需要的是对世界结构的压缩预测，而不是对每个低层细节的逐点复原。这一选择不仅是模型架构差异，更是关于”什么才算理解世界”的方法论分歧——预测可被干预的语义结构，而不是预测每个像素的颜色。

4.4 问题 34：V-JEPA 如何把 JEPA 从图像扩展到视频？

4.4.1 通俗解释

V-JEPA 把 JEPA 从静态图像扩展到视频，让模型通过预测未来或被遮挡的视频表示学习物理动态。V-JEPA 2 进一步使用大规模自然视频预训练，再用少量机器人轨迹做 action-conditioned post-training，使表示可以服务机器人控制。和 MAE 相比，V-JEPA 更强调 semantic predictability：MAE 更接近像素重建任务，JEPA 更接近抽象状态建模。

4.4.2 数学推导

视频 JEPA 的训练目标可写成：

$$\mathcal{L} = \sum_{t,k} \|P_{\theta}(E(o_{\leq t}), a_{t:t+k}) - \text{sg}(E(o_{t+k}))\|_2^2.$$

当不使用动作时，条件中去掉 $a_{t:t+k}$ ；当用于机器人后训练时，则把动作序列作为额外条件加入 predictor。

本题符号说明 其中, $o_{\leq t}$ 表示历史视频观测; o_{t+k} 表示目标视频片段; $a_{t:t+k}$ 表示动作序列; E 表示视频 encoder; P_{θ} 表示 predictor; $\mathcal{L}$ 表示训练损失; $\text{sg}(\cdot)$ 表示 stop-gradient。

4.4.3 总结

V-JEPA 把图像中的空间预测扩展为视频中的时空预测。模型不仅要理解一帧里的结构，还要预测物体、动作和场景如何随时间演化。

4.5 问题 35: V-JEPA 2 为什么被称为 self-supervised foundation world model?

4.5.1 通俗解释

V-JEPA 2 被称为 self-supervised foundation world model, 是因为它把大规模无标注视频中的时空变化当作训练信号, 让模型学习通用视觉动态表示。它不是为单个任务训练一个预测器, 而是先学习“世界如何随时间变化”的基础表示, 再把这个表示迁移到机器人、视频理解和控制任务中。这里的 foundation 体现在预训练规模和迁移能力, world model 体现在对未来表示和动态结构的预测。

4.5.2 数学推导

视频 JEPA 可把历史上下文编码为:

$$z_{\leq t} = E_{\phi}(o_{\leq t}).$$

模型预测未来或被遮挡片段的 target embedding:

$$\hat{z}_{t+k} = P_{\theta}(z_{\leq t}, pos_{t+k}).$$

自监督目标为:

$$\mathcal{L}_{V-JEPA} = \sum_{t,k} \|\hat{z}_{t+k} - \text{sg}(E_{\xi}(o_{t+k}))\|_2^2.$$

本题符号说明 其中, $o_{\leq t}$ 表示历史视频观测; o_{t+k} 表示未来或目标视频片段; E_{ϕ} 表示 context encoder; E_{ξ} 表示 target encoder; P_{θ} 表示预测器; pos_{t+k} 表示目标时空位置; $\text{sg}(\cdot)$ 表示 stop-gradient。

4.5.3 总结

V-JEPA 2 的核心不是生成视频, 而是用视频自监督学习可迁移的时空动态表示。它像一个基础视觉世界模型, 为机器人控制、空间推理和动作预测提供底层表征。

4.6 问题 36: V-JEPA 2 如何结合互联网视频和少量 robot trajectories?

4.6.1 通俗解释

V-JEPA 2 的思路是先用海量互联网视频学习通用视觉动态, 再用少量 robot trajectories 把这种动态表示接到动作空间。互联网视频提供丰富的物体运动、场景变化和时间结构, 但缺少机器人动作标签; 机器人轨迹虽然少, 却提供了“动作如何改变世界”的监督。因此两阶段训练可以兼顾规模和可控性。

4.6.2 数学推导

第一阶段是无动作视频预训练:

$$\mathcal{L}_{video} = \sum_{t,k} \|P_{\theta}(E_{\phi}(o_{\leq t}), pos_{t+k}) - \text{sg}(E_{\xi}(o_{t+k}))\|_2^2.$$

第二阶段加入机器人动作条件:

$$\mathcal{L}_{robot} = \sum_{t,k} \|P_{\theta}(E_{\phi}(o_{\leq t}), a_{t:t+k}, pos_{t+k}) - \text{sg}(E_{\xi}(o_{t+k}))\|_2^2.$$

前者学习自然视频动态, 后者学习 action-conditioned dynamics。

本题符号说明 其中, $o_{\leq t}$ 表示历史视频观测; $a_{t:t+k}$ 表示机器人动作片段; pos_{t+k} 表示预测目标的时空位置; E_ϕ 和 E_ξ 分别表示 context encoder 与 target encoder; P_θ 表示预测器。

4.6.3 总结

互联网视频负责提供大规模视觉世界先验, 少量 robot trajectories 负责提供动作-后果对应关系。V-JEPA 2 的价值在于把“看懂世界如何变化”和“理解机器人动作如何改变世界”接起来。

4.7 问题 37: V-JEPA 2 如何实现 zero-shot robot control?

4.7.1 通俗解释

V-JEPA 2 的 zero-shot robot control 不是说模型凭空学会所有控制, 而是说预训练得到的视觉动态表示足够通用, 可以帮助策略在新任务中更快选择动作。模型先把当前场景和目标映射到 latent space, 再用动作条件预测比较不同候选动作的未来表示, 选择更接近目标的动作。关键在于 latent 表示必须保留与控制相关的信息, 例如物体位置、接触状态和可供性。

4.7.2 数学推导

设当前观测和目标分别编码为:

$$z_t = E_\phi(o_t), \quad z_g = E_\phi(o_g).$$

对候选动作 a 预测未来表示:

$$\hat{z}_{t+1} = P_\theta(z_t, a).$$

zero-shot 控制可以通过 latent 距离选择动作:

$$a^* = \arg \min_a d(\hat{z}_{t+1}, z_g).$$

若使用多步候选轨迹, 则选择使最终 latent 最接近目标的动作序列。

本题符号说明 其中, o_t 表示当前观测; o_g 表示目标观测或目标状态; z_t 和 z_g 表示当前与目标 latent; a 表示候选动作; $\hat{z}_{t+1}$ 表示动作后的预测 latent; $d(\cdot, \cdot)$ 表示 latent 距离或任务代价。

4.7.3 总结

V-JEPA 2 实现 zero-shot control 的关键是把控制问题转化为 latent space 中的目标接近问题。预训练模型提供可迁移的时空表示, 动作条件预测提供比较候选动作后果的接口。

4.8 问题 38: JEPA 路线和生成式 world model 路线有什么根本区别?

4.8.1 通俗解释

两条路线最本质的差异是输出空间: 生成式 world model (Sora、Cosmos、Genie 等) 试图生成像素或视频, 输出是可视化、可被人类直接检查的; JEPA 路线只预测 embedding, 输出是不可视的抽象表示。这带来两套截然不同的工程取舍——

生成式优势: 可视化直观、可作为合成数据用于策略训练、可暴露失败模式给开发者、对内容产业有直接产品价值; **劣势:** 训练昂贵 (需要大量像素监督)、容量被纹理细节占用、长 horizon 累积误差大、在物理/因果一致性上往往脆弱。

JEPA 优势：训练高效、表示紧凑且语义化、长 horizon rollout 漂移更小、更适合作为下游策略 backbone；**劣势：**不能直接渲染、调试困难、缺乏天然的可视化评估手段、需要单独构建 decoder 才能给人看。

V-JEPA-AC、VLA-JEPA 等扩展进一步把 action 放进表示预测，使 JEPA 路线从”被动观察预测”走向”动作后果预测”，从而具备闭环服务策略的能力。

4.8.2 数学推导

生成式 world model 的训练目标在像素或 token 空间，常见形式包括极大似然或 diffusion score matching：

$$\mathcal{L}_{\text{gen}} = -\log p_{\theta}(o_{t+1} | z_t, a_t), \quad \text{或} \quad \mathcal{L}_{\text{diff}} = \mathbb{E}_{\epsilon, t} \left\| \epsilon - \epsilon_{\theta}(o^{(t)}, t, a) \right\|_2^2.$$

JEPA 的训练目标只在 embedding 空间，target 端 stop-gradient：

$$\mathcal{L}_{\text{JEPA}} = \left\| P_{\theta}(z_t, a_t) - \text{sg}(E_{\xi}(o_{t+1})) \right\|_2^2.$$

从信息论角度，前者要求模型保留 $H(o_{t+1} | z_t, a_t)$ 全部熵，后者只需要保留对 z_{t+1} 有用的部分；当 $\dim(z) \ll \dim(o)$ 时，JEPA 的有效优化目标维度大幅降低，但同时也丧失了像素级重建能力。混合路线则把两者组合：

$$\mathcal{L} = \alpha \mathcal{L}_{\text{JEPA}} + \beta \mathcal{L}_{\text{recon}},$$

兼顾抽象预测与可视化，这正是 LeWorldModel、Cosmos-Predict 等系统采用的折中方案。

本题符号说明 其中， o_{t+1} 表示下一帧观测； z_t, z_{t+1} 表示 latent state； a_t 表示动作； p_{θ} 表示生成模型； ϵ_{θ} 表示 diffusion 噪声预测器； P_{θ} 表示 JEPA predictor； E_{ξ} 表示 target encoder； $H(\cdot | \cdot)$ 表示条件熵； $\dim(\cdot)$ 表示维度； α, β 是混合权重； $\text{sg}(\cdot)$ 表示 stop-gradient。

4.8.3 总结

JEPA 路线强调在抽象表示空间预测，生成式路线强调还原或生成可见未来。前者训练高效、表示紧凑、长程稳定，但缺乏直接可视化；后者直观、可被产品使用，但容易被像素细节牵制、训练昂贵且物理一致性脆弱。当前许多前沿世界模型采用混合方案，让两种路线各自发挥所长。

4.9 问题 39：JEPA 的优势是抽象预测，代价是什么？

4.9.1 通俗解释

JEPA 的优势是避开像素级重建，把学习重点放在更稳定、更语义化的表示预测上。这样模型不容易被纹理、噪声和局部细节牵着走，更适合作为视觉理解和世界建模的底座。代价是：latent 表示不一定能直接渲染成人能检查的画面，也不一定天然包含动作、奖励、风险和可控性。因此 JEPA 很强，但它不是自动等于完整 world model。

4.9.2 数学推导

JEPA 的表示预测目标为：

$$\mathcal{L}_{\text{JEPA}} = \left\| P_{\theta}(z_{\text{context}}) - \text{sg}(z_{\text{target}}) \right\|_2^2.$$

如果要变成可用于决策的 world model，还需要动作条件转移：

$$p_{\theta}(z_{t+1} \mid z_t, a_t),$$

以及任务相关预测：

$$p_{\theta}(r_t, c_t \mid z_t, a_t).$$

本题符号说明 其中， $z_{context}$ 表示上下文表示； z_{target} 表示目标表示； P_{θ} 表示预测器； a_t 表示动作； r_t 表示奖励； c_t 表示 cost 或风险； $\text{sg}(\cdot)$ 表示 stop-gradient。

4.9.3 总结

JEPA 的优势是抽象、稳定、少受像素噪声干扰；代价是缺少天然可视化、动作闭环和任务反馈接口。它是很好的世界表示学习方法，但还需要控制、评估和规划模块才能成为完整世界模型。

4.10 问题 40：JEPA 能否单独构成完整 world model?

4.10.1 通俗解释

JEPA 单独通常不能构成完整 world model。它擅长学习预测性表示，但完整世界模型还必须回答动作干预、未来状态、奖励或风险、长期不确定性以及规划接口等问题。也就是说，JEPA 可以是 world model 的 representation backbone，却还需要 action-conditioned dynamics、decoder 或 evaluator、policy/planner 等模块补齐闭环。

4.10.2 数学推导

JEPA 学到的是表示预测：

$$\hat{z}_{target} = P_{\theta}(z_{context}).$$

完整 world model 至少需要动作条件状态转移：

$$p_{\theta}(z_{t+1} \mid z_t, a_t),$$

任务反馈预测：

$$p_{\theta}(r_t, c_t \mid z_t, a_t),$$

以及规划目标：

$$a_{t:t+H}^* = \arg \min_{a_{t:t+H}} \mathbb{E} \left[\sum_{k=0}^H C(z_{t+k}, a_{t+k}) \right].$$

本题符号说明 其中， z_t 表示 latent state； a_t 表示动作； r_t 表示奖励； c_t 表示风险或 cost； C 表示规划代价； H 表示规划 horizon； $\mathbb{E}$ 表示期望。

4.10.3 总结

JEPA 可以提供强大的世界表征，但完整 world model 还需要动作、奖励、风险、不确定性和规划接口。它更准确的定位是”世界模型的重要表示底座”，而不是”完整闭环系统”。

5 V-JEPA 2.1、LeWorldModel 与 Causal-JEPA

本章主线 本章答题主线：这一章关注 JEPA 如何继续向 dense、stable、multi-modal、causal 和 control-aware 方向演进。

5.1 问题 41: V-JEPA 2.1 相比 V-JEPA 2 的关键升级是什么？

5.1.1 通俗解释

V-JEPA 2.1 的关键不是版本号升级，而是把 JEPA 从主要擅长全局语义理解，推进到更强的 dense spatial-temporal representation。它引入 dense predictive loss、deep self-supervision 和 multi-modal tokenizers, 使 image 与 video 的 token 都能获得更好的局部空间和时间 grounding。这种 dense feature 对机器人尤其关键——它们需要局部几何、物体边界和接触区域，而不只是全局语义。

5.1.2 数学推导

普通 JEPA 主要监督 masked targets。Dense Predictive Loss 可以写成对所有 token 的加权预测：

$$\mathcal{L}_{\text{dense}} = \sum_{i \in \mathcal{V} \cup \mathcal{M}} w_i \|\hat{z}_i - \text{sg}(z_i)\|_2^2,$$

其中 $\mathcal{V}$ 是 visible tokens, $\mathcal{M}$ 是 masked tokens。Deep supervision 进一步对多层表示求和：

$$\mathcal{L} = \sum_{\ell} \alpha_{\ell} \mathcal{L}_{\text{dense}}^{(\ell)}.$$

本题符号说明 其中, $\mathcal{L}_{\text{dense}}$ 表示 dense 预测损失; $\hat{z}_i$ 表示第 i 个 token 的预测表示; z_i 表示对应的 target 表示; w_i 表示位置权重; $\mathcal{V}$ 表示 visible tokens 集合; $\mathcal{M}$ 表示 masked tokens 集合; α_{ℓ} 表示第 ℓ 层的 deep supervision 权重; $\text{sg}(\cdot)$ 表示 stop-gradient。

5.1.3 总结

V-JEPA 2.1 的升级重点在于让表示更适合密集时空理解和下游机器人任务。它不是只追求更大数据或更大模型，而是强化中间层、dense prediction 和多模态 token 统一。

5.2 问题 42: Dense Predictive Loss 为什么能增强 spatial-temporal grounding?

5.2.1 通俗解释

Dense Predictive Loss 的价值在于把预测压力从整图级语义推进到局部时空区域。只预测一个全局 embedding 时，模型可能知道“视频大概在发生什么”，但不知道哪个物体在什么位置、如何运动、和哪个局部动作相关。dense prediction 要求每个 patch、region 或 object token 都在时间上可预测，因此更适合支撑控制、跟踪和局部交互。

5.2.2 数学推导

设视频在时刻 t 的第 i 个局部 token 表示为 $z_{t,i}$ 。dense predictive loss 可写成：

$$\hat{z}_{t+k,i} = P_{\theta}(z_{\leq t, \mathcal{N}(i)}, \text{pos}_i, k),$$

$$\mathcal{L}_{dense} = \sum_{t,k} \sum_{i \in \Omega} \|\hat{z}_{t+k,i} - \text{sg}(z_{t+k,i})\|_2^2.$$

其中 $\mathcal{N}(i)$ 表示与第 i 个位置相关的上下文邻域。

本题符号说明 其中, $z_{t,i}$ 表示时刻 t 的第 i 个局部表示; $\hat{z}_{t+k,i}$ 表示预测的未来局部表示; Ω 表示被预测的局部 token 集合; pos_i 表示空间位置编码; k 表示预测步长; $\text{sg}(\cdot)$ 表示 stop-gradient。

5.2.3 总结

Dense Predictive Loss 让模型不能只学全局语义, 而必须学会局部区域如何随时间变化。它增强的是 spatial-temporal grounding, 也就是把“理解视频”推进到“理解每个关键区域的动态”。

5.3 问题 43: Deep Self-Supervision 为什么有助于提升中间层表示?

5.3.1 通俗解释

Deep Self-Supervision 的核心是让自监督信号不只作用在最后一层, 而是同时约束中间层。世界模型需要多层次表示: 浅层保留边缘、纹理和局部运动, 中层保留物体和空间结构, 高层保留语义和任务意图。如果监督只压在最后一层, 中间层可能缺少可预测性; 把预测目标加到多层后, 模型的层级表示会更稳定, 也更适合下游控制。

5.3.2 数学推导

设第 l 层的 context 表示为 z_c^l , target 表示为 z_t^l 。多层自监督目标为:

$$\mathcal{L}_{deep} = \sum_{l \in \mathcal{S}} \alpha_l \left\| P_{\theta}^l(z_c^l) - \text{sg}(z_t^l) \right\|_2^2.$$

其中 $\mathcal{S}$ 是被监督的层集合, α_l 是不同层的权重。

本题符号说明 其中, l 表示网络层编号; z_c^l 和 z_t^l 分别表示第 l 层的 context 与 target 表示; P_{θ}^l 表示第 l 层 predictor; $\mathcal{S}$ 表示参与自监督的层集合; α_l 表示层级损失权重。

5.3.3 总结

Deep Self-Supervision 的价值是让中间层也学会可预测结构。这样模型不是只在最后一层形成语义, 而是在多个层级同时保留局部、物体、时空和任务相关信息。

5.4 问题 44: Multi-Modal Tokenizers 为什么对 image-video unified training 很重要?

5.4.1 通俗解释

image-video unified training 的难点是图像和视频的粒度不同: 图像强调空间结构, 视频还多了时间动态。如果 tokenizer 不能把不同模态映射到可对齐的 token 空间, 模型就很难共享参数和训练目标。Multi-Modal Tokenizers 的作用是给图像、视频甚至动作提供统一入口, 让模型可以在同一 token 表示上学习空间、时间和动作关系。

5.4.2 数学推导

图像 tokenization 可写成:

$$Z^{img} = T_{img}(I).$$

视频 tokenization 可写成:

$$Z^{vid} = T_{vid}(V_{1:T}).$$

统一训练要求二者进入同一表示空间:

$$Z = \text{Align}(Z^{img}, Z^{vid}, Z^{act}).$$

预测目标可以写成:

$$\mathcal{L} = \|P_{\theta}(Z_{context}) - \text{sg}(Z_{target})\|_2^2.$$

本题符号说明 其中, I 表示图像; $V_{1:T}$ 表示视频序列; T_{img} 和 T_{vid} 分别表示图像与视频 tokenizer; Z^{img} 、 Z^{vid} 、 Z^{act} 分别表示图像、视频和动作 token; Align 表示跨模态对齐映射。

5.4.3 总结

Multi-Modal Tokenizers 解决的是统一训练入口问题。它让图像的空间信息、视频的时间信息和动作信息可以进入同一表示空间,从而支撑更通用的 world model 预训练。

5.5 问题 45: LeWorldModel 的核心思想是什么?

5.5.1 通俗解释

LeWorldModel 的核心是用更简单稳定的方式训练端到端 latent world model。它不把重点放在像素重建,而是让模型预测下一步 embedding,并通过 latent regularizer 约束表示空间,减少 collapse 和 rollout 漂移。可以把它理解为:用 JEPA 风格的 next-embedding prediction 学动态,再用正则化让 latent space 更平滑、更可采样。

5.5.2 数学推导

给定当前观测编码:

$$z_t = E_{\phi}(o_t).$$

模型预测下一步 embedding:

$$\hat{z}_{t+1} = P_{\theta}(z_t, a_t).$$

训练目标为:

$$\mathcal{L} = \|\hat{z}_{t+1} - \text{sg}(E_{\phi}(o_{t+1}))\|_2^2 + \lambda \mathcal{R}(z_t).$$

其中 regularizer 用于约束 latent 分布。

本题符号说明 其中, o_t 和 o_{t+1} 表示相邻观测; z_t 表示当前 latent; $\hat{z}_{t+1}$ 表示预测的下一步 latent; a_t 表示动作; E_{ϕ} 表示 encoder; P_{θ} 表示 latent predictor; $\mathcal{R}(z_t)$ 表示 latent regularization; λ 表示正则项权重。

5.5.3 总结

LeWorldModel 的关键不是提出更复杂的生成器，而是把 world model 训练稳定下来。next-embedding prediction 负责学动态，latent regularizer 负责保持表示空间可用，两者共同降低端到端训练难度。

5.6 问题 46：LeWorldModel 为什么要解决 JEPA 的端到端训练稳定性？

5.6.1 通俗解释

LeWorldModel 的重点是稳定端到端的 JEPA 训练。过去 JEPA 类方法常常依赖 EMA、预训练编码器、多项辅助损失或繁琐调参来避免 collapse。LeWM 的价值在于用 next-embedding prediction 加 Gaussian latent regularizer，让模型从 raw pixels 到 latent dynamics 的训练更简单、更稳定。Gaussian latent regularizer 的作用是给隐空间加结构，使 rollout 更平滑、更可采样、更不容易漂移。

5.6.2 数学推导

LeWM 的目标可抽象为：

$$\mathcal{L}_{\text{LeWM}} = \|P_{\theta}(z_t, a_t) - \text{sg}(z_{t+1})\|_2^2 + \lambda \mathcal{R}_{\text{Gauss}}(z).$$

Gaussian regularizer 让 batch latent 接近标准正态：

$$z \sim \mathcal{N}(0, I),$$

从而避免所有样本塌缩到同一点。

本题符号说明 其中， z_t, z_{t+1} 表示相邻 latent state； a_t 表示动作； P_{θ} 表示 predictor； $\mathcal{R}_{\text{Gauss}}$ 表示 Gaussian latent regularizer； $\mathcal{N}(0, I)$ 表示标准正态先验； λ 表示正则项权重； $\text{sg}(\cdot)$ 表示 stop-gradient。

5.6.3 总结

LeWorldModel 关注的是 JEPA 端到端训练中的稳定性问题。它试图让 representation、prediction 和 control 更紧密地联合，而不是先学一个表征再被动接到下游策略。

5.7 问题 47：Gaussian latent regularizer 在 LeWorldModel 中起什么作用？

5.7.1 通俗解释

Gaussian latent regularizer 的作用是给 latent space 加一个稳定的分布约束。没有约束时，embedding 可能分布很散、尺度漂移，导致多步 rollout 越滚越偏。把 latent 拉向近似 Gaussian 的结构后，模型更容易采样、插值和递推，也更不容易在长 horizon 中出现不可控漂移。

5.7.2 数学推导

设 encoder 输出 latent 分布为：

$$q_{\phi}(z_t | o_t) = \mathcal{N}(\mu_t, \sigma_t^2 I).$$

Gaussian regularizer 可写成:

$$\mathcal{R}_{gauss} = D_{KL}(q_\phi(z_t | o_t) \| \mathcal{N}(0, I)).$$

整体目标为:

$$\mathcal{L} = \mathcal{L}_{pred} + \lambda \mathcal{R}_{gauss}.$$

本题符号说明 其中, $q_\phi(z_t | o_t)$ 表示 encoder 产生的 latent 分布; μ_t 和 σ_t 表示均值和标准差; $\mathcal{N}(0, I)$ 表示标准 Gaussian prior; D_{KL} 表示 KL divergence; λ 表示正则项权重。

5.7.3 总结

Gaussian latent regularizer 的价值是让 latent space 更规整、更稳定、更适合 rollout。它不是为了增加复杂度, 而是为了降低端到端 world model 训练和长期预测中的表示漂移。

5.8 问题 48: Causal-JEPA 为什么强调 object-level masking?

5.8.1 通俗解释

Causal-JEPA 强调 object-level masking, 是因为因果关系通常发生在物体层面, 而不是随机 patch 层面。随机遮挡 patch 可能只训练局部补全能力; 遮挡一个完整物体或对象区域, 模型才需要理解该物体和其他物体之间的关系, 例如支撑、遮挡、接触和运动传递。这让表示更接近可干预的世界变量。

5.8.2 数学推导

设场景由 object tokens 组成:

$$Z_t = \{z_t^1, z_t^2, \dots, z_t^N\}.$$

对第 j 个对象做 masking 后, 模型根据其余对象预测目标对象表示:

$$\hat{z}_t^j = P_\theta(Z_t^{\setminus j}, pos_j).$$

object-level predictive loss 为:

$$\mathcal{L}_{obj} = \sum_j \left\| \hat{z}_t^j - \text{sg}(z_t^j) \right\|_2^2.$$

本题符号说明 其中, z_t^j 表示第 j 个对象的 latent token; $Z_t^{\setminus j}$ 表示去掉第 j 个对象后的上下文对象集合; pos_j 表示对象位置; P_θ 表示对象级 predictor; $\text{sg}(\cdot)$ 表示 stop-gradient。

5.8.3 总结

object-level masking 让模型从补 patch 转向理解对象关系。它更适合学习因果变量, 因为真实世界中的动作、遮挡、碰撞和交互大多发生在对象层面。

5.9 问题 49: Causal-JEPA 如何区分 correlation 和 causation?

5.9.1 通俗解释

correlation 只说明两个变量经常一起变化, causation 则要求改变一个变量会导致另一个变量发生相应变化。Causal-JEPA 的关键是引入对象级干预: 不只是观察对象共现, 而是改变、遮挡或替换某个 object token, 再检查其他对象和未来状态是否产生符合因果逻辑的变化。这样模型不能只靠数据相关性取巧。

5.9.2 数学推导

普通相关性建模关注:

$$p(z^i, z^j).$$

因果建模关注干预后的分布变化:

$$p(z_{t+1} \mid do(z_t^j = \tilde{z}^j)).$$

可定义干预敏感性:

$$S_j = d\left(p_\theta(z_{t+1} \mid Z_t), p_\theta(z_{t+1} \mid do(z_t^j = \tilde{z}^j))\right).$$

如果 S_j 与真实环境中的因果影响一致, 说明模型不仅学到相关性, 也学到一定因果结构。

本题符号说明 其中, z^i 和 z^j 表示不同对象表示; $do(\cdot)$ 表示因果干预; $\tilde{z}^j$ 表示被替换或干预后的对象表示; S_j 表示第 j 个对象的干预敏感性; $d(\cdot, \cdot)$ 表示分布差异度量。

5.9.3 总结

Causal-JEPA 区分 correlation 和 causation 的关键是看干预后模型是否产生正确变化。只会利用共现关系的模型是相关性预测器; 能响应对象级干预的模型才更接近因果世界模型。

5.10 问题 50: 为什么 object-level intervention 对可控世界模型重要?

5.10.1 通俗解释

可控世界模型不能只回答“未来会怎样”, 还要回答“如果我改变某个对象或动作, 未来会怎样变”。object-level intervention 提供了这种接口: 移动一辆车、拿走一个障碍物、推动一个杯子、改变一个行人的意图, 模型都应该产生对应的未来变化。这比整体视频条件生成更适合规划和反事实推理。

5.10.2 数学推导

设对象级状态为 $Z_t = \{z_t^1, \dots, z_t^N\}$, 对第 j 个对象施加干预 u^j :

$$\tilde{z}_t^j = I(z_t^j, u^j).$$

干预后的未来预测为:

$$p_\theta(Z_{t+1:t+H} \mid Z_t^{\setminus j}, \tilde{z}_t^j, a_{t:t+H}).$$

可控性可通过干预前后未来差异衡量:

$$D_{int} = d\left(p_\theta(Z' \mid Z_t), p_\theta(Z' \mid Z_t^{\setminus j}, \tilde{z}_t^j)\right).$$

本题符号说明 其中, Z_t 表示对象级 latent 集合; z_t^j 表示第 j 个对象; u^j 表示对象级干预; $I(\cdot)$ 表示干预函数; $a_{t:t+H}$ 表示未来动作序列; D_{int} 表示干预导致的未来分布差异。

5.10.3 总结

object-level intervention 让世界模型具备反事实控制能力。它把模型从“被动预测未来”推进到“主动改变世界变量并评估后果”，这是服务 planning、robotics 和 autonomous driving 的关键。

6 生成式世界模型、Genie 与交互式环境

本章主线 本章答题主线: 生成式世界模型要从视频生成走向实时交互环境, 核心考点是 action conditioning、persistent state、object permanence 和 controllability。

6.1 问题 51: 视频生成模型一定是世界模型吗?

6.1.1 通俗解释

不是。视频生成模型可以只学画面序列的统计分布——给定 prompt 输出一段看起来合理的视频, 但不支持动作控制、不维护环境状态、也无法做反事实比较。一个典型的反例是 text-to-video: 输入“一辆车在路上行驶”, 模型生成的车轨迹可能很自然, 但你无法在中途插入“刹车”指令让车停下, 也无法回到同一初始帧、换一个动作生成不同未来。这种模型本质上是条件视频采样器, 不是可被驱动的环境。

要让视频生成模型成为世界模型, 必须额外满足三件事: (1) **动作条件**——未来必须显式地由动作 a_t 决定, 不同 a 产生可区分的不同未来; (2) **状态持久性**——对环境的修改 (开门、移动物体) 在视角变化或时间流逝后仍要保持; (3) **交互闭环**——动作可以在线注入, 而不是一次性给定整段动作序列。少了任何一条, 模型最多是漂亮的视频续写器。

6.1.2 数学推导

普通视频生成的目标是无控制的条件分布:

$$p_\theta(x_{1:T} | c),$$

其中 c 是文本/图像条件, 整段视频一次性采样。世界模型必须把动作和持久状态显式纳入:

$$m_{t+1} = U_\theta(m_t, x_t, a_t), \quad x_{t+1} \sim G_\theta(m_{t+1}),$$

即“用状态 m_t 和动作 a_t 更新出 m_{t+1} , 再由 m_{t+1} 生成下一帧”。可以用三个判据形式化检验它是否真是世界模型:

$$\underbrace{D_{\text{act}} = d(p_\theta(x' | \text{do}(a)), p_\theta(x' | \text{do}(a'))) > 0,}_{\text{动作敏感}}$$

$$\underbrace{E_{\text{state}} = d(\phi(x_{\text{revisit}}), \phi(x_{\text{memory}})) \rightarrow 0,}_{\text{状态持久}}$$

$$\underbrace{\Delta t_{\text{response}} < \tau.}_{\text{交互延迟可接受}}$$

当 $D_{\text{act}} \rightarrow 0$ 时, 模型对动作不敏感, 本质就是视频生成器; 当 E_{state} 大时, 模型记不住已修改的世界; 只有三个指标同时达标, 它才“用起来像世界”。

本题符号说明 其中, $x_{1:T}$ 表示视频序列; c 表示生成条件; m_t 表示内部世界状态; U_θ 表示状态更新函数; G_θ 表示帧生成器; a_t 、 a 、 a' 表示动作; $\text{do}(\cdot)$ 表示因果干预; D_{act} 表示动作敏感度; E_{state} 表示状态一致性误差; $\Delta t_{\text{response}}$ 表示交互响应延迟; τ 表示延迟阈值; ϕ 表示场景特征提取器; $d(\cdot, \cdot)$ 表示距离度量。

6.1.3 总结

视频生成模型不一定是世界模型。只有当它的未来受动作驱动、保持环境状态、并支持在线交互闭环时, 它才从”内容生成”走向”世界建模”。生成式世界模型的门槛不只是画质, 更是动作条件、状态持久、反事实一致性和低延迟交互这一整套能力。

6.2 问题 52: Sora、Veo、Genie 和普通 text-to-video 模型有什么区别?

6.2.1 通俗解释

Sora、Veo 主要面向高质量长视频生成, 关注画面质量、镜头运动和叙事连贯性, 但通常没有实时交互接口。Genie 系列则把目标推到可实时探索的交互环境: 从文本或图像生成可被持续操作的世界, 并在交互过程中维持一定的一致性和响应性。Genie 类方法的意义在于从视频中学习可玩环境, 把被动观看变成可操作世界。

6.2.2 数学推导

普通 text-to-video 可写成:

$$x_{1:T} \sim p_\theta(x_{1:T} | l).$$

Sora、Veo 更强调高质量长视频生成, 而 Genie 类模型进一步引入交互动作:

$$x_{t+1} \sim p_\theta(x_{t+1} | x_{\leq t}, u_t, l).$$

关键差异在于条件变量是否包含用户或 agent 的实时控制 u_t 。

本题符号说明 其中, l 表示文本条件; $x_{1:T}$ 表示视频序列; $x_{\leq t}$ 表示历史帧; u_t 表示用户或智能体交互动作; p_θ 表示条件生成模型。

6.2.3 总结

Sora、Veo 更偏高质量长视频生成, Genie 更强调从输入中生成可交互环境。普通 text-to-video 主要满足观看, world-model-oriented generation 更关注用户或 agent 能否进入其中并改变世界。

6.3 问题 53: 什么是 interactive world generation?

6.3.1 通俗解释

interactive world generation 不是一次性生成视频, 而是在用户或 agent 移动、转向、操作时实时继续生成世界。它要求低延迟、可控性、空间连续性和状态记忆, 比长视频生成更接近可行行动模拟。在没有人工动作标注的情况下, 工程上常引入 action token, 把它当作从视频中挖出来的隐式控制接口, 使模型也能学习交互。

6.3.2 数学推导

交互式世界生成不是一次性采样完整视频，而是递归更新状态：

$$m_{t+1} = U_{\theta}(m_t, u_t, x_t), \quad x_{t+1} = G_{\theta}(m_{t+1}).$$

这里 m_t 保存环境记忆， u_t 改变环境演化方向。没有 m_t ，模型容易只生成局部连续画面，却无法维持长期一致的世界。

本题符号说明 其中， m_t 表示交互环境的内部记忆； u_t 表示用户或 agent 动作； x_t 表示当前画面； U_{θ} 表示状态更新函数； G_{θ} 表示从状态到画面的生成器。

6.3.3 总结

interactive world generation 指生成一个可被持续探索 and 操作的动态环境。它不是一次性生成视频，而是在用户动作输入后保持状态连续，并实时反馈新的未来。

6.4 问题 54：Genie 2 到 Genie 3 的核心能力变化是什么？

6.4.1 通俗解释

Genie 2 到 Genie 3 的变化，可以理解为从“短时可交互片段”走向“更长时间、更开放场景的实时世界”。能力边界的真正变化在于：交互时间更长、状态更稳定、场景更可探索、用户动作对未来生成有持续影响。它的难点不在于多生成几秒，而在于让环境在多轮操作之后仍然像是同一个世界。

6.4.2 数学推导

可以把 Genie 2 到 Genie 3 的变化理解为交互 horizon 和状态保持能力的提升：

$$p_{\theta}(x_{t+1:t+H} \mid x_{\leq t}, u_{t:t+H}).$$

当 H 增大时，模型必须同时控制延迟、记忆和一致性。若重新访问同一位置，状态应满足：

$$d(\phi(x_{visit1}), \phi(x_{visit2})) < \epsilon.$$

本题符号说明 其中， H 表示可持续交互的时间长度； $u_{t:t+H}$ 表示未来交互动作； $\phi(\cdot)$ 表示用于比较场景状态的特征提取器； d 表示差异度量； ϵ 表示允许误差。

6.4.3 总结

从 Genie 2 到 Genie 3，关键变化在于交互能力、实时性和状态持久性的增强。模型不只是生成一段场景，而是更接近一个可以连续探索的神经环境。

6.5 问题 55：Genie 3 为什么被称为 general-purpose world model？

6.5.1 通俗解释

Genie 3 被称为 general-purpose world model，核心原因是它不只针对某个游戏、某个机器人任务或某个固定场景，而是试图从文本出发生成可探索、可交互、可持续的环境。general-purpose 的关键不是“什么都能画”，而是不同场景都能支持 agent 或用户在其中行动，并让世界随行动持续演化。

6.5.2 数学推导

general-purpose world model 应该支持不同任务条件：

$$p_{\theta}(x_{t+1}, m_{t+1} \mid x_{\leq t}, m_t, u_t, c),$$

其中 c 可以是文本、图像、环境类型或任务目标。它的“通用性”不只是能生成很多场景，而是同一模型能在多种场景中持续响应交互。

本题符号说明 其中， m_t 表示内部世界记忆； u_t 表示交互动作； c 表示任务或场景条件； x_{t+1} 表示下一帧； m_{t+1} 表示更新后的世界状态。

6.5.3 总结

Genie 3 被称为 general-purpose world model，是因为它不局限于某个固定游戏、机器人任务或驾驶场景。它试图从通用视觉经验中生成多样、可交互、可持续演化的环境。

6.6 问题 56：Project Genie 为什么说明 world model 正在走向用户可交互产品？

6.6.1 通俗解释

Project Genie 说明 world model 正在从研究 demo 走向用户可交互产品。过去的视频生成更像内容生产工具，用户输入 prompt 后等待一段视频；交互式 world model 则让用户在生成世界里移动、探索和改变环境。这意味着评估标准也会变化：不仅看画质，还要看响应延迟、状态记忆、可控性和持续交互体验。

6.6.2 数学推导

用户可交互产品需要低延迟闭环：

$$u_t \rightarrow m_{t+1} = U_{\theta}(m_t, u_t) \rightarrow x_{t+1} = G_{\theta}(m_{t+1}).$$

产品形态的关键指标不只是生成质量，还包括响应时间：

$$\Delta t_{\text{response}} < \tau,$$

其中 τ 是用户可接受延迟阈值。

本题符号说明 其中， u_t 表示用户输入； m_t 表示世界状态； U_{θ} 表示状态更新； G_{θ} 表示画面生成； $\Delta t_{\text{response}}$ 表示交互响应延迟； τ 表示延迟阈值。

6.6.3 总结

Project Genie 说明 world model 正在从论文里的预测模块走向用户可体验的交互产品。当普通用户能在生成世界中移动、探索和改变状态时，世界模型就具备了产品化入口。

6.7 问题 57: real-time exploration 为什么比 long video generation 更接近 world model?

6.7.1 通俗解释

real-time exploration 比 long video generation 更接近 world model, 因为它要求模型在线接收动作并立即更新世界。长视频可以离线生成, 甚至提前规划完整时间线; 实时探索不能提前知道用户下一步会做什么, 所以必须维护内部状态、快速响应动作, 并在多轮交互中保持世界一致。

6.7.2 数学推导

long video generation 可以一次性采样:

$$x_{1:T} \sim p_\theta(x_{1:T} \mid c).$$

real-time exploration 必须逐步响应动作:

$$x_{t+1} \sim p_\theta(x_{t+1} \mid x_{\leq t}, u_t, m_t).$$

二者差异在于后者要求因果在线生成, 不能提前知道完整动作序列。

本题符号说明 其中, c 表示初始条件; u_t 表示实时交互动作; m_t 表示内部状态记忆; $x_{\leq t}$ 表示历史画面; p_θ 表示生成模型。

6.7.3 总结

real-time exploration 比 long video generation 更接近 world model, 因为它要求模型对用户动作即时响应。长视频可以离线生成, 实时探索必须在闭环中维护状态、接受干预并持续生成后果。

6.8 问题 58: persistent state 为什么是 interactive world 的核心?

6.8.1 通俗解释

persistent state 是 interactive world 的核心, 因为交互环境必须记住此前发生过什么。门被打开、物体被移动、角色走到某个位置, 这些状态在视角变化或一段时间之后仍然要保持一致; 否则它就只是连续视频, 而不是一个可被操作的世界。也正因如此, 评估交互式世界模型时, 除了画质, 还要看 action controllability、consistency、coverage 和 downstream usefulness。

6.8.2 数学推导

persistent state 可以表示为记忆变量递推:

$$m_{t+1} = U_\theta(m_t, o_t, u_t).$$

如果同一对象在 t 时刻离开视野, 在 $t+k$ 时刻重新出现, 模型应保持身份一致:

$$id_{t+k} = id_t.$$

这比短期帧间平滑更难, 因为它要求跨时间保存世界状态。

本题符号说明 其中, m_t 表示持久世界状态; o_t 表示当前观测; u_t 表示交互动作; id_t 表示对象身份; k 表示重新出现前的时间间隔。

6.8.3 总结

persistent state 是 interactive world 的核心, 因为交互环境必须记住之前发生过什么。门被打开、物体被移动、角色走到某处, 这些状态如果不能持续存在, 世界就只是连续视频而不是环境。

6.9 问题 59: object permanence 为什么是世界模型能力的重要指标?

6.9.1 通俗解释

object permanence 指物体即使被遮挡或暂时离开视野, 它在世界中的存在、身份、位置仍应被模型保持下来。没有这种能力, 模型就只是连续画面生成器, 物体一旦看不见就被“忘掉”; 有了 object permanence, agent 才能在长期交互中把环境当作可记忆、可推理的世界。这也正是把 Genie 3 等系统称为 general-purpose interactive environment 的关键理由——它不是更强的视频生成器, 而是会维护对象级记忆的可操作世界。

6.9.2 数学推导

object permanence 可写成遮挡变量下的状态保持:

$$p_{\theta}(s_{t+k}^i \mid s_t^i, \eta_{t:t+k}^i = 0),$$

其中 $\eta^i = 0$ 表示对象不可见。若对象重新出现, 预测状态应接近真实状态:

$$d(\hat{s}_{t+k}^i, s_{t+k}^i) < \epsilon.$$

本题符号说明 其中, s_t^i 表示第 i 个对象状态; η_t^i 表示可见性; $\hat{s}_{t+k}^i$ 表示模型维护的对象状态; d 表示状态差异; ϵ 表示容忍误差。

6.9.3 总结

object permanence 检验模型是否知道物体即使暂时不可见也仍然存在。对世界模型来说, 这比视觉连续更重要, 因为遮挡后的物体位置会影响未来行动和安全判断。

6.10 问题 60: 如何判断一个 video generator 是否真正具备 world model 能力?

6.10.1 通俗解释

画质漂亮、镜头连贯、运动自然——这些指标只能说明 video generator 学到了视频分布, 但不足以证明它是世界模型。要做严格判断, 需要从五个层面检验:

(1) **动作可控性**——同样初始帧、不同动作输入, 是否产生明显不同的未来; (2) **状态持久性**——开门、移物体后离开再回来, 门是否还开着、物体是否还在新位置; (3) **物理一致性**——是否出现穿模、瞬移、违反重力或动力学; (4) **反事实一致性**——同一初始状态下两个动作的差异是否符合因果逻辑; (5) **长 horizon 稳定性**——交互几十秒后画面是否仍然连贯且符合规则。

只通过其中两三条还不够。许多生成模型在短期可控性上看起来不错 (动作敏感度有), 但状态持久性差 (过几秒就忘记之前发生了什么), 仍然不能用于决策或仿真。完整的 world model 必须五条都达标。

6.10.2 数学推导

对每条检验定义可量化指标：

$$D_{\text{act}}(a, a') = d(p_{\theta}(x' | x_{\leq t}, a), p_{\theta}(x' | x_{\leq t}, a')),$$

$$E_{\text{state}} = d(\phi(x_{\text{revisit}}), \phi(x_{\text{memory}})),$$

$$E_{\text{phys}} = \sum_m \lambda_m \max(0, -v_m(x_t, x_{t+1})),$$

$$E_{\text{cf}} = |D_{\text{act}}^{\text{model}}(a, a') - D_{\text{act}}^{\text{env}}(a, a')|,$$

$$E_{\text{long}} = \sup_{H \leq H_{\text{max}}} \mathbb{E} [\|\phi(x_{t+H}) - \phi(x_{t+H}^{\text{ref}})\|].$$

一个合格的 world model 需要同时满足： D_{act} 显著大于零、 $E_{\text{state}} \rightarrow 0$ 、 $E_{\text{phys}} \rightarrow 0$ 、 E_{cf} 与真实环境一致、 E_{long} 在长 horizon 下不爆炸。任何一项明显不达标，模型就只能算视频生成器，不能用作环境仿真器。

本题符号说明 其中， $x_{\leq t}$ 表示历史； x' 、 x_{t+1} 表示未来帧； a 、 a' 表示动作； x_{revisit} 表示重新访问场景的画面； x_{memory} 表示模型记忆的画面； x_{t+H}^{ref} 表示参考真值； ϕ 表示场景特征提取器； v_m 表示物理约束函数； D_{act} 、 E_{state} 、 E_{phys} 、 E_{cf} 、 E_{long} 分别为五项检验指标； $d(\cdot, \cdot)$ 表示距离； H_{max} 表示长程检验的最大 horizon。

6.10.3 总结

判断 video generator 是否具备 world model 能力，不能只看画质或单条指标。更关键的是动作可控性、状态持久性、物理一致性、反事实一致性和长 horizon 稳定性五项同时达标。这些指标共同区分“会画”与“会算”——前者是漂亮的视频，后者才是可被驱动的世界。

7 World Foundation Model、Cosmos 与 Physical AI

本章主线 本章答题主线：World Foundation Model 是 Physical AI 的基础设施，不只是一个大视频模型，而是数据、仿真、控制、评估、安全的统一底座。

7.1 问题 61：什么是 World Foundation Model?

7.1.1 通俗解释

World Foundation Model 是可迁移、可后训练、可控的通用世界模拟底座。它不是为单一任务训练一个小模型，而是像基础模型一样服务多个 Physical AI 场景，例如自动驾驶、机器人、工业仿真和长尾数据生成。可以把它理解为物理世界的基础模型，目标是服务多任务、多场景、多形态的 agent。

7.1.2 数学推导

World Foundation Model 可以抽象为跨任务、跨模态的条件世界分布：

$$p_{\theta}(\tau_{\text{world}} | c, m, d),$$

其中 c 是任务条件, m 是输入模态, d 是目标域。它的价值在于同一模型可通过后训练适配多个 Physical AI 场景:

$$\theta_d = \text{Adapt}(\theta, \mathcal{D}_d).$$

本题符号说明 其中, τ_{world} 表示生成或模拟出的世界轨迹; c 表示任务条件; m 表示文本、图像或视频等输入模态; d 表示领域; θ_d 表示适配到特定领域后的模型参数。

7.1.3 总结

World Foundation Model 是面向多任务、多场景、多模态物理世界预测的基础模型。它不是某个任务的小 dynamics model, 而是可被定制、评估和复用的世界建模底座。

7.2 问题 62: World Foundation Model 和传统 task-specific world model 有什么区别?

7.2.1 通俗解释

传统 task-specific world model 通常服务一个固定任务, 例如某个游戏、某个机器人环境或某类驾驶场景。World Foundation Model 则像基础模型一样, 先学习通用物理世界先验, 再通过后训练适配不同任务。前者强调专用性能, 后者强调迁移能力、可定制能力和平台化能力。

7.2.2 数学推导

task-specific world model 学一个固定任务分布:

$$p_{\theta_i}(\tau \mid c_i, d_i).$$

World Foundation Model 学一个可迁移底座:

$$p_{\theta}(\tau \mid c, d, m), \quad \theta_i = \text{Tune}(\theta, \mathcal{D}_i).$$

区别在于前者重训练, 后者强调复用和后训练。

本题符号说明 其中, c_i 和 d_i 表示第 i 个任务和领域; θ_i 表示任务专用模型; θ 表示基础模型; $\mathcal{D}_i$ 表示任务数据。

7.2.3 总结

传统 task-specific world model 通常只服务一个任务或环境, World Foundation Model 追求跨任务复用和平台化定制。它更像物理 AI 的基础设施, 而不是某个策略旁边的小辅助模块。

7.3 问题 63: 为什么 Physical AI 需要 world foundation model?

7.3.1 通俗解释

Physical AI 需要 world foundation model, 是因为真实机器人和自动驾驶系统面对的是开放、连续、昂贵且有风险的物理世界。每个任务都从零采数据、训练模拟器和做安全测试成本很高。World Foundation Model 提供一个可复用的物理世界底座, 让系统可以生成长尾场景、预测动作后果, 并在部署前进行闭环评估。

7.3.2 数学推导

Physical AI 的闭环可写成：

$$o_t \rightarrow z_t \rightarrow \hat{z}_{t+1:t+H} \rightarrow a_t.$$

World Foundation Model 提供中间的未来模拟：

$$\hat{z}_{t+1:t+H} \sim p_\theta(\cdot \mid z_t, a_{t:t+H}, c).$$

没有这一步，Physical AI 只能在真实环境中昂贵试错。

本题符号说明 其中， o_t 表示观测； z_t 表示内部状态； $a_{t:t+H}$ 表示候选动作序列； $\hat{z}_{t+1:t+H}$ 表示预测未来； c 表示任务条件。

7.3.3 总结

Physical AI 需要 world foundation model，是因为真实机器人、自动驾驶和具身 agent 都面对长尾、风险和昂贵试错。基础世界模型可以用合成场景和内部模拟降低真实世界训练成本。

7.4 问题 64：NVIDIA Cosmos 为什么是 world foundation model platform?

7.4.1 通俗解释

NVIDIA Cosmos 这类系统之所以更像 world foundation model platform，是因为它不只是一个视频生成模型，而是围绕 Physical AI 提供一整套能力：世界生成、条件控制、后训练、合成数据生产、reasoning/evaluator 评估和安全 guardrails。平台化的核心是让机器人和自动驾驶开发者能把它接入训练、测试和验证闭环。

7.4.2 数学推导

Cosmos 类平台可以拆成三层：世界生成器、后训练工具和评估器。形式上：

$$\tau = G_\theta(c, m, d), \quad \theta' = \text{PostTrain}(\theta, \mathcal{D}_{\text{target}}),$$

$$\text{score} = E_\psi(\tau, \mathcal{C}_{\text{phys}}, \mathcal{C}_{\text{task}}).$$

平台价值来自生成、定制和评估的组合，而不是单一视频模型。

本题符号说明 其中， G_θ 表示世界生成模型； c, m, d 分别表示任务条件、输入模态和领域； E_ψ 表示 evaluator； $\mathcal{C}_{\text{phys}}$ 和 $\mathcal{C}_{\text{task}}$ 分别表示物理与任务约束。

7.4.3 总结

NVIDIA Cosmos 的定位不是单一视频模型，而是面向 Physical AI 的世界模型平台。它把生成、预测、后训练、数据合成、评估和 guardrails 组合成一套可用于机器人和自动驾驶的基础设施。

7.5 问题 65: Cosmos Predict 如何统一 Text2World、Image2World、Video2World?

7.5.1 通俗解释

Cosmos Predict 的统一性体现在条件输入不再固定为一种模态。Text2World 用语言描述任务、天气、道路或机器人场景；Image2World 用图像指定初始状态；Video2World 用历史视频提供动态上下文。统一模型把这些条件映射到同一个世界模拟空间，因此更适合 Physical AI 的 post-training 和长尾场景生成。

7.5.2 数学推导

可以把三种输入统一为条件变量 m :

$$m \in \{l, I_0, x_{1:t}\}.$$

统一世界预测写成:

$$p_\theta(x_{t+1:t+H} \mid m, c, d),$$

其中 c 是任务条件, d 是域条件。三种模式的区别只在条件编码器, 主干世界动态共享:

$$z_m = E(m), \quad x_{t+1:t+H} \sim G_\theta(z_m, c, d).$$

本题符号说明 其中, m 表示统一的输入模态条件 (文本 l 、初始图像 I_0 或历史视频 $x_{1:t}$); c 表示任务条件; d 表示领域条件; z_m 表示模态编码; E 表示条件编码器; G_θ 表示世界生成器; H 表示未来 rollout 的步数。

7.5.3 总结

Cosmos Predict 统一 Text2World、Image2World、Video2World, 说明世界生成不再只是从文本到视频。不同条件形式对应不同使用场景: 文本控制语义, 图像控制初始状态, 视频控制历史动态。

7.6 问题 66: Cosmos-Predict2.5 为什么结合 Cosmos-Reason1?

7.6.1 通俗解释

Cosmos-Predict2.5 负责模拟未来世界, Cosmos-Reason1 则提供更强物理语义理解和文本 grounding。单纯视频生成模型可能知道“画面应该动”, 但不一定理解 prompt 中的物理关系、任务约束和安全意图; Reason 模块相当于把语言条件变成更可靠的世界控制信号。可控生成是 World Foundation Model 的核心接口, 因为工程系统需要指定场景、动作、天气、风险和目标, 而不是接受任意采样。

7.6.2 数学推导

设文本条件为 l , reasoning encoder 给出物理语义表示:

$$r = R_\psi(l, I_t).$$

世界生成器再条件化于该表示:

$$p_\theta(x_{t+1:t+H} \mid x_{\leq t}, r).$$

如果 r 编码了物体、动作和约束, 则生成过程更容易满足:

$$C_{phys}(x_t, x_{t+1}) \leq 0.$$

本题符号说明 其中, l 表示语言指令; I_t 表示当前图像; r 表示 reasoning 提取的物理语义表示; R_ψ 表示 reasoning encoder (Cosmos-Reason1); $x_{t+1:t+H}$ 表示生成的未来视频; C_{phys} 表示物理约束; H 表示生成 horizon。

7.6.3 总结

Cosmos-Predict2.5 结合 Cosmos-Reason1 的意义在于把世界生成和世界推理连接起来。生成模型负责产生未来, reasoning 模型帮助理解、筛选和评估这些未来是否满足任务与物理约束。

7.7 问题 67: Physical AI Data Factory 如何生成 long-tail synthetic data?

7.7.1 通俗解释

Physical AI 真正缺的不是常见数据, 而是危险、罕见、极端的场景。Physical AI Data Factory 的思路是把 world model 当成可控合成器: 先指定罕见条件 (恶劣天气、行人异常行为、关键事故等), 再生成对应轨迹, 最后用 evaluator 筛选出对策略有挑战性的样本。这套流程的目标不是生成更多好看的视频, 而是系统化地暴露系统薄弱点。

7.7.2 数学推导

long-tail synthetic data factory 的流程是先采样罕见条件:

$$c_{tail} \sim p(c \mid rarity, risk),$$

再生成对应世界轨迹:

$$\tau_{tail} \sim p_\theta(\tau \mid c_{tail}, d).$$

最后用 evaluator 过滤:

$$\mathcal{D}_{syn} = \{\tau : E(\tau) > \delta\}.$$

本题符号说明 其中, c_{tail} 表示长尾场景条件; $rarity$ 和 $risk$ 分别表示稀有度与风险; τ_{tail} 表示合成轨迹; $E(\tau)$ 表示质量或安全评估分数; δ 表示筛选阈值。

7.7.3 总结

Physical AI Data Factory 的价值在于系统化制造真实数据中稀缺的长尾场景。它不是简单扩增数据, 而是围绕风险、罕见事件和策略弱点生成可训练、可评估的合成经验。

7.8 问题 68: 为什么 world model 需要 guardrails 和 evaluator?

7.8.1 通俗解释

world model 越强, 越需要 guardrails 和 evaluator——因为它生成的世界可能被用于训练策略, 一旦模拟违反物理、安全或任务约束, 错误会被策略继承并放大。两者的分工不同: guardrail 限制生成和使用的边界, evaluator 判断生成的未来是否可信、危险或违反物理规律。世界模型的完整数据闭环应包括真实数据、合成数据、策略训练、评估反馈和再采样, guardrail 与 evaluator 结合才构成可部署闭环。

7.8.2 数学推导

guardrail 可以写成约束集合:

$$\mathcal{C} = \{C_{phys}, C_{safety}, C_{domain}, C_{task}\}.$$

evaluator 对生成轨迹打分:

$$score = E_{\psi}(\tau, \mathcal{C}).$$

若 $score < \delta$, 生成结果应被拒绝、重采样或进入人工检查。

本题符号说明 其中, C_{phys} 、 C_{safety} 、 C_{domain} 、 C_{task} 分别表示物理、安全、领域和任务约束; E_{ψ} 表示评估器; τ 表示世界轨迹; δ 表示通过阈值。

7.8.3 总结

world model 需要 guardrails 和 evaluator, 因为生成的未来可能视觉合理但物理错误、任务无效或安全危险。guardrails 约束生成边界, evaluator 判断结果是否可用于训练和决策。

7.9 问题 69: closed-loop policy training 为什么需要 world simulation?

7.9.1 通俗解释

开环训练只在固定数据集上评估策略, 但策略一旦执行不同动作, 后续观测分布就会改变。closed-loop policy training 需要 world simulation, 是因为只有在能响应动作的环境里反复 rollout, 才能暴露策略的连锁后果——这正是世界模型与静态 LLM 评估最根本的区别: 它面对的是连续时空、物理约束和动作干预, 而不是固定 prompt。

7.9.2 数学推导

closed-loop policy training 需要在模拟世界中反复执行:

$$a_t = \pi_{\psi}(z_t), \quad z_{t+1} \sim p_{\theta}(z_{t+1} \mid z_t, a_t).$$

策略目标为:

$$J(\psi) = \mathbb{E}_{p_{\theta}, \pi_{\psi}} \left[\sum_{t=0}^T \gamma^t r_t \right].$$

关键是 policy 的动作会改变后续状态, 而不是只在静态数据上评估。

本题符号说明 其中, π_{ψ} 表示策略; z_t 表示世界状态; p_{θ} 表示 world simulator; $J(\psi)$ 表示策略目标; γ 表示折扣因子; r_t 表示奖励。

7.9.3 总结

closed-loop policy training 需要 world simulation, 因为策略的动作会改变后续观测分布。开环预测只能评估模型对已有轨迹的拟合, 闭环模拟才能暴露策略执行后的真实连锁后果。

7.10 问题 70: World Foundation Model 的核心评估指标是什么?

7.10.1 通俗解释

World Foundation Model 的评估不能只看 FVD 这类视频生成指标。它的真正价值是支撑下游 Physical AI 系统, 因此评估必须覆盖多个维度: 视觉质量、物理一致性、动作可控性、反事实一致性、长时稳定性、安全失败率, 以及最关键的——下游策略或感知模型在使用合成数据/模拟环境后是否真的获得收益。任何一个维度短板, 都会让模型在产品化时露出问题。

7.10.2 数学推导

World Foundation Model 的评估可以分成生成质量、物理一致性、动作可控性和迁移收益:

$$Score = \alpha Q_{visual} + \beta Q_{phys} + \gamma Q_{ctrl} + \eta Q_{transfer}.$$

其中 $Q_{transfer}$ 应该通过下游 policy 或感知模型收益验证, 而不是只看视频指标。

本题符号说明 其中, Q_{visual} 表示视觉质量; Q_{phys} 表示物理一致性; Q_{ctrl} 表示动作可控性; $Q_{transfer}$ 表示下游迁移收益; $\alpha, \beta, \gamma, \eta$ 表示权重。

7.10.3 总结

World Foundation Model 的评估不能只看 FVD 或视觉质量。核心指标应包括物理一致性、动作可控性、反事实一致性、长时稳定性、下游策略收益和安全失败率。

8 自动驾驶世界模型

本章主线 本章答题主线: 自动驾驶世界模型要围绕安全相关的未来展开, 包括多视角、BEV、occupancy、agent interaction、反事实和长尾场景。

8.1 问题 71: 为什么自动驾驶需要世界模型?

8.1.1 通俗解释

自动驾驶最难的不是识别当前帧上有什么, 而是回答”我做出某个决定后, 未来几秒交通场景会怎样演化”。检测模型告诉你”前方 30 米有一辆车”, 但 planner 真正需要的是”如果我加速并线, 3 秒后我和这辆车的位置关系会怎样、和后方车会不会冲突、行人会不会突然横穿”。这是一个动作条件下的多 agent 预测问题, 正是世界模型擅长的。

它在自动驾驶里有四个不可替代的角色: (1) **安全规划**——在内部 rollout 中比较”刹车 / 维持 / 变道”等候选动作的风险; (2) **长尾测试**——生成真实路测难以采到的恶劣天气、施工、异常行为场景; (3) **反事实分析**——同一场景下评估”如果我提前刹车会不会避免碰撞”, 用于复盘和监管; (4) **弱监督训练**——驾驶数据中动作标签远少于视频帧数, 用未来预测作为辅助任务可以从海量未标注视频中学到 dynamics。这些能力是单纯检测、跟踪、轨迹预测模型做不到的。

8.1.2 数学推导

自动驾驶世界状态由多视角图像、地图先验、交通参与者状态共同编码:

$$z_t = f_\phi(I_t^{1:N}, M_t, E_t, P_t),$$

其中 $I_t^{1:N}$ 是 N 路相机, M_t 是高精地图, E_t 是交通参与者状态, P_t 是路线先验。动作条件下的未来分布为:

$$p_\theta(z_{t+1:t+H} \mid z_t, a_{t:t+H}^{\text{ego}}).$$

规划阶段, agent 在 imagined rollout 上最小化加权代价:

$$a_{t:t+H}^* = \arg \min_{a_{t:t+H}^{\text{ego}}} \mathbb{E}_{p_\theta} \left[\sum_{k=1}^H \gamma^k (\lambda_c C_{\text{coll}} + \lambda_r C_{\text{rule}} + \lambda_g C_{\text{goal}} + \lambda_s C_{\text{smooth}}) \right].$$

反事实分析则比较两个动作下的代价差:

$$\Delta C(a, a') = \mathbb{E}_{p_\theta}[C \mid \text{do}(a)] - \mathbb{E}_{p_\theta}[C \mid \text{do}(a')],$$

如果 $\Delta C < 0$, 说明 a 比 a' 更安全, 可作为系统决策的 explainability 依据。

本题符号说明 其中, $I_t^{1:N}$ 表示多视角相机输入; M_t 表示高精地图; E_t 表示交通参与者状态; P_t 表示路线先验; z_t 表示驾驶场景 latent; $a_{t:t+H}^{\text{ego}}$ 表示自车未来动作; C_{coll} 、 C_{rule} 、 C_{goal} 、 C_{smooth} 分别表示碰撞、规则、目标、平滑代价; λ_* 是各项权重; ΔC 表示反事实代价差; $\text{do}(\cdot)$ 表示因果干预。

8.1.3 总结

自动驾驶需要世界模型, 不是为了替代检测、分割、跟踪, 而是把这些感知结果变成”动作条件下的未来推演”, 从而支撑安全规划、长尾测试、反事实复盘和弱监督训练。它的核心价值是把”现在看到什么”翻译为”未来会怎样”, 让车在行动前先在内部走一遍可能的世界。

8.2 问题 72: 自动驾驶世界模型和普通感知模型有什么区别?

8.2.1 通俗解释

普通感知模型 (detection、segmentation、tracking) 回答”当前场景里有什么”——前方 30 米有一辆白色卡车、左侧有行人、车道线在哪里。它的输入是当前帧、输出是结构化的当前状态, 主要服务于”看清楚”。

自动驾驶世界模型则要回答”这些东西接下来会怎样变化、我的动作会如何改变它们”。它输入的是历史观测 + 候选动作, 输出的是动作条件下的未来分布——卡车会刹车还是加速? 行人会不会突然横穿? 我现在变道 3 秒后会不会冲突? 这种”动作条件未来”是普通感知模型完全不做的事。

两者的关系不是替代而是接力: world model 通常以感知输出为输入, 把检测到的 agent、车道、占用作为初始状态, 再在其上做动作条件 rollout。可以理解为: 感知告诉系统”现在世界长什么样”, world model 告诉系统”我做出某个决定后, 世界会变成什么样”。前者面向识别, 后者面向决策与反事实, 这正是它和规划系统强相关的根本原因。

8.2.2 数学推导

普通感知模型给出当前状态的结构化估计:

$$y_t = f_\phi(I_t), \quad y_t \in \{\text{boxes, masks, tracks, lanes}\}.$$

自动驾驶世界模型在融合多视角和地图后给出动作条件未来:

$$z_t = g_\phi(I_t^{1:N}, M_t, y_t), \quad p_\theta(z_{t+1:t+H} \mid z_t, a_{t:t+H}^{\text{ego}}).$$

两者的对比可由对动作的依赖量化：感知模型的输出对 ego action 不变，

$$\frac{\partial y_t}{\partial a_t^{\text{ego}}} = 0,$$

而 world model 必须有非零依赖：

$$\frac{\partial p_\theta(z_{t+k} \mid z_t, a^{\text{ego}})}{\partial a^{\text{ego}}} \neq 0.$$

在规划阶段，world model 进一步给出动作下的代价差：

$$\Delta C(a, a') = \mathbb{E}_{p_\theta}[C \mid z_t, a] - \mathbb{E}_{p_\theta}[C \mid z_t, a'],$$

这是反事实推理的形式化基础——感知模型无法给出这种比较。

本题符号说明 其中， I_t 、 $I_t^{1:N}$ 表示单视角与多视角图像； y_t 表示感知输出； M_t 表示地图； z_t 表示驾驶 latent； f_ϕ 表示感知网络； g_ϕ 表示融合编码器； p_θ 表示世界模型； a_t^{ego} 、 a 、 a' 表示自车动作； C 表示规划代价； ΔC 表示反事实代价差； H 表示规划 horizon。

8.2.3 总结

普通感知模型看当前，自动驾驶世界模型看未来；前者回答”识别问题”，后者回答”动作后果问题”。两者不是替代而是接力——感知输出常作为 world model 的初始状态，world model 再在其上做动作条件 rollout 以服务规划与反事实复盘。判别两者的形式化指标，是输出是否对 ego action 有非平凡的依赖。

8.3 问题 73：future physical world generation 在自动驾驶中解决什么问题？

8.3.1 通俗解释

future physical world generation 解决的是“未来交通场景如何物理演化”的问题。它不是简单补帧，而是要生成满足道路结构、车辆动力学、多智能体交互和安全约束的未来。对自动驾驶来说，这种能力可以用于长尾数据生成、planner 评估、危险场景回放和反事实测试。

8.3.2 数学推导

给定当前驾驶状态和候选 ego 动作：

$$\hat{W}_{t+1:t+H} = G_\theta(W_t, a_{t:t+H}^{\text{ego}}, M_t),$$

其中 $\hat{W}$ 可以是未来多视角视频、BEV occupancy 或 agent trajectories。安全评估可写成：

$$R(a) = \sum_{k=1}^H \left[\lambda_c C_{\text{collision}}(\hat{W}_{t+k}) + \lambda_v C_{\text{violation}}(\hat{W}_{t+k}) + \lambda_g C_{\text{goal}}(\hat{W}_{t+k}) \right].$$

本题符号说明 其中， W_t 表示当前驾驶世界状态； G_θ 表示 future world generator； $a_{t:t+H}^{\text{ego}}$ 表示自车未来动作； M_t 表示地图约束； $C_{\text{collision}}$ 、 $C_{\text{violation}}$ 、 C_{goal} 分别表示碰撞、交通规则和目标代价； λ_c 、 λ_v 、 λ_g 是权重。

8.3.3 总结

future physical world generation 的价值是把自动驾驶测试从“看当前”推进到“看未来”。它可以让系统提前评估不同动作下的风险，而不是等真实危险发生后才知道策略错了。

8.4 问题 74: image-based、BEV-based、occupancy-based、point-cloud-based world model 各有什么优缺点？

8.4.1 通俗解释

不同表示对应不同的“看世界”方式：image-based 直接预测多视角图像，保留纹理但几何不稳定；BEV-based 在鸟瞰图上建模，最贴近规划坐标系，但依赖准确的视角投影；occupancy-based 把空间离散为占用网格，最方便做安全和碰撞分析；point-cloud-based 直接处理 LiDAR 点集，几何精确但稀疏且计算昂贵。选择哪一种，取决于下游任务是要规划、感知还是仿真。

8.4.2 数学推导

image-based 表达保留纹理：

$$z_t^{img} = E_{img}(I_t^{1:N}).$$

BEV 表达适合规划：

$$z_t^{bev} = E_{bev}(I_t^{1:N}, M_t).$$

occupancy 表达空间占用：

$$O_t(x, y, z) \in [0, 1].$$

point-cloud 表达几何精确点集：

$$P_t = \{(x_i, y_i, z_i, r_i)\}_{i=1}^N.$$

优缺点来自信息保真度、规划友好性、计算成本和跨视角一致性的权衡。

本题符号说明 其中， $I_t^{1:N}$ 表示多视角图像； M_t 表示地图； O_t 表示三维占用概率； P_t 表示点云； r_i 表示反射强度或点特征； E_{img} 和 E_{bev} 表示编码器。

8.4.3 总结

image-based 模型直观但几何不稳定，BEV-based 模型适合规划但依赖准确投影，occupancy-based 模型强调空间占据和安全，point-cloud-based 模型几何真实但稀疏昂贵。选择哪种表示取决于最终服务感知、预测还是规划。

8.5 问题 75: 4D occupancy forecasting 为什么重要？

8.5.1 通俗解释

单帧 3D occupancy 只回答“现在哪些空间被占据”，但驾驶要安全，关心的是“未来几秒哪些空间会被占据”——这就是 4D occupancy (3D 空间 + 时间) 的目标。它的吸引力在于既绕开了传统 detection-tracking-prediction 流水线的级联误差（不需要先把世界拆成有限个物体），又比未来图像更贴近规划：planner 需要的就是“自行车轨迹会不会撞到任何 voxel”这种几何级查询。

它特别适合处理两类传统方法的痛点：(1) 异形物体——掉落货物、突然飞起的塑料袋，难以分类却必须避让；(2) 部分遮挡——只能看到一角的车辆、半掩在卡车后的行人。occupancy 不要求模型给出准确的 bounding box 或类别，只要“这块空间未来可能不安全”就足以驱动避

让。和 BEV 相比，它额外保留了高度信息，可以正确处理高架、桥梁、隧道、限高这类垂直结构。

8.5.2 数学推导

设三维空间被离散化为 voxel 网格 $\mathcal{V} \subset \mathbb{R}^3$ 。当前占用图为 $O_t : \mathcal{V} \rightarrow [0, 1]$ ，每个 voxel 给出占据概率。4D occupancy forecasting 在自车动作条件下递推未来占用：

$$O_{t+k} = F_\theta(O_{\leq t}, a_{t:t+k-1}^{\text{ego}}), \quad k = 1, \dots, H.$$

训练目标是 voxel-wise 的二元交叉熵在时空上的求和：

$$\mathcal{L}_{4D} = \sum_{k=1}^H \sum_{v \in \mathcal{V}} \text{BCE}(O_{t+k}(v), O_{t+k}^{\text{gt}}(v)).$$

在规划阶段，自车未来轨迹 $\{(x_k, y_k, z_k)\}_{k=1}^H$ 与未来 occupancy 的卷积给出碰撞代价：

$$C_{\text{coll}}(\{a_{t:t+H}\}) = \sum_{k=1}^H \gamma^k \sum_{v \in B(\text{ego}_k)} O_{t+k}(v),$$

其中 $B(\text{ego}_k)$ 是自车在第 k 步占据的 voxel 集合。和 4D 形式相比，单帧 occupancy 假设 $O_{t+k} \equiv O_t$ ，对动态物体几乎没有预测能力。

本题符号说明 其中， $\mathcal{V}$ 表示 voxel 网格； O_t, O_{t+k} 表示当前与未来 occupancy； $a_{t:t+k-1}^{\text{ego}}$ 表示自车动作序列； F_θ 表示占用预测网络；BCE 表示二元交叉熵； O_{t+k}^{gt} 表示真值； $B(\text{ego}_k)$ 表示自车在第 k 步的占据 voxel 集合； C_{coll} 表示碰撞代价； γ 是时间折扣； H 表示预测 horizon。

8.5.3 总结

4D occupancy forecasting 重要，是因为自动驾驶最终关心的是“三维空间随时间如何被占据”，而不是物体类别或视频外观。它绕开了 detection-tracking-prediction 级联误差，统一处理异形物体和部分遮挡，又比 BEV 多保留了高度信息，是介于“未来图像”和“未来 trajectory”之间最贴近 planner 输入的中间表示。

8.6 问题 76: GAIA-2 为什么被称为 controllable multi-view generative world model?

8.6.1 通俗解释

GAIA-2 这类 controllable multi-view generative world model 的关键是多摄像头一致性和可控条件。它不仅生成前视视频，还要让左右后视等多个视角、道路布局、ego action 和天气等环境条件之间保持一致；任何一个视角漂移或与控制指令不符，都会让生成的场景失去仿真价值。这种“多视角同步 + 可控条件”的能力，正是它与单视角视频生成器拉开差距的地方。

8.6.2 数学推导

controllable multi-view generation 要同时满足控制条件和视角一致性：

$$\hat{I}_{t+1:t+H}^{1:N} = G_\theta(I_{\leq t}^{1:N}, a_{t:t+H}^{\text{ego}}, c).$$

多视角一致性可以通过共享 3D latent 表示实现：

$$z_t^{3D} = E(I_t^{1:N}), \quad \hat{I}_{t+k}^n = R_n(z_{t+k}^{3D}).$$

本题符号说明 其中, $I^{1:N}$ 表示 N 个相机视角; a^{ego} 表示自车控制; c 表示场景或文本条件; z^{3D} 表示共享三维 latent; R_n 表示第 n 个视角的渲染函数。

8.6.3 总结

GAIA-2 被称为 controllable multi-view generative world model, 是因为它不仅生成驾驶视频, 还能控制视角、场景和动作条件。多视角一致性与可控性让它更接近自动驾驶仿真, 而不是普通视频合成。

8.7 问题 77: Waymo World Model 如何用于 rare edge-case simulation?

8.7.1 通俗解释

Waymo World Model 的价值在于 rare edge-case simulation。真实路测很难遇到足够多的恶劣天气、动物横穿、突发事故或复杂施工场景, 而 world model 可以把这些场景变成可控、可重复的测试环境。它支撑的核心能力是反事实模拟——在同一场景下换一个动作或一个对象行为, 看未来是否变得更安全, 这是真实道路上无法反复尝试的。

8.7.2 数学推导

rare edge-case simulation 先定义稀有事件条件:

$$c_{edge} = \{weather, traffic, occlusion, behavior\}.$$

world model 生成对应未来轨迹:

$$\tau_{edge} \sim p_{\theta}(\tau | c_{edge}, M).$$

再用安全指标筛选:

$$risk(\tau_{edge}) > \delta.$$

这样可以系统构造真实路测中很难大量采集的危险场景。

本题符号说明 其中, c_{edge} 表示边缘场景条件; M 表示地图或道路结构; τ_{edge} 表示生成的边缘场景轨迹; $risk(\cdot)$ 表示风险评分; δ 表示风险阈值。

8.7.3 总结

Waymo World Model 的价值在于用学习式仿真扩展 rare edge cases。现实道路中极端场景少、采集贵、风险高, 世界模型可以在可控条件下反复生成和测试这些低频高风险事件。

8.8 问题 78: driving action control、scene layout control、language control 分别是什么?

8.8.1 通俗解释

这三种控制分别从行为、空间结构和语义三个层面引导世界模型的生成。driving action control 用自车控制量 (油门、刹车、方向) 改变未来轨迹; scene layout control 用车道、车辆、障碍物布局调整场景结构; language control 用自然语言指定天气、意图或风险事件。地图和交通规则不是装饰性的输入, 而是约束未来预测的重要先验, 三者结合才能让生成场景真正满足驾驶约束。

8.8.2 数学推导

driving action control 改变自车动作：

$$\tau \sim p_{\theta}(\tau \mid a^{ego}).$$

scene layout control 改变道路、车辆或障碍布局：

$$\tau \sim p_{\theta}(\tau \mid L).$$

language control 用文本指定场景和意图：

$$\tau \sim p_{\theta}(\tau \mid l).$$

三者分别控制行为、结构和语义。

本题符号说明 其中， a^{ego} 表示自车控制； L 表示场景布局； l 表示语言条件； τ 表示生成的驾驶世界轨迹； p_{θ} 表示条件世界模型。

8.8.3 总结

driving action control 控制自车动作, scene layout control 控制道路和交通结构, language control 用自然语言指定场景语义。三者分别对应行为、空间和语义层面的可控生成。

8.9 问题 79：为什么自动驾驶 world model 必须支持 counterfactual simulation?

8.9.1 通俗解释

自动驾驶最需要反事实能力：如果刚才刹车会怎样，如果换道会怎样，如果前方行人突然出现会怎样。world model 让系统能在不真实冒险的情况下比较这些候选未来。它和感知模型是上下游互补关系：感知给出当前状态，world model 给出动作后果。反事实能力的核心不是换一个 prompt 生成新视频，而是在同一初始世界状态下只改变动作或对象变量，并观察未来是否产生符合因果逻辑的变化。

8.9.2 数学推导

counterfactual simulation 比较同一初始状态下不同动作的未来：

$$\tau^a \sim p_{\theta}(\tau \mid z_t, do(a)), \quad \tau^{a'} \sim p_{\theta}(\tau \mid z_t, do(a')).$$

反事实风险差异为：

$$\Delta R = R(\tau^a) - R(\tau^{a'}).$$

自动驾驶需要它来回答“如果刚才不变道，会不会更安全”。

本题符号说明 其中， z_t 表示初始驾驶状态； $do(a)$ 表示对动作 a 的干预； τ^a 和 $\tau^{a'}$ 表示不同动作下的未来轨迹； R 表示风险或代价函数； ΔR 表示反事实风险差。

8.9.3 总结

自动驾驶必须支持 counterfactual simulation，因为安全决策依赖比较不同动作的后果。只知道真实轨迹发生了什么还不够，系统还要知道如果提前刹车、晚一点并线或选择另一条车道会怎样。

8.10 问题 80：如何评估自动驾驶 world model 的安全性和可控性？

8.10.1 通俗解释

评估自动驾驶 world model 不能只看视觉相似度，要同时回答两个问题：未来是否安全，未来是否真的受控。安全性看模型生成的轨迹中碰撞率、越界率、急刹率和规则违反率；可控性看不同的 ego 动作是否真的导致显著不同的未来。二者必须同时满足——只安全不可控，模型对动作不敏感；只可控不安全，模型可以乱走但生成结果不可信。工程上还要把 guardrail（限制生成边界）和 evaluator（判断未来质量）分开使用，组合起来才构成可部署的闭环评估。

8.10.2 数学推导

安全性可用碰撞、越界、规则违背等代价评估：

$$S_{safe} = 1 - \frac{1}{H} \sum_{k=1}^H \mathbb{I}[\text{violation}_{t+k}].$$

可控性可用动作敏感性评估：

$$D_{ctrl} = d(p_{\theta}(\tau | a), p_{\theta}(\tau | a')).$$

两者必须同时满足：安全但不受动作控制，或可控但不安全，都不是好的驾驶 world model。

本题符号说明 其中， S_{safe} 表示安全得分； H 表示预测 horizon； $\mathbb{I}[\cdot]$ 表示指示函数； D_{ctrl} 表示动作可控性； d 表示分布差异； a 和 a' 表示不同动作。

8.10.3 总结

自动驾驶 world model 的安全评估应关注生成未来是否会误导规划。关键不是视频像不像，而是碰撞风险、交通规则、物理可行性、动作响应和长尾场景覆盖是否可靠。

9 VLA Driving World Model

本章主线 本章答题主线：VLA driving 的关键是让视觉、语言、动作和未来想象进入同一个规划闭环。

9.1 问题 81：VLA 为什么需要 world model？

9.1.1 通俗解释

VLA 模型把 vision、language 和 action 连起来，但如果缺少 world model，它仍然可能只是在“看懂并执行”，而不是“预测执行后的后果”。驾驶或机器人场景中，语言给出目标，视觉给出当前状态，world model 负责预测候选动作会把世界带向哪里。没有这一步，VLA 容易出现看懂指令但动作不安全、不可执行或短视的问题。

9.1.2 数学推导

VLA 可先把观测和语言编码为状态：

$$z_t = f_{\phi}(o_t, l).$$

world model 预测动作后的未来:

$$\hat{z}_{t+1:t+H} = g_{\theta}(z_t, a_{t:t+H}, l).$$

策略选择满足语言目标且风险最小的动作:

$$a^* = \arg \min_a C(\hat{z}_{t+1:t+H}, a, l).$$

本题符号说明 其中, o_t 表示视觉观测; l 表示语言指令; z_t 表示融合后的 latent state; $a_{t:t+H}$ 表示候选动作序列; $\hat{z}_{t+1:t+H}$ 表示预测未来状态; C 表示同时考虑任务目标和安全风险的代价函数。

9.1.3 总结

VLA 需要 world model, 是因为从“理解任务”到“安全行动”之间需要后果预测。world model 给 VLA 增加了内部试错能力, 让模型能先想象再执行。

9.2 问题 82: DriveWorld-VLA 为什么要统一 world modeling 和 planning?

9.2.1 通俗解释

DriveWorld-VLA 要统一 world modeling 和 planning, 是因为未来预测如果和动作规划分开训练, planner 很难真正利用 world model 的想象结果。统一之后, 模型在同一个 latent space 中表示当前场景、语言目标、未来状态和候选轨迹, planner 可以直接读取 predicted latent 来比较不同动作的风险和收益。

9.2.2 数学推导

世界建模阶段预测未来 latent:

$$\hat{z}_{t+1:t+H} = g_{\theta}(z_t, a_{t:t+H}, l).$$

规划阶段不是另起炉灶, 而是在同一 latent 空间中优化:

$$a_{t:t+H}^* = \arg \min_{a_{t:t+H}} [C_{task}(\hat{z}, l) + C_{safe}(\hat{z}) + C_{smooth}(a)].$$

这使 world model 的预测结果直接进入 planning objective。

本题符号说明 其中, z_t 表示当前驾驶 latent; l 表示语言或导航目标; $a_{t:t+H}$ 表示候选轨迹; $\hat{z}$ 表示预测未来 latent; C_{task} 、 C_{safe} 、 C_{smooth} 分别表示任务、安全和舒适性代价。

9.2.3 总结

统一 world modeling 和 planning 的目的, 是避免“预测归预测、规划归规划”的脱节。DriveWorld-VLA 的核心价值是让预测出来的 latent future 直接服务轨迹选择。

9.3 问题 83: latent-space imagination 为什么比 pixel-space rollout 更适合驾驶规划?

9.3.1 通俗解释

latent-space imagination 比 pixel rollout 更适合驾驶规划, 因为规划不一定需要每个像素都真实, 只需要保留道路、车辆、行人、风险和可行空间等决策相关信息。这样不仅计算更轻, 也更容易和 planner 共享表示。一个工程上常被忽视的细节是: 作为接口的 action token 必须和真实控制量或轨迹目标对齐, 否则从语言到动作的 grounding 会在 latent 层断开。

9.3.2 数学推导

pixel-space rollout 直接预测图像:

$$\hat{I}_{t+1:t+H} = G_\theta(I_{\leq t}, a_{t:t+H}).$$

latent-space imagination 预测紧凑状态:

$$\hat{z}_{t+1:t+H} = F_\theta(z_t, a_{t:t+H}).$$

规划只需要和风险、轨迹、占用相关的状态, 因此 latent rollout 通常更高效、更稳定。

本题符号说明 其中, I_t 表示图像帧; z_t 表示驾驶 latent state; $a_{t:t+H}$ 表示候选动作; G_θ 表示像素生成器; F_θ 表示 latent dynamics; H 表示规划 horizon。

9.3.3 总结

latent-space imagination 比 pixel-space rollout 更适合驾驶规划, 因为规划关心的是车道、障碍物、速度、风险等结构变量。像素未来很难长期稳定, 也不一定包含最适合控制的抽象信息。

9.4 问题 84: DriveWorld-VLA 如何让 planner 使用 world model latent state?

9.4.1 通俗解释

DriveWorld-VLA 让 planner 使用 latent state 的方式, 是把未来场景压缩成对决策有用的内部表示, 而不是让 planner 直接读完整视频。latent state 中应包含道路结构、车辆运动、潜在风险、语言目标和候选动作后果。planner 在这个 latent 空间里比较轨迹, 因此可以更快、更稳定地做决策。

9.4.2 数学推导

先用 world model 递推候选动作下的 latent future:

$$\hat{z}_{t+k+1} = g_\theta(\hat{z}_{t+k}, a_{t+k}, l), \quad k = 0, \dots, H-1.$$

planner 读取这些 latent states 并计算代价:

$$J(a_{t:t+H}) = \sum_{k=1}^H C_\psi(\hat{z}_{t+k}, a_{t+k}, l).$$

最终选择:

$$a_{t:t+H}^* = \arg \min_{a_{t:t+H}} J(a_{t:t+H}).$$

本题符号说明 其中, $\hat{z}_{t+k}$ 表示预测的第 $t+k$ 步 latent state; a_{t+k} 表示候选动作; l 表示语言目标; C_ψ 表示由 planner 或 evaluator 学到的代价函数; J 表示轨迹总代价。

9.4.3 总结

planner 使用 world model latent state, 本质是用紧凑、可预测、任务相关的内部状态替代完整未来视频。这样 planning 可以直接基于风险、目标和动态信息选择动作。

9.5 问题 85: DriveVLA-W0 如何用 future image prediction 缓解 supervision deficit?

9.5.1 通俗解释

Driving VLA 只用动作标签训练时, 监督信号非常稀疏: 一个复杂视觉场景最后只对应几个低维控制量。DriveVLA-W0 加入 future image prediction, 让模型学习环境动态, 从而获得 dense self-supervised signals。

9.5.2 数学推导

动作监督为:

$$\mathcal{L}_{act} = \|a_t - \hat{a}_t\|.$$

未来图像监督为:

$$\mathcal{L}_{future} = \sum_{k=1}^H \|\hat{I}_{t+k} - I_{t+k}\|_1.$$

总目标:

$$\mathcal{L} = \mathcal{L}_{act} + \lambda \mathcal{L}_{future}.$$

本题符号说明 其中, a_t 表示真实动作; $\hat{a}_t$ 表示预测动作; I_{t+k} 表示真实未来图像; $\hat{I}_{t+k}$ 表示预测未来图像; $\mathcal{L}_{act}$ 表示动作监督损失; $\mathcal{L}_{future}$ 表示未来帧重建损失; λ 表示权衡系数; H 表示预测 horizon。

9.5.3 总结

DriveVLA-W0 用 future image prediction 缓解 supervision deficit, 是因为真实驾驶数据中直接的高质量动作监督有限。预测未来画面可以提供额外时序学习信号, 让模型从未标注或弱标注视频中学习驾驶动态。

9.6 问题 86: VLA-World 为什么结合 predictive imagination 和 reflective reasoning?

9.6.1 通俗解释

VLA-World 的思路是先根据候选轨迹想象未来, 再根据生成的未来帧反思轨迹是否合理。它把 predictive imagination 和 reflective reasoning 连接起来, 使驾驶决策更可解释。这一框架真正难的地方, 是把高层语言命令一直 grounding 到低层安全可执行轨迹——任何一环脱节, 反思阶段都会失去依据。

9.6.2 数学推导

预测想象阶段：

$$\hat{x}_{t+1:t+H} = G_{\theta}(x_{\leq t}, a_{t:t+H}, l).$$

反思评分阶段：

$$s = R_{\phi}(\hat{x}_{t+1:t+H}, a_{t:t+H}, l), \quad a^* = \arg \max_a s.$$

两阶段共享同一组候选轨迹，但角色不同：前者负责想象，后者负责评判。

本题符号说明 其中， $x_{\leq t}$ 表示历史观测； $\hat{x}_{t+1:t+H}$ 表示生成的未来帧； $a_{t:t+H}$ 表示候选动作序列； l 表示语言指令； G_{θ} 表示生成模型； R_{ϕ} 表示反思评分模型； s 表示候选轨迹的合理性评分； H 表示 rollout horizon。

9.6.3 总结

VLA-World 结合 predictive imagination 和 reflective reasoning，是因为驾驶既要想象未来，也要解释和修正自己的计划。前者提供候选未来，后者检查这些未来是否合理、安全并符合任务目标。

9.7 问题 87：为什么“生成未来帧后再反思轨迹”有助于自动驾驶解释性？

9.7.1 通俗解释

”先生成未来帧，再反思轨迹”的好处，是把驾驶决策从黑盒动作输出变成一条可被检查的因果链。系统不仅要给出”为什么选择这条轨迹”，还能用想象到的未来作为佐证：例如即将并线的相邻车辆、被遮挡的行人、与导航路线的冲突等。要真正实现这种解释性，多模态监督必须同时约束语义理解、空间定位、动作后果和安全边界，否则解释只是事后合理化。

9.7.2 数学推导

先生成未来帧：

$$\hat{x}_{t+1:t+H} = G_{\theta}(x_{\leq t}, a_{t:t+H}, l).$$

再由 reasoning/evaluator 解释轨迹风险：

$$e = R_{\phi}(\hat{x}_{t+1:t+H}, a_{t:t+H}, l).$$

解释性来自把“为什么选择这条轨迹”绑定到可观察未来，例如即将并线车辆、遮挡行人或路线冲突。

本题符号说明 其中， $\hat{x}_{t+1:t+H}$ 表示预测未来帧； $a_{t:t+H}$ 表示候选轨迹； l 表示语言目标； R_{ϕ} 表示反思或评价模型； e 表示解释或评分。

9.7.3 总结

生成未来帧后再反思轨迹，可以把自动驾驶决策从黑盒动作输出变成可检查的因果链。系统不仅给出轨迹，还能用想象到的未来说明为什么该减速、变道或保持车道。

9.8 问题 88: Vega 这类 Vision-Language-World-Action 模型为什么结合 AR 和 diffusion 范式?

9.8.1 通俗解释

Vega 这类 Vision-Language-World-Action 模型尝试把世界预测和动作生成纳入统一架构。AR 适合离散序列和长程推理,能保持任务级一致性;diffusion 适合连续轨迹和多模态动作分布,能生成高质量视觉与控制信号。混合范式让模型同时具备结构化推理与连续控制能力,因此评估时也要覆盖语言遵循度、轨迹质量、风险控制和闭环稳定性等多个维度。

9.8.2 数学推导

AR 范式适合顺序决策和离散 token 生成:

$$p(z_{1:T}) = \prod_t p(z_t | z_{<t}).$$

diffusion 范式适合高保真连续视觉细节:

$$z^{(n-1)} = D_\theta(z^{(n)}, c).$$

Vision-Language-World-Action 模型结合二者,是为了同时获得动作序列建模能力和视觉生成质量。

本题符号说明 其中, $z_{1:T}$ 表示长度为 T 的序列 token; $z_{<t}$ 表示第 t 步之前的历史 token; T 表示序列长度; D_θ 表示 diffusion denoiser; $z^{(n)}$ 表示扩散过程第 n 步的变量; c 表示条件信息。

9.8.3 总结

Vega 这类模型结合 AR 和 diffusion,是因为两种范式优势不同。AR 适合序列化决策和长程依赖, diffusion 适合高质量连续生成;结合后可同时兼顾动作序列建模和视觉未来生成。

9.9 问题 89: VLA driving world model 如何处理语言指令、世界状态和轨迹生成?

9.9.1 通俗解释

VLA driving world model 需要把三类信息放到同一个闭环里:语言指令定义目标和偏好,世界状态描述当前交通环境,轨迹生成给出可执行行为。难点是三者必须一致:语言说“安全靠右停车”,视觉状态必须识别可停区域,轨迹生成必须满足道路和安全约束。如果任何一环脱节,就会出现看懂了指令但开不对的情况。

9.9.2 数学推导

语言和观测先融合成驾驶状态:

$$z_t = f_\phi(o_t, l).$$

轨迹 generator 产生候选动作序列:

$$a_{t:t+H} = \pi_\psi(z_t, l).$$

world model 预测该轨迹带来的未来:

$$\hat{z}_{t+1:t+H} = g_\theta(z_t, a_{t:t+H}, l).$$

最终用任务和安全代价筛选:

$$a^* = \arg \min_a [C_{lang}(\hat{z}, l) + C_{safe}(\hat{z}) + C_{route}(\hat{z})].$$

本题符号说明 其中, o_t 表示当前驾驶观测; l 表示语言指令; z_t 表示融合后的世界状态; π_ψ 表示轨迹生成策略; g_θ 表示 world model; C_{lang} 、 C_{safe} 、 C_{route} 分别表示语言符合度、安全和路线代价。

9.9.3 总结

VLA driving world model 的核心是把语言目标、当前世界状态和未来轨迹统一起来。语言决定想做什么, 世界状态决定能不能做, 轨迹生成决定怎么做。

9.10 问题 90: 自动驾驶 VLA world model 的最大风险是什么?

9.10.1 通俗解释

自动驾驶 VLA world model 最大的风险, 是模型生成一个看似合理却在安全上完全错误的未来。错误来源主要有三类: 未来帧预测在关键物体、遮挡或速度上出错; 反思理由看似自洽但与真实物理冲突; 分布外场景下模型仍然过度自信。任何一类都可能把规划推向危险动作, 而且因为未来“长得像”, 错误反而更难被发现。

9.10.2 数学推导

VLA world model 的统一形式可写为:

$$z_t = f_\phi(o_t, l), \quad \hat{z}_{t+1:t+H} = g_\theta(z_t, a_{t:t+H}, l), \\ a^* = \arg \min_a C(\hat{z}_{t+1:t+H}, a, l).$$

当 g_θ 在分布外场景失真时, $\hat{z}$ 与真实 z 的偏差为 $\delta = \hat{z} - z$; 规划误判风险则正比于 $\|\delta\|$ 与代价函数 C 的局部敏感度, 即 $\Delta C \approx \nabla_z C \cdot \delta$ 。这意味着在 C 对状态变化敏感的场景 (紧急刹车、并线冲突), 即使 δ 很小也可能把决策推向危险动作。

本题符号说明 其中, o_t 表示当前观测; l 表示语言指令; z_t 表示融合后的 latent state; $\hat{z}_{t+1:t+H}$ 表示模型预测的未来 latent; $a_{t:t+H}$ 表示候选动作序列; C 表示规划代价; δ 表示状态预测偏差; $\nabla_z C$ 表示代价对状态的局部梯度; H 表示规划 horizon。

9.10.3 总结

自动驾驶 VLA world model 的最大风险是生成一个看似合理但会误导规划的未来。只要未来预测在关键物体、遮挡、速度或动作响应上出错, 规划模块就可能做出系统性危险决策。

10 机器人世界模型与 VLA-JEPA

本章主线 本章答题主线: 机器人世界模型比驾驶更依赖接触和可供性, 必须预测对控制有用的未来, 而不是只预测视觉上合理的未来。

10.1 问题 91：机器人世界模型和自动驾驶世界模型有什么不同？

10.1.1 通俗解释

两者都是 world model，但关注的物理现象差异很大。**自动驾驶**主要在道路拓扑和交通参与者之间移动，关心的是大范围空间几何、相对速度、其他 agent 意图和长程规划——状态变化在秒到分钟尺度，动作和物体之间几乎没有直接接触。**机器人**（尤其是 manipulation）则要处理夹爪、摩擦、物体形变、遮挡、失败接触——状态变化在毫秒到秒尺度，且动作和环境之间几乎完全是接触主导。

四个核心维度的对比：(1) **时间尺度**——驾驶秒级、机器人毫秒级；(2) **空间尺度**——驾驶百米、机器人厘米毫米；(3) **动作维度**——驾驶低维（油门、方向、刹车），机器人高维（关节角、力矩、夹爪开合）；(4) **失败模式**——驾驶以碰撞和违规为主，机器人以滑落、夹偏、物体破损为主。

技术上最难的是接触动力学：物体被推、夹、插入瞬间，状态会因为摩擦、刚度、间隙的微小差异发生非线性突变，预测稍有偏差，整个操作就会失败。这也是为什么机器人 world model 通常需要 force/tactile 模态，而自动驾驶不需要。

10.1.2 数学推导

自动驾驶世界模型主要建模交通中的多 agent 运动：

$$p_{\theta}(z_{t+1} \mid z_t, a_t^{\text{ego}}, A_t^{\text{others}}, M_t),$$

其中 A_t^{others} 是其他交通参与者状态， M_t 是地图。机器人世界模型必须显式区分本体、接触、物体三类状态：

$$p_{\theta}(o_{t+1}, q_{t+1}, h_{t+1}, s_{t+1}^{\text{obj}} \mid o_t, q_t, h_t, s_t^{\text{obj}}, a_t),$$

并往往加入力觉/触觉模态：

$$a_t \in \mathbb{R}^{d_q + d_g}, \quad o_t = \{I_t, F_t, \tau_t\},$$

其中 I_t 是图像， F_t 是力， τ_t 是力矩。两者动作维度差异很大：驾驶 $\dim(a) \sim 2-3$ ，机器人 $\dim(a) \sim 7-30$ （视自由度而定），并且机器人有强非线性接触切换，对应的 transition 在接触瞬间可能 Lipschitz 系数突增多个数量级——这就是为什么直接搬驾驶 world model 用于 manipulation 通常会失败。

本题符号说明 其中， z_t 表示驾驶 latent； a_t^{ego} 表示自车动作； A_t^{others} 表示其他参与者； M_t 表示地图； o_t 表示观测； q_t 表示机器人本体状态； h_t 表示接触状态； s_t^{obj} 表示物体状态； a_t 表示控制动作； F_t 、 τ_t 表示力与力矩； d_q 、 d_g 表示关节与夹爪维度。

10.1.3 总结

自动驾驶世界模型更强调大范围交通动态、多 agent 意图和长程安全预测，机器人世界模型更强调近距离接触、力觉融合、物体精细状态变化。两者共享 world model 的基本范式（latent state + transition + reward），但在时间尺度、空间尺度、动作维度、模态需求和失败模式上差异巨大，因此架构和训练数据通常无法直接复用。

10.2 问题 92：为什么机器人任务更依赖 contact dynamics？

10.2.1 通俗解释

机器人操作的难点在接触动态：轻微的摩擦、抓取点位置或物体姿态变化都可能导致完全不同的结果。navigation 主要预测空间可达性，manipulation 则要进一步预测力、接触和物体状态

变化。这也是 VLA-JEPA 等方法的关注点——把视觉表示预测和动作条件控制联系起来，让 latent 不只是好看，还能反映接触关系的微小差异。

10.2.2 数学推导

contact dynamics 可以用接触状态 h_t 表示：

$$h_{t+1} = F_\theta(h_t, q_t, a_t, o_t).$$

若接触状态预测错误，下一步物体状态也会错：

$$p(s_{t+1}^{obj} | s_t^{obj}, h_{t+1}, a_t).$$

因此机器人世界模型必须显式或隐式建模接触变化。

本题符号说明 其中， h_t 表示接触状态； q_t 表示机器人关节或末端状态； a_t 表示控制动作； s_t^{obj} 表示物体状态； F_θ 表示接触动态模型。

10.2.3 总结

机器人任务依赖 contact dynamics，因为抓取、推动、插入、旋转都由接触关系决定。视觉上差不多的状态，接触点、摩擦和力的微小差异可能导致完全不同的结果。

10.3 问题 93：manipulation task 为什么比 navigation task 更难建模？

10.3.1 通俗解释

manipulation 比 navigation 难，根本原因是它直接改变物体状态。navigation 大多只关心自身位姿与可通行空间，manipulation 则要建模物体位姿、形变、接触点和力，并且这些变量对动作敏感、对噪声脆弱。也因此机器人世界模型常常采用 object-centric state，因为任务大多围绕物体、工具、手部和接触关系展开，而不是整个画面。

10.3.2 数学推导

navigation 多数时候关注位姿转移：

$$p(p_{t+1} | p_t, a_t, map).$$

manipulation 还要建模物体、接触和力：

$$p(s_{t+1}^{obj}, h_{t+1} | s_t^{obj}, q_t, a_t).$$

后者的状态维度更高，且小误差会放大为任务失败。

本题符号说明 其中， p_t 表示机器人位姿； map 表示环境地图； s_t^{obj} 表示被操作物体状态； q_t 表示机器人本体状态； h_t 表示接触状态。

10.3.3 总结

manipulation 比 navigation 难，是因为它需要改变物体状态并处理接触不确定性。导航主要在空间中移动自身，操作任务还要让外部物体按照预期运动、变形或组合。

10.4 问题 94: VLA-JEPA 为什么要学习 action-relevant state transition?

10.4.1 通俗解释

VLA-JEPA 关注的是 action-relevant state transition: 它不希望模型被背景变化、相机抖动或无关像素运动吸引, 而是让 latent prediction 对真正影响动作成败的状态敏感。所谓 action relevance, 就是要求表示保留那些会被动作改变的因素 (物体位姿、接触状态、可供性), 丢掉与控制无关的纹理噪声。

10.4.2 数学推导

action-relevant transition 关注的是动作导致的任务相关变化:

$$\hat{z}_{t+1} = P_{\theta}(z_t, a_t).$$

训练时应提高与任务成功相关的状态维度权重:

$$\mathcal{L} = \|W(\hat{z}_{t+1} - z_{t+1})\|_2^2,$$

其中 W 强调物体位姿、接触和 affordance 等关键因素。

本题符号说明 其中, z_t 表示机器人任务 latent; a_t 表示动作; $\hat{z}_{t+1}$ 表示预测未来 latent; W 表示任务相关权重矩阵; P_{θ} 表示转移预测器。

10.4.3 总结

VLA-JEPA 学习 action-relevant state transition, 是为了让 latent 只保留对动作后果有用的变化。机器人不需要预测所有像素扰动, 而需要预测与抓取、推动、放置成功相关的状态变化。

10.5 问题 95: VLA-JEPA 如何避免 pixel variation 和 nuisance motion?

10.5.1 通俗解释

机器人摄像头拍到的画面里, 绝大多数像素变化和动作根本无关: 光照波动、相机抖动、风吹窗帘、背景人员走动、桌面反光、镜头炫光。如果模型在像素空间训练, loss 会被这些 nuisance motion 主导——因为它们的像素变化幅度往往比手部动作或物体接触大得多——结果学到的 transition 看似”动得很真”, 对机器人控制却毫无用处。

VLA-JEPA 的解法分三层。第一层是把目标搬到 embedding 空间: 因为 target encoder 已经过滤掉部分低层纹理, loss 自然不再奖励背景像素的变化。第二层是 stop-gradient: target encoder 不接收来自 nuisance 的梯度, 避免被无关运动拉偏表示。第三层是 action-conditioned predictor: predictor 必须用动作 a_t 解释 $z_{t+1} - z_t$ 的变化, 与动作无关的变化即使发生也不会被预测到, 从而被自然压低。这三层组合起来, 等价于”只学习被动作 explain 的那部分动态”。

10.5.2 数学推导

原始像素 transition 的损失会同时惩罚动作相关和动作无关的差异:

$$\mathcal{L}_{\text{pix}} = \|G_{\theta}(o_t, a_t) - o_{t+1}\|_2^2,$$

其中 $o_{t+1} - G_{\theta}(o_t, a_t)$ 包含光照漂移、背景运动等 nuisance。VLA-JEPA 改在 embedding 空间预测, 并对 target 加 stop-gradient:

$$z_t = E_{\phi}(o_t), \quad z_{t+1} = E_{\xi}(o_{t+1}), \quad \mathcal{L}_{\text{vJ}} = \|P_{\theta}(z_t, a_t) - \text{sg}(z_{t+1})\|_2^2.$$

当 nuisance 噪声 η 满足 $E_\xi(o + \eta) \approx E_\xi(o)$ (target encoder 对 nuisance 不变) 时, loss 对 nuisance 不再敏感。可进一步把表示拆为任务相关分量与无关分量:

$$z_t = z_t^{\text{task}} \oplus z_t^{\text{nui}},$$

并通过对比正则强制 $\text{sg}(z_{t+1}^{\text{nui}})$ 不可被 a_t 预测, 从而让 predictor 只关注动作可解释的子空间。这正是 V-JEPA 系列 + action conditioning 在 robot manipulation 上比像素预测更稳的根本原因。

本题符号说明 其中, o_t, o_{t+1} 表示相邻观测; η 表示 nuisance 噪声; z_t, z_{t+1} 表示 latent; $z_t^{\text{task}}, z_t^{\text{nui}}$ 表示任务相关与无关分量; E_ϕ, E_ξ 表示 context 与 target encoder; P_θ 表示 action-conditioned predictor; G_θ 表示像素生成器; a_t 表示动作; $\text{sg}(\cdot)$ 表示 stop-gradient; $\oplus$ 表示分量分解。

10.5.3 总结

VLA-JEPA 通过三个机制系统地压低 nuisance motion: embedding 空间预测过滤低层纹理、stop-gradient 防止 target 被无关变化拖偏、action-conditioned predictor 只学习被动作解释的那部分动态。结果是 latent transition 集中在物体姿态、接触状态、可供性这些真正决定操作成败的变量上, 而不是把容量浪费在光照、背景、相机抖动上。

10.6 问题 96: learned simulator 如何用于 robot policy post-training?

10.6.1 通俗解释

learned simulator 可以在机器人策略后训练中提供可控的想象环境: 策略可以在模型里尝试更多动作、发现失败模式, 再把经验用于 RL、评估或数据生成。其中真正决定成败的是 contact-aware prediction——抓取、推、拉、放置这类任务都依赖于接触瞬间的细节, learned simulator 如果只能保证视觉合理, 却预测不准接触发生的时机和位置, 就无法支撑严肃的 post-training。

10.6.2 数学推导

learned simulator 用于 post-training 时, 策略在模型内产生轨迹:

$$z_{t+1} \sim p_\theta(z_{t+1} | z_t, a_t), \quad a_t = \pi_\psi(z_t).$$

策略更新为:

$$\psi \leftarrow \psi + \eta \nabla_\psi \mathbb{E} \left[\sum_t r(z_t, a_t) \right].$$

风险是 simulator bias 会被 policy 利用。

本题符号说明 其中, p_θ 表示 learned simulator; π_ψ 表示机器人策略; ψ 表示策略参数; r 表示任务奖励; η 表示学习率。

10.6.3 总结

learned simulator 可用于 robot policy post-training, 因为它能在真实机器人之外提供大量可交互经验。策略可以在模拟的接触、失败和扰动中继续优化, 再迁移回真实系统。

10.7 问题 97: 什么是 control-consistent future?

10.7.1 通俗解释

plausible future 看起来合理, control-consistent future 对控制有用。机器人策略真正需要的是后者: 接触点、物体姿态和可操作性必须和真实执行保持一致, 否则画面再好也会误导动作。要做到这一点, 机器人世界模型往往要同时建模视觉、本体感知、力觉和语言目标, 而不能只依赖单一模态。

10.7.2 数学推导

control-consistent future 要求预测未来和真实控制代价排序一致:

$$C(\hat{z}_{t+1}, a) \approx C(z_{t+1}, a).$$

更强的条件是最优动作一致:

$$\arg \min_a C(\hat{z}_{t+1}, a) = \arg \min_a C(z_{t+1}, a).$$

这比视觉逼真更贴近控制需求。

本题符号说明 其中, $\hat{z}_{t+1}$ 表示预测未来状态; z_{t+1} 表示真实未来状态; C 表示控制代价; a 表示候选动作。

10.7.3 总结

control-consistent future 指预测未来必须和给定控制动作一致。未来看起来合理还不够; 如果给定向左推, 模型却生成像没有推或向右推的结果, 这个未来就不能用于控制学习。

10.8 问题 98: 为什么 plausible future 不等于 useful future?

10.8.1 通俗解释

”plausible” 和 ”useful” 是两件事: 前者只要求未来看起来自然, 后者要求未来对决策有用。一个生成得很漂亮但接触点错位、物体响应不到动作的未来, 对策略学习几乎是负贡献。也因此评估机器人 world model 不能停留在视觉指标, 要看任务成功率、动作一致性、接触预测以及最终的 sim-to-real 迁移效果。

10.8.2 数学推导

plausible future 只要求未来在数据分布上合理:

$$\hat{z} \sim p_{data}(z).$$

useful future 还要求它保留决策所需信息:

$$\arg \min_a C(\hat{z}, a) \approx \arg \min_a C(z, a).$$

因此生成得像, 不代表对控制有用。

本题符号说明 其中, p_{data} 表示真实数据分布; $\hat{z}$ 表示生成未来; z 表示真实未来; C 表示任务或控制代价; a 表示动作。

10.8.3 总结

plausible future 不等于 useful future。对机器人来说，有用的未来必须区分动作差异、保留任务变量并支持成功判断；仅仅视觉自然的未来可能对策略学习没有帮助。

10.9 问题 99: world model 和 robot policy 应该分开训练还是联合训练?

10.9.1 通俗解释

world model 和 robot policy 可以分开训练，也可以联合训练。分开训练更稳定、可复用，模型可以服务多个策略；联合训练能让表示更贴近当前控制目标，但代价是模型容易学偏到当前策略覆盖的分布上，对其他任务反而不友好。和驾驶相比，机器人场景更强调近距离交互、接触和物体可操作性，所以通常会先用大规模无动作视频学通用表示，再针对具体任务做有限的联合微调。

10.9.2 数学推导

分开训练可写成先学习模型再学习策略：

$$\theta^* = \arg \min_{\theta} \mathcal{L}_{model}, \quad \psi^* = \arg \max_{\psi} J(\pi_{\psi}; p_{\theta^*}).$$

联合训练则优化：

$$\min_{\theta} \mathcal{L}_{model} - \lambda J(\pi_{\psi}; p_{\theta}).$$

前者稳定，后者更任务相关但更容易互相污染。

本题符号说明 其中， θ 表示世界模型参数； ψ 表示策略参数； $\mathcal{L}_{model}$ 表示模型损失； J 表示策略回报； λ 表示任务回报权重。

10.9.3 总结

world model 和 robot policy 可以分开训练，也可以联合训练。分开训练更稳定、可复用，联合训练更贴近任务目标；关键取舍在于泛化能力、训练稳定性和是否会让模型只服务当前策略。

10.10 问题 100: 世界模型如何支持 sim-to-real transfer?

10.10.1 通俗解释

sim-to-real 的核心难点是 reality gap：工程模拟器算得快、可控好，但摩擦、形变、传感噪声、行人意图、光照分布往往与真实世界存在系统性差异，仅在模拟器训练的策略一上真机就退化。世界模型的价值在于它从真实数据中学到了模拟器学不到的那部分动态，从而可以用四种互补方式弥合 gap：

(1) **学习残差**：保留模拟器的物理骨架，让神经网络学习真实-模拟之间的动态差，把 sim trajectory 修正到 real trajectory；(2) **域随机化扩展**：用世界模型生成大量带随机化参数的轨迹（光照、纹理、摩擦），策略训练时见过更广分布；(3) **长尾合成**：用真实数据驱动世界模型生成现实中罕见但关键的失败场景（碰撞、滑落、物体破损），让策略提前演练；(4) **不确定性指导**：策略在 epistemic 高的状态自动保守、降级或请求人类接管，避免在模型未覆盖区域硬冲。

10.10.2 数学推导

工程模拟器是带物理参数的确定动态：

$$z_{t+1}^{\text{sim}} = F_{\text{sim}}(z_t, a_t; \xi),$$

而真实环境则需要额外的修正：

$$z_{t+1}^{\text{real}} = z_{t+1}^{\text{sim}} + \Delta_{\theta}(z_t, a_t, o_t) + \epsilon_t.$$

残差网络 Δ_{θ} 在真实数据上训练以最小化：

$$\mathcal{L}_{\text{res}} = \mathbb{E}_{(z_t, a_t, z_{t+1}) \sim \mathcal{D}_{\text{real}}} \|z_{t+1} - F_{\text{sim}}(z_t, a_t; \xi) - \Delta_{\theta}(z_t, a_t, o_t)\|_2^2.$$

在策略侧，rollout 在混合动态上进行：

$$\hat{z}_{t+1} \sim p_{\theta}^{\text{hybrid}}(\hat{z}_{t+1} | \hat{z}_t, a_t) = F_{\text{sim}}(\hat{z}_t, a_t; \xi) + \Delta_{\theta} + \sigma_{\text{epi}} \cdot \mathcal{N}(0, I).$$

进一步可加入 uncertainty penalty 防止策略利用模型漏洞：

$$\tilde{r}(z, a) = \hat{r}(z, a) - \beta \sigma_{\text{epi}}(z, a), \quad J = \mathbb{E} \left[\sum_t \gamma^t \tilde{r}_t \right].$$

如果 Δ_{θ} 与 σ_{epi} 估计准确，sim-to-real gap 可以从单纯依赖 domain randomization 的”广撒网”提升为针对性补全。

本题符号说明 其中， z_t 表示状态； a_t 表示动作； F_{sim} 表示工程模拟器； ξ 表示物理参数； Δ_{θ} 表示学习残差； ϵ_t 表示真实世界中无法建模的随机性； σ_{epi} 表示 epistemic 不确定性； $\mathcal{D}_{\text{real}}$ 表示真实数据集； $\hat{r}$ 表示预测奖励； $\tilde{r}$ 表示风险调整奖励； β 表示惩罚权重。

10.10.3 总结

世界模型支持 sim-to-real transfer 的关键，是用真实数据补足模拟器学不到的那部分动态，并在策略训练中用残差校正、域随机化扩展、长尾合成、不确定性惩罚四种机制系统地缩小 reality gap。它的前提是残差与 epistemic 估计本身不能系统性误导策略——否则 gap 不仅没缩小，反而被神经网络放大。

11 空间智能、3D 世界与可交互环境

本章主线 本章答题主线：空间智能要求模型从二维识别升级到三维记忆、几何一致性、视角变换和可交互环境。

11.1 问题 101：什么是 Spatial Intelligence？它和 world model 有什么关系？

11.1.1 通俗解释

Spatial Intelligence 指模型理解三维空间、物体关系、可达性和交互可能性的能力。它和 world model 紧密相关——世界模型不仅要预测画面，还要维护一个可行动的空间结构。具体来说，空间智能要求模型知道物体在哪里、如何遮挡、是否持续存在，以及 agent 如何到达或操作它，这些信息正是世界模型用于规划和决策的基础。

11.1.2 数学推导

Spatial Intelligence 可以表示为从观测构建可行动空间状态：

$$S_t = f_\theta(o_{\leq t}), \quad p(S_{t+1} | S_t, a_t).$$

它和 world model 的关系在于：空间智能提供世界状态，world model 预测这个状态如何随动作变化。

本题符号说明 其中， S_t 表示空间世界状态； $o_{\leq t}$ 表示历史观测； a_t 表示动作； $p(S_{t+1} | S_t, a_t)$ 表示空间状态转移。

11.1.3 总结

Spatial Intelligence 是理解空间结构、几何关系和可行动性的能力。world model 需要它来判断物体在哪里、空间如何连通、动作会穿过哪里以及哪些区域可达。

11.2 问题 102：为什么 2D image understanding 不足以支持世界模型？

11.2.1 通俗解释

二维图像理解只能看到投影，无法稳定表示深度、遮挡、物体背面和可达路径。世界模型要支持导航和操作，就必须把 2D 观测提升为 3D 或对象级空间状态。换个角度说：2D 图像识别给的是语义（这是什么），3D 表示给的是结构（在哪里、能不能到达），世界模型需要把两者合并成可交互状态，缺一不可。

11.2.2 数学推导

2D 图像理解通常输出：

$$y = f_\theta(I).$$

但世界模型需要三维状态和动作转移：

$$p_\theta(S_{t+1} | S_t, a_t).$$

二维标签无法完整表达深度、遮挡、可达性和多视角一致性。

本题符号说明 其中， I 表示二维图像； y 表示分类、检测或分割输出； S_t 表示三维空间状态； a_t 表示智能体动作。

11.2.3 总结

2D image understanding 不足以支持世界模型，因为二维图像丢失了深度、尺度、遮挡后结构和可通行空间。世界模型要服务行动，就必须超越图像分类式理解，恢复空间和动态关系。

11.3 问题 103：3D geometry 在 world model 中起什么作用？

11.3.1 通俗解释

3D geometry 为世界模型提供空间骨架，告诉模型哪些位置是可达的、哪些会发生碰撞、视角变化时画面应该如何投影。它让未来预测不只是图像续写，而是建立在物理结构之上的空间推

演；这也是 object permanence 等能力的基础——没有几何骨架，模型很容易把暂时看不见的物体当成不存在。

11.3.2 数学推导

3D geometry 提供可渲染、可规划的空间约束：

$$x = \Pi(K, T, X_{3D}),$$

其中三维点 X_{3D} 经过相机参数投影到图像 x 。world model 中的动态必须同时满足几何投影和状态转移：

$$S_{t+1} \sim p_\theta(S_{t+1} | S_t, a_t).$$

本题符号说明 其中， X_{3D} 表示三维点； K 表示相机内参； T 表示相机位姿； Π 表示投影函数； S_t 表示三维状态。

11.3.3 总结

3D geometry 在 world model 中提供物理世界的骨架。没有几何结构，模型很难判断碰撞、遮挡、距离、可达性和视角变化；有了几何，预测才更容易满足物理约束。

11.4 问题 104：NeRF、3D Gaussian Splatting、occupancy、mesh 在世界模型中各有什么优缺点？

11.4.1 通俗解释

四种 3D 表达适合不同任务：NeRF 学连续辐射场，适合自由视角合成，但实时渲染开销大；3D Gaussian Splatting 用显式高斯集合替代神经辐射场，渲染高效但动态建模仍较复杂；occupancy 把空间离散为占用网格，最便于安全规划与碰撞检测；mesh 显式表示表面拓扑，适合物理交互。它们的几何表达只是空间骨架，要走到 agent 行动，还需要 affordance——表示物体能被怎样使用，把几何与语义连到具体动作。

11.4.2 数学推导

不同 3D 表达对应不同函数形式。NeRF 学连续辐射场：

$$F_\theta(x, d) = (\sigma, c).$$

3D Gaussian Splatting 学显式高斯集合：

$$G = \{(\mu_i, \Sigma_i, c_i, \alpha_i)\}.$$

occupancy 学空间是否被占据：

$$O(x) \in \{0, 1\}.$$

mesh 则显式表示表面拓扑：

$$M = (V, E, F).$$

本题符号说明 其中， x 表示空间位置； d 表示视角方向； σ 表示密度； c 表示颜色； μ_i, Σ_i 表示高斯中心和协方差； $O(x)$ 表示占用； $M = (V, E, F)$ 表示 mesh 的顶点、边和面。

11.4.3 总结

NeRF 表达连续视图但实时性和编辑较难, 3D Gaussian Splatting 渲染高效但动态建模仍复杂, occupancy 适合安全和规划, mesh 结构清晰但拓扑受限。不同表示对应不同的世界建模取舍。

11.5 问题 105: 可交互 3D 环境和普通 3D asset generation 有什么区别?

11.5.1 通俗解释

普通 3D asset generation 生成的是静态资产, 例如一把椅子、一栋建筑或一个房间布置; 可交互 3D world 必须进一步响应动作、保持状态、支持碰撞和任务目标, 更像一个模拟环境而非视觉内容。前者关心“长得像不像”, 后者关心“能不能在里面行动”。

11.5.2 数学推导

普通 3D asset generation 主要生成静态资产:

$$A \sim p_\theta(A | c).$$

可交互 3D 环境还需要状态更新:

$$S_{t+1} = F_\theta(S_t, a_t), \quad o_t = R(S_t, view_t).$$

也就是说, 它不仅要“长出来”, 还要“能被行动改变”。

本题符号说明 其中, A 表示静态 3D 资产; c 表示生成条件; S_t 表示环境状态; a_t 表示动作; R 表示从三维状态渲染观测的函数; $view_t$ 表示视角。

11.5.3 总结

可交互 3D 环境强调状态会随动作改变, 普通 3D asset generation 主要生成静态或可观看资产。前者服务 agent 训练和模拟, 后者服务内容生产; 差异在于是否有动态、物理和交互闭环。

11.6 问题 106: object permanence 如何在 3D world model 中体现?

11.6.1 通俗解释

3D world model 中的 object permanence 指的是: 物体即使被遮挡、离开视野或暂时不可见, 模型仍然保留它的三维位置、身份和状态。二维视频连贯性只要求画面看起来顺; 三维 object permanence 要求模型维护一个跨视角、跨时间的空间记忆。比如杯子被机器人手臂挡住后, 模型仍应知道杯子还在原来的桌面位置, 而不是重新生成一个新杯子。

11.6.2 数学推导

三维对象状态可写成:

$$S_t = \{(id_i, g_i, p_i, v_i, c_i, \eta_i)\}_{i=1}^N.$$

其中 $\eta_i \in \{0, 1\}$ 表示第 i 个物体当前是否可见。object permanence 要求即使 $\eta_i = 0$, 状态仍被递推:

$$p_\theta(s_{t+1}^i | s_t^i, a_t), \quad s_t^i = (id_i, g_i, p_i, v_i, c_i).$$

重新可见时, 模型应保持身份一致:

$$id_i^{reappear} = id_i^{before}.$$

本题符号说明 其中, S_t 表示三维对象集合; id_i 表示对象身份; g_i 表示几何; p_i 表示位姿; v_i 表示速度; c_i 表示语义类别; η_i 表示可见性; a_t 表示智能体动作。

11.6.3 总结

object permanence 在 3D world model 中体现为遮挡后的身份、位置和状态保持。它要求模型维护真实的空间记忆, 而不是只根据当前画面重新生成看起来合理的物体。

11.7 问题 107: affordance understanding 为什么是空间智能的核心?

11.7.1 通俗解释

affordance understanding 是知道物体能被怎样使用: 椅子可以坐, 杯子可以拿, 门把手可以拉。这种知识直接影响 agent 的行动选择, 是空间智能从识别走向行动的桥梁。和它配套的另一个能力是 spatial memory——agent 在移动视角、绕行或时间流逝之后仍要保持对场景的连续理解, 否则 affordance 知识也无法落地到具体决策。

11.7.2 数学推导

affordance 可以表示为状态-动作可行性函数:

$$A(s, a) = P(\text{success} \mid s, a).$$

世界模型如果只知道物体类别, 不知道哪些动作可执行, 就无法支持 embodied agent。空间智能需要预测“这个物体能被怎样操作”。

本题符号说明 其中, $A(s, a)$ 表示 affordance score; s 表示场景或物体状态; a 表示候选动作; $P(\text{success} \mid s, a)$ 表示动作成功概率。

11.7.3 总结

affordance understanding 是空间智能的核心, 因为智能体关心的不只是物体是什么, 而是它能被怎样使用。椅子可坐、门可开、把手可抓, 这些可行动属性直接连接到规划。

11.8 问题 108: 生成式 3D 环境如何用于 embodied agent 训练?

11.8.1 通俗解释

真实 embodied 数据采集成本高、风险大、覆盖有限, 生成式 3D 环境的价值就是提供大量可控、可重复、可干预的训练场景。Agent 可以在其中练习导航、抓取、搜索、协作等任务, 并暴露真实环境中少见的失败模式。从 2D 到 3D 的变化不是简单的维度增加, 而是从”识别画面”转向”维护一个能被多视角重访的世界坐标系”, 因此训练目标也从图像分类提升到状态-动作-后果建模。

11.8.2 数学推导

生成式 3D 环境可以作为训练分布:

$$S_0 \sim p_\theta(S \mid c), \quad S_{t+1} = F_\theta(S_t, a_t).$$

embodied agent 在其中学习策略：

$$\max_{\pi} \mathbb{E} \left[\sum_t r(S_t, a_t) \right].$$

关键是环境必须可交互，而不是只有静态渲染。

本题符号说明 其中， S_0 表示初始三维环境； c 表示场景条件； F_{θ} 表示环境动态； π 表示策略； r 表示任务奖励。

11.8.3 总结

生成式 3D 环境可用于 embodied agent 训练，因为它提供可控、多样、低成本的交互场景。agent 可以在其中练习导航、抓取、搜索和协作，并暴露现实数据中少见的失败情况。

11.9 问题 109：spatial memory 为什么比视觉短期一致性更难？

11.9.1 通俗解释

spatial memory 要求模型记住已经离开视野的物体和已经修改过的环境，并在回到该位置时保持一致。这比短期视觉连贯难得多——它需要跨视角、跨时间的全局空间记忆。一个动态 3D 世界模型同时要处理三类变化：物体自身的运动、相机的运动以及 agent 主动产生的改变；任何一类被忽略，记忆都会失真，未来预测就会脱节。

11.9.2 数学推导

spatial memory 需要把多时刻观测融合进同一地图：

$$M_t = U(M_{t-1}, o_t, T_t).$$

短期一致性只要求相邻帧平滑：

$$d(\phi(o_t), \phi(o_{t+1})) < \epsilon.$$

前者要跨视角维护坐标和身份，因此更难。

本题符号说明 其中， M_t 表示空间记忆或地图； o_t 表示当前观测； T_t 表示相机或智能体位姿； U 表示地图更新函数； ϕ 表示视觉特征； d 表示差异度量。

11.9.3 总结

spatial memory 比视觉短期一致性更难，因为它要求模型长期维护不可见空间的结构。短期一致只需相邻帧平滑，空间记忆则要在绕行、遮挡和视角变化后仍保持全局地图一致。

11.10 问题 110：spatial intelligence 和 language intelligence 的根本区别是什么？

11.10.1 通俗解释

language intelligence 处理的是符号序列，受语义、语法和上下文约束；spatial intelligence 处理的是可行动的三维状态，受几何、物理、可达性和动作后果约束。前者把世界压缩成 token 流，

后者必须维护一个能被反复进入、操作和重访的世界坐标系。也正因为如此，单靠语言模型很难真正理解”穿过门、绕过桌子、再把杯子放回原位”这种空间任务；它需要的是世界模型，而不是更长的 prompt。

11.10.2 数学推导

language intelligence 主要处理符号序列：

$$p(w_{t+1} \mid w_{\leq t}).$$

spatial intelligence 处理可行动三维状态：

$$p(S_{t+1} \mid S_t, a_t).$$

前者重在语义和逻辑连贯，后者还必须满足几何、物理和可达性约束。

本题符号说明 其中， w_t 表示语言 token； S_t 表示空间状态； a_t 表示动作； $p(w_{t+1} \mid w_{\leq t})$ 表示语言预测； $p(S_{t+1} \mid S_t, a_t)$ 表示空间动态预测。

11.10.3 总结

spatial intelligence 和 language intelligence 的根本区别在于约束来源不同。语言智能主要受符号、语义和上下文约束，空间智能受几何、物理、可达性和动作后果约束。

12 Agentic World Modeling、评估与未来趋势

本章主线 本章答题主线：Agentic World Modeling 的终点是让 agent 在行动前进行内部试错，并用一致性指标判断这些想象是否可信。

12.1 问题 111：什么是 Agentic World Modeling?

12.1.1 通俗解释

Agentic World Modeling 把世界模型从一个被动预测器升级为 agent 的内部模拟与决策机制：它要能预测动作后果、比较反事实方案、在新证据出现时修正模型，并在不同规律体系下保持一致。它的核心是闭环使用——模型预测未来，agent 比较候选动作，再根据真实结果更新模型和策略，从而形成”想象-行动-反馈-修正”的循环。

12.1.2 数学推导

动作搜索为：

$$a^* = \arg \max_a \mathbb{E}_{p_\theta}[R(\tau, a)].$$

失败后可更新：

$$\theta \leftarrow \theta - \eta \nabla_\theta \mathcal{L}_{failure}.$$

本题符号说明 其中， p_θ 表示参数为 θ 的学习式概率模型； $\mathbb{E}$ 表示期望； τ 表示一整段轨迹； $\mathcal{L}$ 表示训练损失； η 表示学习率； ∇ 表示梯度。

12.1.3 总结

Agentic World Modeling 指世界模型不只是被动预测未来，而是主动服务 agent 的目标、计划和反思。它要能根据任务选择预测什么、模拟哪些动作、评估哪些风险。

12.2 问题 112: L1 Predictor、L2 Simulator、L3 Evolver 分别代表什么能力层级?

12.2.1 通俗解释

这是对 world model 能力的一个层级化划分: L1 Predictor 只做一步局部预测, 回答”下一个状态是什么”; L2 Simulator 把一步预测组合成长时、多步、动作条件下的 rollout, 可以支撑规划; L3 Evolver 则更进一步——在预测失败后根据新证据修正自身。这一层级强调: 高级 world model 的关键不只是预测得准, 还要拥有持续记忆与自我演化能力, 能利用过去经验维护任务状态。

12.2.2 数学推导

L1:

$$z_{t+1} = f_{\theta}(z_t, a_t).$$

L2:

$$z_{t:t+H} = F_{\theta}(z_t, a_{t:t+H}).$$

L3:

$$\theta_{new} = \theta - \eta \nabla_{\theta} \mathcal{L}(\hat{z}_{t+1}, z_{t+1}).$$

本题符号说明 其中, z_t 表示模型内部 latent state; $a_{t:t+H}$ 表示未来动作序列; a_t 表示当前动作或控制输入; H 表示未来规划或 rollout 的步数; $\mathcal{L}$ 表示训练损失; 带帽子的变量表示模型预测值; η 表示学习率。

12.2.3 总结

L1 Predictor 只预测未来, L2 Simulator 支持交互模拟, L3 Evolver 能在经验中更新和改进自身世界假设。三个层级对应从被动预测到闭环仿真再到自我演化的能力提升。

12.3 问题 113: world model 中的 law regime 是什么?

12.3.1 通俗解释

law regime 指世界模型必须遵守的规律体系。物理世界有力学和几何约束, 数字世界有代码和协议约束, 社会世界有互动和意图约束, 科学世界有方程和守恒约束。不同 regime 的失败模式不同——物理 regime 的错误是穿模、瞬移, 社会 regime 的错误是行为不合理, 科学 regime 的错误是违反守恒。Agentic 系统通常需要反思机制, 把失败轨迹转化为模型更新或策略修正, 避免在同一类规律上反复犯错。

12.3.2 数学推导

可把约束写成:

$$C_{regime}(z_t, a_t, z_{t+1}) \leq 0.$$

总损失包含预测误差和规律违背惩罚：

$$\mathcal{L} = \mathcal{L}_{pred} + \lambda \mathcal{L}_{law}.$$

本题符号说明 其中, z_t, z_{t+1} 表示相邻 latent state; a_t 表示动作; C_{regime} 表示该 regime 的约束函数; $\mathcal{L}_{pred}$ 表示常规预测损失; $\mathcal{L}_{law}$ 表示规律违背惩罚; λ 表示惩罚权重。

12.3.3 总结

law regime 指世界运行所遵循的规则体系, 例如物理规律、交通规则、游戏机制或社会约束。世界模型不仅要预测状态, 还要隐式或显式学习这些决定状态转移边界的规则。

12.4 问题 114: 什么是 action-conditioned imagination?

12.4.1 通俗解释

action-conditioned imagination 指在给定候选动作后想象未来, 把”未来会怎样”改写成”如果我这样做, 未来会怎样”。它是 planning、反事实推理和安全评估的基础——agent 可以先在内部把多个动作走一遍, 再选择代价最小、最满足任务的那个。在 agentic 系统中, 它常常和工具调用配合: 工具提供外部事实或能力, world model 负责预测每一步行动后果, 二者结合才能让决策可解释、可比较。

12.4.2 数学推导

给定 latent state z_t 和候选动作序列 $a_{t:t+H}$, action-conditioned imagination 递推未来:

$$\hat{z}_{t+k+1} \sim p_{\theta}(\hat{z}_{t+k+1} | \hat{z}_{t+k}, a_{t+k}), \quad k = 0, \dots, H-1.$$

最优动作通过最大化想象轨迹上的期望回报得到:

$$a_{t:t+H}^* = \arg \max_{a_{t:t+H}} \mathbb{E}_{p_{\theta}} \left[\sum_{k=0}^H \gamma^k r_{t+k} \right].$$

本题符号说明 其中, $\hat{z}_t$ 表示想象轨迹中的 latent state; $a_{t:t+H}$ 表示候选动作序列; p_{θ} 表示 latent dynamics; r_t 表示预测奖励; γ 是折扣因子; H 表示 rollout horizon。

12.4.3 总结

action-conditioned imagination 是在给定候选动作后想象未来。它把”未来会怎样”改成”如果我这样做, 未来会怎样”, 因此是 planning、反事实推理和安全评估的基础。

12.5 问题 115: 什么是 counterfactual simulation?

12.5.1 通俗解释

counterfactual simulation 是比较那些没有真实发生过的动作后果。它对 agent 很关键, 因为决策本质上是在多个未发生的未来之间做选择, 自动驾驶、机器人和策略评估都依赖这种能力。需要强调的是: 反事实能力的核心不是换一个 prompt 生成新视频, 而是在同一初始世界状态下只改变动作或对象变量, 并观察未来是否产生符合因果逻辑的变化。

12.5.2 数学推导

给定两个动作 a 和 a' :

$$\Delta R = \mathbb{E}_{p_\theta}[R | a] - \mathbb{E}_{p_\theta}[R | a'].$$

反事实模拟就是估计这个差异，而不必真实执行危险动作。

本题符号说明 其中， p_θ 表示参数为 θ 的学习式概率模型； $\mathbb{E}$ 表示期望。

12.5.3 总结

counterfactual simulation 是模拟没有真实发生的替代未来。它让 agent 能比较不同选择的后果，例如如果提前刹车、换另一条路线或选择不同抓取姿态，结果会不会更好。

12.6 问题 116: world model 如何支持 long-horizon planning?

12.6.1 通俗解释

long-horizon planning 要求世界模型在较长时间内保持动态、约束和目标的一致性。它的难点是误差累积、状态漂移和目标稀疏——短期预测漂亮，不代表长期规划可靠。在开放世界中尤其如此：模型必须知道自己不知道，在不确定时请求更多信息、调用工具或交由人类介入，而不是硬着头皮把错误向前传播。

12.6.2 数学推导

长时目标可写成：

$$a_{t:t+H}^* = \arg \max_{a_{t:t+H}} \mathbb{E} \left[\sum_{k=0}^H \gamma^k r_{t+k} \right].$$

rollout 误差会随 horizon 增大而累积：

$$\epsilon_H \lesssim \sum_{k=1}^H \epsilon_k,$$

其中 ϵ_k 表示第 k 步的局部预测误差。当 ϵ_k 不为零时，长时规划越往后越不可靠，因此实际系统通常配合 receding horizon 控制和 uncertainty penalty。

本题符号说明 其中， $a_{t:t+H}$ 表示未来动作序列； r_{t+k} 表示第 $t+k$ 步的奖励； $\mathbb{E}$ 表示期望； γ 是折扣因子； H 表示规划 horizon； ϵ_k 表示第 k 步的预测误差； ϵ_H 表示 H 步累积误差。

12.6.3 总结

world model 支持 long-horizon planning 的方式，是把短期状态转移递归展开，并在中间不断评估风险和目标进展。难点在于误差累积、分支爆炸和长期目标与短期动作之间的信用分配。

12.7 问题 117：如何评估 physical consistency?

12.7.1 通俗解释

physical consistency 评估的是模型生成或预测的未来是否遵守物理约束。它不关心画面“漂不漂亮”，而关心是否出现穿模、瞬移、无因加速、接触错位、重力违反、刚体形变、车辆运动违反动力学等问题。对世界模型来说，视觉逼真只是第一层判据，物理可信才决定它能否用于控制和仿真——一个像素级精美的世界，只要物体在画面里“飘起来”，就不能用作 planner 输入。

实际评估通常拆成五类约束：(1) **几何**——不可穿透、深度一致、视角变换可对齐；(2) **动力学**——刚体运动方程（速度、加速度上界、动量守恒）；(3) **接触**——接触法向受压、摩擦锥、不可拉接触；(4) **守恒**——质量、能量、电荷在闭系统中守恒；(5) **对象身份**——同一对象不应分裂、合并或漂移。每类约束都可写成 inequality 或 equality，用于自动检测违规帧并量化整体物理不一致度。

12.7.2 数学推导

设物理约束集合为 $\{v_m(z_t, a_t, z_{t+1}) \geq 0\}_{m=1}^M$ ，其中 $v_m \geq 0$ 表示满足。生成轨迹的违反程度通过 hinge 形式累加：

$$E_{\text{phys}} = \sum_{m=1}^M \lambda_m \mathbb{E}_{\tau \sim p_{\theta}} \left[\sum_t \max(0, -v_m(z_t, a_t, z_{t+1})) \right].$$

具体例子：

$$\begin{aligned} v_{\text{coll}} &= \text{dist}(z_t^{\text{ego}}, z_t^{\text{obs}}) - d_{\min}, & v_{\text{accel}} &= a_{\max} - \|\dot{v}_t\|, \\ v_{\text{contact}} &= F_{\text{normal}} \cdot n - 0, & v_{\text{conserve}} &= |\Delta E_t| - \epsilon. \end{aligned}$$

工程上还可以用 Pass Rate 这种通过率指标更直观：

$$\text{PR}_m = \Pr[v_m(z_t, a_t, z_{t+1}) \geq 0].$$

一个合格的 world model 应在每类约束上 PR 都接近 1，且 E_{phys} 不随 horizon 增长——后者是衡量长程物理稳定性的关键。

本题符号说明 其中， z_t 、 z_{t+1} 表示相邻状态； a_t 表示动作； v_m 表示第 m 个物理约束； E_{phys} 表示物理不一致度； λ_m 表示约束权重； dist 、 $d_{\min}$ 表示距离与最小允许距离； $\dot{v}_t$ 表示加速度； $a_{\max}$ 表示加速度上界； F_{normal} 、 n 表示接触法向力与法向方向； ΔE_t 表示能量变化； PR_m 表示第 m 项约束的通过率。

12.7.3 总结

physical consistency 的核心是检查未来是否符合几何、动力学、接触、守恒、对象身份五类约束。一个世界模型即使生成质量很高，只要物理规律经常被破坏，就不能作为可靠的仿真器或规划底座。指标设计的关键是按违规类型分项打分，并检验长 horizon 是否退化。

12.8 问题 118：如何评估 causal consistency?

12.8.1 通俗解释

causal consistency 评估的是模型是否正确响应干预。它不是看未来是否自然，而是看当我们改变动作、对象或环境变量时，未来是否发生符合因果逻辑的变化。例如：刹车应该导致速度下

降；推杯子应该导致杯子位移；改变前车轨迹应该影响自车风险；遮挡某物体不应让其他无关物体发生变化。如果模型只生成训练集里常见的画面，却不响应干预，那它本质是相关性生成器，而不是世界模型。

按 Pearl 因果阶梯，causal consistency 检验的是第二层（干预）和第三层（反事实）。一个模型可以在第一层（观察）上完美——预测出来的视频和真实视频分布一致，但仍然在第二层失败：例如训练数据里”刹车 → 减速”是高频共现，模型见过 80% 的”刹车-减速”轨迹后会反射式生成减速画面，但当我们干预非典型动作（在湿滑路面以特定角度刹车）时，模型可能仍然给出训练分布中的典型未来——这就是只学到相关性的标志。

12.8.2 数学推导

给定相同初始状态 z_t 和两个干预动作 a, a' ，模型给出两个未来分布：

$$p_\theta(z_{t+1} | z_t, \text{do}(a)), \quad p_\theta(z_{t+1} | z_t, \text{do}(a')).$$

模型预测的干预差异：

$$D^{\text{model}}(a, a') = d(p_\theta(z_{t+1} | z_t, \text{do}(a)), p_\theta(z_{t+1} | z_t, \text{do}(a'))).$$

真实环境的干预差异：

$$D^{\text{env}}(a, a') = d(p_{\text{env}}(z_{t+1} | z_t, \text{do}(a)), p_{\text{env}}(z_{t+1} | z_t, \text{do}(a'))).$$

因果一致性误差：

$$E_{\text{causal}} = \mathbb{E}_{(a, a') \sim \mathcal{P}_{\text{int}}} [|D^{\text{model}}(a, a') - D^{\text{env}}(a, a')|].$$

进一步可分层评估：

$$E_{\text{intra}} = \mathbb{E} \left[|D^{\text{model}} - D^{\text{env}}|_{a, a' \in \mathcal{D}_{\text{train}}} \right], \quad E_{\text{ood}} = \mathbb{E} \left[|D^{\text{model}} - D^{\text{env}}|_{a \notin \mathcal{D}_{\text{train}}} \right].$$

当 $E_{\text{intra}} \rightarrow 0$ 但 E_{ood} 大时，说明模型只在训练分布内学到了”看似因果”的相关性。这种分层评估是 OOD causal benchmark 的标准设计。

本题符号说明 其中， z_t, z_{t+1} 表示相邻状态； $\text{do}(a)$ 表示因果干预； a, a' 表示干预动作； p_θ 表示模型分布； p_{env} 表示真实环境分布； $D^{\text{model}}, D^{\text{env}}$ 表示模型与环境的干预差异； E_{causal} 表示总体因果一致性误差； $E_{\text{intra}}, E_{\text{ood}}$ 表示分布内与分布外误差； $\mathcal{P}_{\text{int}}, \mathcal{D}_{\text{train}}$ 表示干预动作分布与训练动作集合； $d(\cdot, \cdot)$ 表示分布距离。

12.8.3 总结

causal consistency 检验干预后未来是否按因果逻辑正确变化，这是世界模型与单纯相关性生成器的关键分水岭。它评估的是动作和世界变量之间的因果关系，而非生成结果是否符合训练分布。OOD 干预下的一致性误差通常远大于 in-distribution，是判断模型是否学到真因果的最严苛 benchmark。

12.9 问题 119：如何评估 action controllability?

12.9.1 通俗解释

action controllability 评估的是模型未来是否真的受动作控制。一个模型即使能生成非常真实的视频，如果加速、刹车、转向、抓取、推拉产生的未来几乎一样，它对规划就完全没用。一个合格的 world model 必须满足两个层次的要求：

(1) **敏感**——不同动作必须产生明显不同的未来，量化为非零的动作敏感度；(2) **合理**——这种敏感不能是杂乱无章的，必须符合真实动力学。模型若把”加速 1m/s^2 ”和”加速 10m/s^2 ”映射到几乎相同的未来，是欠敏感；若把”小幅刹车”想象成车直接消失或瞬移，是过敏感。

可控性是世界模型能否服务控制的最核心指标。它和 causal consistency 互补：causal consistency 检验干预的方向对不对，controllability 检验干预的强度对不对。两者都达标，模型才真正可被 planner 利用。

12.9.2 数学推导

直接的量化方式有两条。第一是**互信息**：

$$I(a_{t:t+H}; z_{t+1:t+H} | z_t),$$

当 $I \rightarrow 0$ 时模型忽略动作，当 I 充分大时未来由动作主导。第二是**动作敏感度**：

$$D_{\text{ctrl}}(a, a') = d(p_\theta(z_{t+1:t+H} | z_t, a), p_\theta(z_{t+1:t+H} | z_t, a')).$$

要进一步检验”敏感是否合理”，比较模型敏感度与真实环境敏感度的相关性：

$$\rho = \text{corr}(D_{\text{ctrl}}^{\text{model}}(a, a'), D_{\text{ctrl}}^{\text{env}}(a, a')),$$

当 $\rho \rightarrow 1$ 时，模型的动作敏感模式与真实世界一致； ρ 低则说明模型敏感但敏感得不对。还可以用动作-状态雅可比的范数检测过/欠敏感：

$$\left\| \frac{\partial \hat{z}_{t+1}}{\partial a_t} \right\| \quad \text{vs.} \quad \left\| \frac{\partial z_{t+1}^{\text{env}}}{\partial a_t} \right\|.$$

在驾驶/机器人 benchmark 中，常用 **Action Sweep**：固定 z_t ，对动作做扫频，观察 $\hat{z}_{t+1}$ 是否随动作平滑变化、单调性是否正确、极端动作是否保持稳定。

本题符号说明 其中， $a_{t:t+H}$ 表示未来动作序列； $z_t, z_{t+1:t+H}$ 表示当前与未来状态； $I(\cdot; \cdot | \cdot)$ 表示条件互信息； D_{ctrl} 表示动作敏感度； ρ 表示模型与环境敏感度的相关系数； $\partial \hat{z} / \partial a$ 表示动作-状态雅可比； $d(\cdot, \cdot)$ 表示分布距离； H 表示预测 horizon。

12.9.3 总结

action controllability 检查动作是否真正改变未来，并且改变是否合理。它是世界模型能否服务控制的最核心指标：未来既要随动作变化（敏感性），又要按物理和任务逻辑变化（合理性）。和 causal consistency 互补——前者检干预的强度，后者检干预的方向，两者都达标，模型才真正可被 planner 利用。

12.10 问题 120：世界模型会不会成为下一代 Agent 的核心基础设施？

12.10.1 通俗解释

下一代 Agent 需要的不只是 LLM 推理，还需要一个能模拟行动后果的内部环境。世界模型可以提供反事实评估、风险预测、长期计划和失败复盘，因此很可能成为 Agent 系统的核心基础设施。落到工程层面，真正要回答的不是”模型能不能生成世界”，而是”它生成的世界能不能让 agent 更安全、更高效地行动”。

12.10.2 数学推导

Agent 系统的整体决策可写成：

$$a_{1:H}^* = \arg \max_{a_{1:H}} \mathbb{E}_{p_\theta}[R(\tau)].$$

通常由 LLM 负责生成候选计划，由 world model 负责评估其后果：

$$a_{1:H} \sim \text{LLM}(l, c), \quad \tau \sim p_\theta(\tau \mid a_{1:H}).$$

在闭环系统中，模型预测、agent 行动、环境反馈和模型更新构成一个完整的“想象-行动-反馈-更新”训练目标，其中条件变量、预测对象和优化目标分别对应系统中的不同模块。

本题符号说明 其中， p_θ 表示 world model； τ 表示一整段轨迹； $R(\tau)$ 表示轨迹回报； $a_{1:H}$ 表示长度为 H 的动作序列； l 表示语言指令； c 表示任务上下文； $\mathbb{E}$ 表示期望。

12.10.3 总结

世界模型很可能成为下一代 Agent 的核心基础设施，因为 Agent 的瓶颈正在从“会不会说”转向“会不会预判行动后果”。未来强 Agent 需要的不只是 LLM planner，还需要可交互、可评估、可反事实推演的内部世界。

美国的牛粪博士